CFGS Desarrollo de Aplicaciones Multiplataforma

Programación de Servicios y Procesos

con Python

Módulo profesional 0490 · 66 horas

Segundo curso

Apuntes del módulo — versión 0.4.2 (borrador)
Fecha: 27/10/2026
Curso 2026–2027

Basado en el currículo del módulo para la Comunidad de Castilla y León.

Contenido generado con ayuda de inteligencia artificial: puede contener errores.

Presentación

Aviso sobre el contenido de estos apuntes. Los contenidos de este documento (explicaciones, ejemplos de código, ejercicios y proyectos) se han **generado con ayuda de herramientas de inteligencia artificial** y **pueden contener errores**, imprecisiones u omisiones. Los ejemplos de código pueden comportarse de forma distinta según la versión de Python, de las bibliotecas o del sistema operativo que utilices; en los programas concurrentes, además, el orden de los mensajes puede cambiar de una ejecución a otra.
Úsalos como material de apoyo, contrasta la información con la documentación oficial de cada herramienta (enlazada a lo largo del texto y recogida en el Anexo C) y, si encuentras un error, comunícaselo al profesor para corregirlo en las siguientes versiones.

Estos apuntes acompañan las clases del módulo *Programación de Servicios y Procesos* del segundo curso del ciclo de grado superior de Desarrollo de Aplicaciones Multiplataforma. El módulo trata de algo que cualquier aplicación profesional necesita tarde o temprano: **hacer varias cosas a la vez, comunicarse con otras aplicaciones a través de la red y hacerlo de forma segura**. Aprenderás a repartir el trabajo entre procesos e hilos, a sincronizarlos, a programar clientes y servidores con *sockets*, a construir servicios web y a proteger los datos con técnicas criptográficas.

A lo largo del curso trabajaremos con **Python**. Su biblioteca estándar trae todo lo necesario para el módulo (`multiprocessing`, `threading`, `socket`, `asyncio`, `ssl`…) y, desde la versión 3.14, una versión del intérprete **sin GIL** en la que los hilos se ejecutan de verdad en paralelo. Como no todo el mundo lo ha usado antes, el Tema 0 repasa los conceptos que necesitarás durante el curso.

Cómo están organizados los apuntes

Cada tema sigue la misma estructura:

- Un recuadro inicial con lo que aprenderás en el tema y el resultado de aprendizaje con el que se relaciona.
- Explicaciones teóricas acompañadas de **ejemplos de código completos** que puedes copiar y ejecutar.
- Recuadros de **Nota** con aclaraciones y ampliaciones, y de **Cuidado** con errores habituales.
- Recuadros de **Actualización del currículo** donde un contenido del currículo ha quedado anticuado y se explica qué se estudia en su lugar.
- **Ejercicios** cortos para practicar cada apartado, con una indicación de dificultad (★ a ★★★).
- Uno o varios **Proyectos** finales, más largos, para trabajar de forma autónoma.

Relación entre temas y resultados de aprendizaje

Tema	Contenido	RA
0	Repaso de Python	—
1	Programación multiproceso	RA1
2	Programación multihilo	RA2
3	Programación de comunicaciones en red	RA3
4	Generación de servicios en red	RA4
5	Utilización de técnicas de programación segura	RA5
A	Apéndice: preparación del entorno de trabajo	—
B	Anexo: pruebas y documentación del código (`pytest`, *docstrings*)	Todos
C	Anexo: bibliografía y recursos	—

Todos los resultados de aprendizaje terminan con el criterio *"se han depurado y documentado las aplicaciones desarrolladas"*: el Tema 0 explica cómo depurar y registrar lo que hace un programa con `logging`, y el Anexo B, cómo probarlo y documentarlo.

Convenciones

- El código Python aparece así:

```python
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"   # comentario
```

- Las órdenes que se escriben en la terminal, y lo que muestran los programas al ejecutarse, aparecen sobre fondo oscuro:

```
$ python3.15t -m venv .venv
$ pip install requests
```

- Los nombres de bibliotecas y módulos aparecen en este color y enlazan con su documentación oficial. La bibliografía completa está en el Anexo C.
- Todos los ejemplos están pensados para **Python 3.14 o superior**, preferiblemente en su versión sin GIL (`python3.15t`).

Ejecuta los programas desde una terminal. Muchos ejemplos de los temas 1 y 2 crean procesos e hilos que escriben por pantalla. Algunos entornos (los cuadernos Jupyter, ciertos botones de "ejecutar" de los editores) no muestran bien esa salida o directamente fallan con `multiprocessing`. Ante la duda, guarda el código en un fichero `.py` y ejecútalo con `python fichero.py`. Los ejemplos de red (temas 3 a 5) necesitan normalmente dos terminales: una para el servidor y otra para el cliente.

Software necesario

El Apéndice A explica cómo preparar el entorno de trabajo: Python con y sin GIL, un editor (Visual Studio Code o PyCharm), entornos virtuales, las bibliotecas del curso y las herramientas de red (`curl`, `netcat`, Wireshark), además de Docker para levantar un servidor de correo de pruebas en el Tema 4.

Índice

Presentación I

0. Repaso de Python 1

0.1. Primeros pasos . . . 1
0.2. Tipos de datos básicos . . . 4
0.3. Colecciones . . . 6
0.4. Control de flujo . . . 7
0.5. Comprensiones . . . 12
0.6. Funciones . . . 14
0.7. Anotaciones de tipo . . . 17
0.8. Decoradores . . . 21
0.9. Módulos y paquetes . . . 22
0.10. Excepciones . . . 23
0.11. Gestores de contexto: la sentencia `with` . . . 25
0.12. Programación orientada a objetos . . . 25
0.13. Iteradores y generadores . . . 34
0.14. Enumeraciones . . . 35
0.15. Texto y bytes . . . 35
0.16. Fechas y horas . . . 36
0.17. Argumentos de línea de comandos . . . 40
0.18. Registro de eventos y depuración . . . 45
0.19. Ejercicios . . . 50

1. Programación multiproceso 55

1.1. Ejecutables, procesos y servicios . . . 55
1.2. Estados de un proceso y planificación . . . 56
1.3. Hilos . . . 57
1.4. Programación concurrente, paralela y distribuida . . . 57
1.5. Gestión de procesos y herramientas de monitorización . . . 59
1.6. Lanzar programas externos: `subprocess` . . . 60
1.7. El módulo `multiprocessing` . . . 61
1.8. Comunicación entre procesos . . . 66
1.9. Obtener resultados: `Pool` y `ProcessPoolExecutor` . . . 72
1.10. Depuración y buenas prácticas . . . 73
1.11. Ejercicios . . . 74

2. Programación multihilo **76**
2.1. ¿Cuándo usar hilos? . . . 76
2.2. Contexto de ejecución de los hilos . . . 78
2.3. Crear, iniciar y finalizar hilos . . . 79
2.4. Estados de un hilo . . . 82
2.5. Prioridades de los hilos . . . 83
2.6. Compartir información entre hilos y sus problemas . . . 84
2.7. Sincronización de hilos . . . 85
2.8. Interbloqueos . . . 88
2.9. Conjuntos de hilos: `ThreadPoolExecutor` . . . 89
2.10. Depuración de programas multihilo . . . 90
2.11. Ejercicios . . . 90

3. Programación de comunicaciones en red **93**
3.1. Comunicación entre aplicaciones . . . 93
3.2. Repaso de redes . . . 94
3.3. Sockets . . . 94
3.4. Sockets TCP . . . 95
3.5. Sockets UDP . . . 98
3.6. Diseñar un protocolo de aplicación . . . 99
3.7. Servidores con varios clientes simultáneos . . . 99
3.8. Depuración de aplicaciones en red . . . 102
3.9. Ejercicios . . . 103

4. Generación de servicios en red **105**
4.1. Protocolos estándar de nivel de aplicación . . . 105
4.2. HTTP . . . 106
4.3. Clientes HTTP . . . 107
4.4. Programar un servidor HTTP: `http.server` . . . 108
4.5. Servicios web . . . 109
4.6. Diseño de una API REST . . . 110
4.7. FastAPI: primeros pasos . . . 115
4.8. Un proyecto completo con FastAPI . . . 117
4.9. Otro protocolo estándar: el correo electrónico . . . 126
4.10. Conexiones, eficiencia y disponibilidad . . . 127
4.11. Documentación y depuración de servicios . . . 128
4.12. Ejercicios . . . 129

5. Técnicas de programación segura **132**
5.1. Prácticas de programación segura . . . 132
5.2. Criptografía . . . 134

5.3. Política de seguridad y control de acceso . . . 138
5.4. Comunicaciones seguras . . . 141
5.5. Depuración y documentación de aplicaciones seguras . . . 143
5.6. Ejercicios . . . 144

A. Preparación del entorno de trabajo **146**
A.1. Python con y sin GIL . . . 146
A.2. El editor . . . 147
A.3. Bibliotecas del curso . . . 147
A.4. Herramientas de red . . . 148
A.5. Servidor de correo de pruebas con Docker . . . 148

B. Pruebas y documentación del código **150**
B.1. Por qué escribir pruebas automáticas . . . 150
B.2. De `assert` a las pruebas . . . 150
B.3. Un ejemplo completo . . . 150
B.4. Herramientas de `pytest` . . . 152
B.5. Probar código concurrente y de red . . . 155
B.6. Documentar el código . . . 157
B.7. Guía de estilo . . . 160
B.8. Ejercicios . . . 160

C. Bibliografía y recursos **161**
C.1. Documentación general . . . 161
C.2. Tema 0: Repaso de Python . . . 161
C.3. Tema 1: Programación multiproceso . . . 162
C.4. Tema 2: Programación multihilo . . . 162
C.5. Tema 3: Programación de comunicaciones en red . . . 162
C.6. Tema 4: Generación de servicios en red . . . 162
C.7. Tema 5: Técnicas de programación segura . . . 163
C.8. Anexo B: pruebas y documentación . . . 163
C.9. Especificaciones y estándares . . . 164
C.10. Libros . . . 164
C.11. Sitios de práctica . . . 165

TEMA 0

Repaso de Python

En este tema aprenderás a…

- Ejecutar programas Python, elegir la versión del intérprete (con o sin GIL) y gestionar dependencias con entornos virtuales y `pip`.
- Manejar los tipos de datos básicos y las colecciones (listas, tuplas, diccionarios y conjuntos).
- Usar las estructuras de control (`if`, `for`, `while`, `match`) y las comprensiones.
- Escribir funciones, pasarlas como argumento, anotar sus tipos y comprobarlos con herramientas de análisis estático.
- Organizar el código en módulos y paquetes.
- Tratar errores mediante excepciones, comprobar suposiciones con `assert` y usar gestores de contexto (`with`).
- Programar con clases, distinguiendo atributos de clase y de instancia y métodos de instancia, de clase y estáticos, y aplicando los convenios de visibilidad, herencia y propiedades.
- Elegir entre clases normales, *dataclasses* y modelos de Pydantic para representar datos.
- Entender iteradores, generadores y decoradores, que aparecerán en el resto de temas.
- Distinguir entre texto (`str`) y bytes, que es lo que viaja por la red, e intercambiar datos en JSON.
- Trabajar con fechas, horas y zonas horarias, y escribir programas de línea de comandos con `argparse`.
- Registrar lo que hace un programa con `logging` y depurarlo.

Este tema no pretende ser un curso completo de Python, sino un repaso de las piezas que usaremos constantemente en el módulo: funciones como objetos, gestores de contexto, clases y herencia, la diferencia entre texto y bytes, excepciones, decoradores y el registro de eventos. Si ya programas en Java o C#, verás que los conceptos son los mismos; lo que cambia es la sintaxis y algunas costumbres del lenguaje.

0.1 Primeros pasos

0.1.1 El intérprete y los scripts

Python es un lenguaje **interpretado** y de **tipado dinámico**: no hay fase de compilación explícita y las variables no se declaran con un tipo. Hay dos formas de ejecutar código:

- En el **intérprete interactivo** (REPL), escribiendo `python` en la terminal. Es muy útil para probar cosas rápidamente.
- Guardando el código en un fichero `.py` y ejecutándolo con `python fichero.py`.

```
$ python
>>> 2 + 3
5
>>> nombre = "Ada"
>>> f"Hola, {nombre}"
'Hola, Ada'
>>> exit()
```

En Python **la indentación forma parte de la sintaxis**: los bloques no se delimitan con llaves sino con sangría (por convenio, cuatro espacios), y la línea que abre un bloque termina en dos puntos.

```
edad = 19
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

0.1.2 La versión de Python del curso: con y sin GIL

En este módulo vamos a programar con procesos e hilos, y eso hace que la versión de Python importe más de lo habitual. Durante treinta años, el intérprete oficial (CPython) ha tenido un cerrojo global, el **GIL** (*Global Interpreter Lock*), que impide que dos hilos de un mismo programa ejecuten código Python a la vez. Lo estudiaremos en detalle en los temas 1 y 2; por ahora basta con saber que, con el GIL, los hilos *no* aprovechan varios núcleos para hacer cálculos.

Desde Python 3.13 existe una segunda versión del intérprete, la **compilación sin GIL** (*free-threaded build*), en la que los hilos sí se ejecutan en paralelo:

Versión	Fecha	Estado de la compilación sin GIL
3.13	octubre 2024	Experimental (PEP 703).
3.14	octubre 2025	Soportada oficialmente, aunque opcional (PEP 779).
3.15	octubre 2026	Soportada; el instalador oficial de macOS ya la instala por defecto. Es la versión más reciente.

La compilación sin GIL se instala **junto a** la normal, no en su lugar, y su ejecutable lleva una `t` al final: `python3.15t`, `python3.14t`. En el curso usaremos **Python 3.15t** (o, si no está disponible en tu sistema, 3.14t). Si en algún momento una librería no funciona con ella, puedes recurrir a la versión normal de la misma versión de Python.

El Apéndice A explica cómo instalar las dos versiones en Windows, macOS y Linux. La forma más sencilla, igual en todos los sistemas, es la herramienta `uv`:

```
$ uv python install 3.15t          # Python 3.15 sin GIL
$ uv python install 3.15           # (opcional) la versión normal
$ uv venv --python 3.15t           # entorno virtual del proyecto, sin GIL
```

Comprobar que el GIL está desactivado

```
$ python -VV
Python 3.15.0 free-threading build (main, Oct  9 2026, ...)
```

Desde un programa, `sys` y `sysconfig` nos dan la información:

```python
import sys
import sysconfig

# ¿Este intérprete se compiló sin GIL?
print("Compilación sin GIL:", sysconfig.get_config_var("Py_GIL_DISABLED") == 1)
# ¿Está el GIL desactivado AHORA, en este proceso?
print("GIL activo:", sys._is_gil_enabled())
```

Lo que debes saber de la versión sin GIL.

- Algunas librerías con partes escritas en C aún no están preparadas. Si importas una de ellas, el intérprete **vuelve a activar el GIL** y muestra un aviso. Puedes forzarlo desactivado con la variable de entorno `PYTHON_GIL=0` (o `python -X gil=0`), bajo tu responsabilidad. Hay una lista de compatibilidad en `https://py-free-threading.github.io/tracking/`.
- Los programas con un solo hilo van algo más lentos (en torno a un 1–8 %) y consumen algo más de memoria.
- Quitar el GIL **no hace los programas más seguros**, al contrario: los hilos se intercalan con más libertad y las condiciones de carrera del Tema 2 aparecen con más facilidad. Todo lo que aprenderemos sobre sincronización sigue siendo imprescindible.

0.1.3 Entornos virtuales y paquetes

Durante el curso instalaremos bibliotecas de terceros (`psutil`, `requests`, FastAPI, `cryptography`...). Para que cada proyecto tenga sus propias dependencias y no interfieran entre sí, se usa un **entorno virtual**: una carpeta con una copia aislada del intérprete y sus paquetes.

```
$ python3.15t -m venv .venv          # crea el entorno en la carpeta .venv
$ source .venv/bin/activate          # Linux / macOS
> .venv\Scripts\activate             # Windows
(.venv) $ python -VV                 # comprueba la versión del entorno
(.venv) $ pip install psutil         # instala un paquete
(.venv) $ pip freeze > requirements.txt   # guarda las dependencias
(.venv) $ pip install -r requirements.txt # las reinstala en otro equipo
```

Nota. Dentro de un entorno activado, `python` y `pip` se refieren siempre a la versión con la que se creó el entorno. Existen herramientas más modernas que combinan la instalación de Python, los entornos virtuales y la gestión de dependencias, como `uv` (la recomendada en este curso, Apéndice A) o `poetry`. Con `uv`, el entorno anterior se crea con `uv venv --python 3.15t` y los paquetes se instalan con `uv pip install psutil`.

0.2 Tipos de datos básicos

Tipo	Ejemplo	Comentario
`int`	`42`, `-7`, `1_000_000`	Enteros de precisión ilimitada.
`float`	`3.14`, `1e-3`	Coma flotante de doble precisión.
`bool`	`True`, `False`	Subtipo de `int`.
`str`	`"hola"`, `'hola'`	Cadenas Unicode inmutables.
`bytes`	`b"\x00\xff"`	Secuencias inmutables de bytes: lo que viaja por la red (apartado 0.15).
`NoneType`	`None`	Ausencia de valor (equivale a `null`).

```
x = 10            # int
x = "diez"        # ahora es str: el tipo lo tiene el valor, no la variable
print(type(x))    # <class 'str'>
print(7 / 2)      # 3.5  (división real)
print(7 // 2)     # 3    (división entera)
print(7 % 2)      # 1    (resto)
print(2 ** 10)    # 1024 (potencia)
```

0.2.1 Cadenas de caracteres

Las cadenas son inmutables y admiten índices (empezando en 0), índices negativos (contando desde el final) y *slicing* `[inicio:fin:paso]`.

```
s = "Servicios y Procesos"
print(s[0], s[-1])        # S s
print(s[0:9])             # Servicios
print(s.upper())          # SERVICIOS Y PROCESOS
print(s.split(" "))       # ['Servicios', 'y', 'Procesos']
print("-".join(["a", "b", "c"]))   # a-b-c
print("  texto  ".strip())         # 'texto'
print(s.replace("Procesos", "Hilos"))   # Servicios y Hilos
print("Procesos" in s)    # True
```

Estos métodos se usan constantemente para interpretar los mensajes de los protocolos de red, que suelen ser líneas de texto con campos separados por espacios:

```
linea = "  GET /index.html HTTP/1.1  \r\n"
linea = linea.strip()                 # quita espacios y saltos de línea
metodo, ruta, version = linea.split() # trocea por espacios
print(metodo.lower(), ruta.startswith("/"), "HTTP" in version)
print(",".join(["a", "b", "c"]))      # a,b,c
print(linea[0:3], linea[-8:])         # slicing: 'GET' 'HTTP/1.1'
```

Las **f-strings** son la forma recomendada de construir cadenas con valores:

```
nombre, nota = "Lucía", 8.456
print(f"{nombre} ha sacado un {nota:.2f}")   # Lucía ha sacado un 8.46
print(f"{nombre:>10}|")                       # alineado a la derecha en 10 posiciones
print(f"{nota=}")                             # nota=8.456 (útil para depurar)
```

Resumen de modificadores de formato

Dentro de las llaves de una f-string puede ir cualquier expresión de Python (`f"{x * 2}"`, `f"{nombre.upper()}"`), seguida opcionalmente de una **conversión** (tras `!`) y de una **especificación de formato** (tras `:`):

```
{expresión!conversión:formato}

formato = [[relleno]alineación][signo][#][0][ancho][agrupación][.precisión][tipo]
```

Todas las partes son opcionales, pero si aparecen deben ir en ese orden. Los resultados de la tabla se han obtenido con `nombre = "Lucía"`, `nota = 8.456`, `x = 42`, `n = 1234567.891`, `r = 0.256` y `hoy = date(2026, 10, 19)`; el símbolo ␣ representa un espacio.

Modificador	Ejemplo	Resultado
Precisión y tipo		
Decimales: `.N` con el tipo `f`	`f"{nota:.2f}"`	`8.46`
Notación científica: `e`	`f"{n:.3e}"`	`1.235e+06`
Porcentaje (multiplica por 100): `%`	`f"{r:.1%}"`	`25.6%`
Entero: `d`	`f"{x:d}"`	`42`
Binario, octal, hexadecimal: `b`, `o`, `x`, `X`	`f"{x:b} {x:x}"`	`101010 2a`
Con prefijo de base: `#`	`f"{x:#x}"`	`0x2a`
Ancho, alineación y relleno		
Ancho mínimo (los números se alinean a la derecha)	`f"{x:5d}|"`	␣␣␣42|
Relleno con ceros: `0` delante del ancho	`f"{x:05d}"`	`00042`
Alinear a la izquierda: `<`	`f"{nombre:<10}|"`	Lucía␣␣␣␣␣|
Alinear a la derecha: `>`	`f"{nombre:>10}|"`	␣␣␣␣␣Lucía|
Centrar: `^`	`f"{nombre:^10}|"`	␣␣Lucía␣␣␣|
Carácter de relleno (antes de la alineación)	`f"{nombre:*^11}"`	`***Lucía***`
Ancho tomado de una variable	`f"{nota:{ancho}.2f}"`	␣␣␣␣8.46
Signo y separador de miles		
Mostrar siempre el signo: `+`	`f"{x:+d}"`	`+42`
Separador de miles: `,` o `_`	`f"{n:,.2f}"`	`1,234,567.89`
Otros		
Fechas: códigos de `strftime`	`f"{hoy:%d/%m/%Y}"`	`19/10/2026`
Depuración: `=` muestra la expresión y su valor	`f"{nota = :.1f}"`	`nota = 8.5`
Llaves literales: se duplican	`f"{{x}} vale {x}"`	`{x} vale 42`

Conversiones: `!r` y `!s`. Antes de aplicar el formato, la f-string convierte el valor en texto. Por defecto usa `str()`, la representación pensada para el usuario, pero con `!r` se usa `repr()`, la pensada para el programador (son los métodos `__str__` y `__repr__` que veremos en el apartado de clases). La diferencia se aprecia bien con una fecha, y `!r` es especialmente útil en mensajes de error, porque las comillas dejan ver espacios sobrantes o si el valor es una cadena:

```
hoy = date(2026, 10, 19)
print(f"{hoy}")        # 2026-10-19                  (str, por defecto)
print(f"{hoy!r}")      # datetime.date(2026, 10, 19)  (repr)

nombre = "Ana "
```

```
print(f"No existe el alumno {nombre!r}")   # No existe el alumno 'Ana '
```

El especificador `=` de depuración usa `repr()` automáticamente (`f"{nombre=}"` muestra `nombre='Ana '`). Existe también `!a`, que usa `ascii()`, pero apenas tiene uso práctico.

> **Nota.** El separador de miles y el punto decimal siguen siempre el formato inglés (`1,234,567.89`). Para obtener el formato español (`1.234.567,89`) se puede usar el módulo `locale` o, de forma sencilla, intercambiar los caracteres: `f"{n:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")`.

0.3 Colecciones

0.3.1 Listas

Secuencias **mutables** y ordenadas. Pueden contener elementos de cualquier tipo.

```
modulos = ["Acceso a Datos", "PSP", "DI"]
modulos.append("PMDM")          # añade al final
modulos.insert(0, "SGE")        # inserta en una posición
modulos.remove("DI")            # elimina por valor
ultimo = modulos.pop()          # extrae el último
print(len(modulos), modulos)
print(sorted(modulos))          # nueva lista ordenada
modulos.sort(key=len)           # ordena en el sitio por longitud
```

0.3.2 Tuplas

Secuencias **inmutables**. Se usan para agrupar valores relacionados, por ejemplo una dirección de red formada por el servidor y el puerto: `("localhost", 5000)`.

```
direccion = ("192.168.1.10", 5000)
host, puerto = direccion        # desempaquetado
print(puerto)
```

> **La tupla de un solo elemento.** Para crear una tupla de un elemento hace falta una **coma**: `(5,)`. Sin ella, `(5)` es simplemente el número 5 entre paréntesis. Este detalle es fuente de errores constante en los temas 1 y 2, porque los argumentos de procesos e hilos se pasan como tupla: `Thread(target=f, args=(5,))`.

0.3.3 Diccionarios

Colecciones de pares *clave: valor*. Son, con diferencia, la estructura más utilizada en Python y la que encaja de forma natural con JSON y con los mensajes que intercambian clientes y servidores en los temas 3 y 4.

```
alumno = {"nombre": "Ana", "edad": 20, "modulos": ["AD", "PSP"]}
print(alumno["nombre"])
print(alumno.get("email", "sin email"))   # valor por defecto si no existe
alumno["email"] = "ana@ejemplo.es"         # añadir o modificar
```

```python
del alumno["edad"]
for clave, valor in alumno.items():
    print(clave, "->", valor)
```

0.3.4 Conjuntos

Colecciones sin orden y **sin elementos repetidos**.

```python
a = {"python", "java", "sql"}
b = {"sql", "xml"}
print(a | b, a & b, a - b)     # unión, intersección, diferencia
```

0.4 Control de flujo

0.4.1 Condicionales: `if`, `elif` y `else`

La sentencia `if` ejecuta un bloque sólo si se cumple una condición. Se pueden encadenar tantas alternativas `elif` (*else if*) como se quiera y, opcionalmente, un `else` final para el resto de casos. Python evalúa las condiciones en orden y ejecuta **sólo el primer bloque** cuya condición sea verdadera.

```python
nota = 7
if nota >= 9:
    calificacion = "Sobresaliente"
elif nota >= 7:
    calificacion = "Notable"          # se ejecuta este y no se mira nada más
elif nota >= 5:
    calificacion = "Aprobado"
else:
    calificacion = "Suspenso"
```

Algunos detalles útiles de las condiciones en Python:

- **Operadores lógicos**: se escriben con palabras: `and`, `or` y `not`.
- **Comparaciones encadenadas**: `0 <= nota <= 10` equivale a `0 <= nota and nota <= 10`.
- **Pertenencia**: `in` y `not in` comprueban si un elemento está en una cadena, lista, diccionario (sus claves) o conjunto.
- **Comparación con `None`**: se usa `is None` / `is not None`, no `== None`.
- **Valores "verdaderos" y "falsos"**: cualquier valor puede usarse como condición. Se consideran falsos `False`, `None`, el cero (`0`, `0.0`) y las colecciones vacías (`""`, `[]`, `{}`, `set()`); todo lo demás es verdadero. Así, `if lista:` significa "si la lista no está vacía".

```python
modulos = ["AD", "PSP"]
if "AD" in modulos and "DI" not in modulos:
    print("Matriculado en AD pero no en DI")

email = None
if email is None:
    print("Sin email")
```

```
if not modulos:                        # lista vacía
    print("No hay módulos")
```

Para elegir entre dos valores en una sola línea existe la **expresión condicional** (el equivalente al operador ternario `? :` de Java):

```
resultado = "Aprobado" if nota >= 5 else "Suspenso"
```

0.4.2 El bucle `for`

En Python, `for` no cuenta de un número a otro como en Java o C, sino que **recorre los elementos de un iterable**: una lista, una cadena, un diccionario, un fichero, las líneas que llegan por un *socket*... En cada vuelta, la variable del bucle toma el valor del siguiente elemento.

```
for modulo in ["AD", "PSP", "DI"]:
    print(modulo)

for letra in "Hola":                   # una cadena es una secuencia de caracteres
    print(letra)

notas = {"Ana": 8, "Luis": 4}
for nombre in notas:                   # un diccionario se recorre por sus claves
    print(nombre)
for nombre, nota in notas.items():     # claves y valores a la vez
    print(nombre, nota)
```

`range`: secuencias de números

Cuando sí se necesita repetir algo un número de veces o recorrer posiciones, se usa `range`, que genera números enteros **sin incluir el final**:

Llamada	Números que genera
`range(5)`	`0, 1, 2, 3, 4`
`range(1, 6)`	`1, 2, 3, 4, 5`
`range(0, 10, 2)`	`0, 2, 4, 6, 8`
`range(10, 0, -3)`	`10, 7, 4, 1`

`range` no crea una lista en memoria: va generando los números a medida que el bucle los pide, por lo que `range(1_000_000)` no ocupa más que `range(5)`. Si se necesita la lista, se convierte con `list(range(5))`.

`enumerate`: elemento y posición

Si además del elemento se necesita su posición, no hay que recurrir a `range(len(lista))`: `enumerate` devuelve en cada vuelta una tupla `(posición, elemento)`, que se desempaqueta en dos variables. El parámetro `start` indica desde qué número contar.

```
modulos = ["AD", "PSP", "DI"]
```

```python
for i, modulo in enumerate(modulos, start=1):
    print(f"{i}. {modulo}")           # 1. AD   2. PSP   3. DI

# Equivalente, pero menos claro y más propenso a errores:
for i in range(len(modulos)):
    print(f"{i + 1}. {modulos[i]}")
```

`zip`: recorrer varias secuencias a la vez

`zip` combina varias secuencias elemento a elemento: en cada vuelta devuelve una tupla con el primer elemento de cada una, luego el segundo, etc. Se detiene al acabarse la secuencia más corta; con `strict=True` lanza un `ValueError` si no tienen la misma longitud, lo que evita perder datos sin darse cuenta.

```python
nombres = ["Ana", "Luis", "Eva"]
notas = [8, 4, 6]
for nombre, nota in zip(nombres, notas):
    print(nombre, nota)               # Ana 8 / Luis 4 / Eva 6

print(dict(zip(nombres, notas)))      # {'Ana': 8, 'Luis': 4, 'Eva': 6}
list(zip(nombres, [8, 4], strict=True))   # ValueError: zip() argument 2 is shorter...
```

`break`, `continue` y `else`

- `break` sale del bucle inmediatamente.
- `continue` salta a la siguiente vuelta sin ejecutar el resto del bloque.
- Un bucle puede llevar un bloque `else`, que se ejecuta cuando el bucle **termina normalmente**, es decir, cuando se recorren todos los elementos **sin salir por un `break`**.

El `else` es útil en las búsquedas: permite saber si el bucle encontró lo que buscaba sin necesidad de una variable auxiliar.

```python
alumnos = ["Ana", "Luis", "Eva"]
buscado = "Marta"

for alumno in alumnos:
    if alumno == buscado:
        print("Encontrado")
        break                         # se sale del bucle: NO se ejecuta el else
else:
    print(f"{buscado} no está en la lista")   # sólo si no hubo break

for n in range(10):
    if n % 2 == 0:
        continue                      # se salta los pares
    print(n)                          # 1 3 5 7 9
```

> **Nota.** El nombre `else` resulta poco intuitivo en un bucle. Puedes leerlo como *"si no se salió con break"*.

0.4.3 El bucle `while`

`while` repite un bloque **mientras se cumpla una condición**, que se comprueba antes de cada vuelta. Se usa cuando no se sabe de antemano cuántas vueltas habrá: leer datos hasta que se acaben, reintentar una conexión, esperar una respuesta válida del usuario... Hay que asegurarse de que algo dentro del bucle acabe haciendo falsa la condición; si no, el bucle no termina nunca.

```
intentos = 0
while intentos < 3:
    intentos += 1
    print("Intento", intentos)
```

Admite también `break`, `continue` y `else` (que se ejecuta cuando la condición deja de cumplirse sin haber salido por `break`). Un patrón muy habitual es el bucle "infinito" con `while True` del que se sale con `break` cuando se cumple cierta condición:

```
while True:
    texto = input("Edad: ")
    if texto.isdigit() and 0 < int(texto) < 120:
        edad = int(texto)
        break                          # dato válido: salimos del bucle
    print("Edad no válida, inténtalo de nuevo")
```

0.4.4 `match`: coincidencia de patrones

La sentencia `match` (Python 3.10) compara un valor con una serie de **patrones** y ejecuta el bloque del primero que coincida. En su forma más sencilla sustituye a una cadena de `if/elif` que compara con valores concretos, como el `switch` de Java:

```
opcion = input("Opción: ")
match opcion:
    case "1":
        print("Alta")
    case "2" | "3":                    # varios valores con |
        print("Modificación o baja")
    case "q" | "Q":
        print("Salir")
    case _:                            # comodín: cualquier otro valor
        print("Opción no válida")
```

A diferencia del `switch` de otros lenguajes, no hace falta `break`: sólo se ejecuta un caso. Pero `match` es mucho más potente que un `switch`, porque los patrones pueden describir la **estructura** del valor y **capturar** partes de él en variables:

```
def procesar(orden: list[str]) -> str:
    match orden:
        case ["listar"]:
            return "Listado completo"
        case ["buscar", texto]:                # lista de 2 elementos: captura el 2.º
            return f"Buscando {texto!r}"
        case ["borrar", *ids] if ids:          # 1 o más elementos tras "borrar" (
    ↪ guarda)
            return f"Borrando {len(ids)} registros"
        case _:
```

```
            return "Orden no reconocida"

print(procesar("buscar Ana".split()))           # Buscando 'Ana'
print(procesar("borrar 3 7 9".split()))         # Borrando 3 registros

def describir(evento: dict) -> str:
    match evento:
        case {"tipo": "alta", "usuario": nombre}:    # diccionario con esas claves
            return f"Nuevo usuario: {nombre}"
        case {"tipo": "error", "codigo": int(c)} if c >= 500:
            return f"Error del servidor ({c})"
        case _:
            return "Evento desconocido"

print(describir({"tipo": "error", "codigo": 503}))  # Error del servidor (503)
```

Patrón	Coincide con
`"1"`, `42`, `None`	Ese valor concreto (literal).
`"a" | "b"`	Cualquiera de las alternativas.
`nombre`	Cualquier valor, que se guarda en la variable `nombre`.
`_`	Cualquier valor, sin guardarlo (comodín).
`[a, b]`, `[a, *resto]`	Una secuencia con esa forma; `*resto` recoge los elementos sobrantes.
`{"clave": valor}`	Un diccionario que tenga (al menos) esas claves.
`int(x)`, `Punto(x=0)`	Un objeto de ese tipo (con esos atributos).
`patrón if condición`	El patrón, si además se cumple la condición (*guarda*).

Este tipo de patrones resulta muy cómodo para interpretar las órdenes que recibe un servidor (Tema 3) o los mensajes JSON de una API (Tema 4).

0.4.5 El operador morsa (:=)

El operador `:=`, llamado **operador morsa** (*walrus operator*) por su parecido con los ojos y colmillos de una morsa, se introdujo en Python 3.8. Es una **expresión de asignación**: asigna un valor a una variable y, a la vez, **devuelve** ese valor, de modo que se puede usar dentro de otra expresión (una condición, un `while`, una comprensión...). El `=` normal es una *sentencia*: asigna, pero no devuelve nada y no puede ir dentro de una condición.

Su uso más habitual es en los bucles que leen datos hasta que se acaban. Compara las dos versiones:

```
# Sin operador morsa: hay que repetir la llamada a input()
nombres = []
nombre = input("Nombre (vacío para terminar): ")
while nombre != "":
    nombres.append(nombre)
    nombre = input("Nombre (vacío para terminar): ")

# Con operador morsa: se lee y se comprueba en la misma línea
nombres = []
while (nombre := input("Nombre (vacío para terminar): ")) != "":
```

```
    nombres.append(nombre)
```

Es exactamente el patrón que usaremos en el Tema 3 para recibir datos por un *socket* hasta que el otro extremo cierra la conexión, y en el Tema 1 para leer un fichero por bloques: `recv()` y `read()` devuelven un bloque vacío (que se evalúa como falso) cuando ya no hay más datos.

```
while datos := conexion.recv(1024):        # se detiene cuando datos == b""
    conexion.sendall(datos.upper())

with open("foto.jpg", "rb") as f:
    while bloque := f.read(64 * 1024):     # se detiene al llegar al final
        resumen.update(bloque)
```

También evita llamar dos veces a una función cuyo resultado se comprueba y luego se usa (las comprensiones del segundo ejemplo se explican en el apartado siguiente):

```
agenda = {"Ana": "600111222", "Luis": "611222333"}
if (telefono := agenda.get("Ana")) is not None:
    print("Llamando al", telefono)

# En una comprensión: calcular el precio con IVA una sola vez por elemento
precios = [10.0, 25.5, 8.0]
caros = [con_iva for p in precios if (con_iva := round(p * 1.21, 2)) > 15]
print(caros)           # [30.86]
```

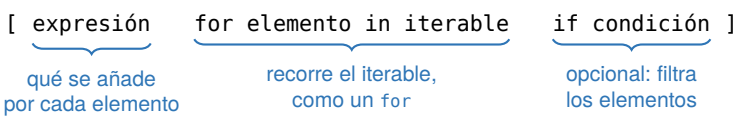
Cuidado.

- **Paréntesis**: `:=` tiene muy poca prioridad, así que casi siempre hay que rodear la asignación con paréntesis. `if n := len(nombres) > 1:` asigna a `n` el resultado de la comparación (`True`), no la longitud; lo correcto es `if (n := len(nombres)) > 1:`.
- No se puede usar como sentencia suelta: `x := 5` en una línea da `SyntaxError`. Para asignar sin más, se usa `=`.
- Úsalo sólo cuando haga el código más claro (bucles de lectura, evitar repetir una llamada). Encadenar varias asignaciones dentro de una misma expresión lo hace difícil de leer.

0.5 Comprensiones

Una **comprensión** construye una lista, un diccionario o un conjunto a partir de otro iterable en una sola expresión. Es una forma compacta, y muy habitual en Python, de escribir un bucle `for` cuyo único propósito es llenar una colección.

0.5.1 Sintaxis



Una comprensión equivale exactamente a este bucle:

```python
# Comprensión
cuadrados_pares = [n * n for n in range(10) if n % 2 == 0]

# Bucle equivalente
cuadrados_pares = []
for n in range(10):                # for elemento in iterable
    if n % 2 == 0:                 # if condición
        cuadrados_pares.append(n * n)    # expresión
print(cuadrados_pares)             # [0, 4, 16, 36, 64]
```

Se lee casi como una frase: "*el cuadrado de n, para cada n de 0 a 9, si n es par*".

0.5.2 Tipos de comprensiones

Según los delimitadores, se obtiene una colección u otra:

Tipo	Sintaxis	Resultado
Lista	`[expr for x in it]`	`list` (con orden y repetidos)
Conjunto	`{expr for x in it}`	`set` (sin repetidos)
Diccionario	`{clave: valor for x in it}`	`dict`
Generador	`(expr for x in it)`	Un generador: produce los valores de uno en uno, sin crear la colección en memoria (ver Iteradores y generadores).

```python
palabras = ["Python", "java", "SQL", "python", "XML"]

minusculas = [p.lower() for p in palabras]
# ['python', 'java', 'sql', 'python', 'xml']

distintas = {p.lower() for p in palabras}
# {'java', 'python', 'sql', 'xml'}  (sin repetidos; el orden puede variar)

longitudes = {p: len(p) for p in palabras}
# {'Python': 6, 'java': 4, 'SQL': 3, 'python': 6, 'XML': 3}

notas = {"Ana": 8, "Luis": 4, "Eva": 6}
aprobados = {nombre: nota for nombre, nota in notas.items() if nota >= 5}
# {'Ana': 8, 'Eva': 6}

total = sum(len(p) for p in palabras)    # generador: no crea ninguna lista
# 22
```

0.5.3 Condiciones: filtrar o transformar

Hay que distinguir dos usos del `if`, que se escriben en lugares distintos:

- **Al final**, `if condición` **filtra**: los elementos que no la cumplen no aparecen en el resultado.
- **En la expresión**, `valor1 if condición else valor2` es una expresión condicional que **transforma**: todos los elementos aparecen, pero con un valor u otro. En este caso el `else` es obligatorio.

```
notas = [8, 4, 6, 3]
[n for n in notas if n >= 5]                          # [8, 6]  -> filtra
["Apto" if n >= 5 else "No apto" for n in notas]      # ['Apto', 'No apto', 'Apto', 'No
    ↪ apto']
```

0.5.4 Varios `for`

Una comprensión puede tener varios `for`, que se leen **en el mismo orden que si fueran bucles anidados**:

```
grupos = [["Ana", "Luis"], ["Eva"], ["Marta", "Iker"]]
alumnos = [alumno for grupo in grupos for alumno in grupo]
# ['Ana', 'Luis', 'Eva', 'Marta', 'Iker']

# Equivale a:
alumnos = []
for grupo in grupos:
    for alumno in grupo:
        alumnos.append(alumno)

pares = [(x, y) for x in "AB" for y in range(2)]
# [('A', 0), ('A', 1), ('B', 0), ('B', 1)]
```

Cuándo no usarlas. Las comprensiones son ideales para transformaciones y filtros sencillos. Si la expresión se vuelve larga, hay más de dos `for` o hace falta más de una condición compleja, un bucle normal es más legible. Tampoco deben usarse por sus efectos secundarios (por ejemplo, `[print(x) for x in lista]`): si no se necesita la colección resultante, se usa un `for`.

0.6 Funciones

Una función se define con `def`, seguida del nombre, los parámetros entre paréntesis y dos puntos. Con `return` devuelve un valor; si no hay `return`, devuelve `None`. La cadena entre triples comillas al principio es la documentación de la función (*docstring*), que muestran los editores y `help(media)`.

```
def media(valores: list[float], decimales: int = 2) -> float:
    """Devuelve la media aritmética redondeada."""
    if not valores:
        raise ValueError("La lista está vacía")
    return round(sum(valores) / len(valores), decimales)

print(media([7, 8, 9]))
print(media([7, 8, 9], decimales=0))   # argumento por nombre
```

Las anotaciones `list[float]` y `-> float` indican los tipos de los parámetros y del resultado; las veremos con detalle en el apartado siguiente.

0.6.1 Argumentos posicionales, por nombre y valores por defecto

Al llamar a una función, los argumentos se pueden pasar de dos formas, que se pueden combinar (primero los posicionales):

- **Posicionales**: se asignan a los parámetros por orden. `media([7, 8], 1)`.
- **Por nombre** (*keyword arguments*): se indica el nombre del parámetro, y el orden da igual. `media(decimales=1, valores=[7, 8])`. Hacen las llamadas más legibles, sobre todo con valores booleanos o numéricos.

Un parámetro con **valor por defecto** (`decimales: int = 2`) es opcional en la llamada. Los parámetros con valor por defecto deben ir después de los que no lo tienen.

> **Cuidado.** Nunca uses una lista o un diccionario como valor por defecto (`def f(datos=[])`): el valor por defecto se crea una sola vez y lo comparten todas las llamadas. Usa `None` y crea la colección dentro: `if datos is None: datos = []`. Volveremos sobre este problema en el apartado de *dataclasses*.

Una función puede devolver varios valores separándolos por comas; en realidad devuelve una tupla, que se puede desempaquetar:

```
def minimo_maximo(valores: list[float]) -> tuple[float, float]:
    return min(valores), max(valores)

menor, mayor = minimo_maximo([7, 3, 9])     # menor = 3, mayor = 9
```

0.6.2 Número variable de argumentos: `*args` y `**kwargs`

A veces una función debe aceptar **cualquier número de argumentos**, como hace `print`, que admite uno, dos o veinte. Para ello existen dos parámetros especiales:

- `*args`: recoge en una **tupla** todos los argumentos **posicionales** que sobran.
- `**kwargs`: recoge en un **diccionario** todos los argumentos **por nombre** que no corresponden a ningún parámetro (las claves son los nombres y los valores, los valores).

Lo que importa son los asteriscos; los nombres `args` y `kwargs` (*keyword arguments*) son sólo un convenio y se pueden cambiar por otros más descriptivos:

```
def sumar(*numeros: float) -> float:
    print(numeros)                    # una tupla
    return sum(numeros)

sumar()                               # ()          -> 0
sumar(4)                              # (4,)        -> 4
sumar(1, 2, 3)                        # (1, 2, 3)   -> 6

def crear_alumno(nombre: str, **datos) -> dict:
    print(datos)                      # un diccionario
    return {"nombre": nombre, **datos}

crear_alumno("Ana", edad=20, ciclo="DAM")
# {'edad': 20, 'ciclo': 'DAM'}  -> {'nombre': 'Ana', 'edad': 20, 'ciclo': 'DAM'}
```

Se pueden combinar con parámetros normales. El orden en la definición es siempre: parámetros normales, `*args`, parámetros que sólo pueden pasarse por nombre y `**kwargs`:

```
def registrar(mensaje: str, *etiquetas: str, nivel: str = "INFO", **datos) -> None:
    print(f"[{nivel}] {mensaje} {etiquetas} {datos}")

registrar("Alta", "alumno", "2DAM", id=3, nombre="Ana")
# [INFO] Alta ('alumno', '2DAM') {'id': 3, 'nombre': 'Ana'}
registrar("Error al guardar", nivel="ERROR")
# [ERROR] Error al guardar () {}
```

Como `nivel` va después de `*etiquetas`, sólo se puede indicar por nombre: cualquier argumento posicional extra irá a `etiquetas`. Para obligar a pasar ciertos parámetros por nombre sin admitir argumentos variables, se usa un `*` solo: `def conectar(host, *, puerto=5432, usuario="dam")` obliga a escribir `conectar("localhost", puerto=5433)`.

Desempaquetar al llamar

Los asteriscos también se usan en la **llamada**, con el efecto contrario: `*` reparte los elementos de una secuencia como argumentos posicionales, y `**` reparte un diccionario como argumentos por nombre. Es muy habitual para pasar a una función datos que ya tenemos en una lista o en un diccionario (por ejemplo, un mensaje JSON recibido de un cliente):

```
def presentar(nombre: str, edad: int, ciclo: str) -> str:
    return f"{nombre}, {edad} años, {ciclo}"

datos = ["Ana", 20, "DAM"]
print(presentar(*datos))              # presentar("Ana", 20, "DAM")

fila = {"nombre": "Luis", "edad": 22, "ciclo": "DAW"}
print(presentar(**fila))              # presentar(nombre="Luis", edad=22, ciclo="DAW")
```

El uso más frecuente de `*args, **kwargs` juntos es en funciones que **reenvían** sus argumentos a otra sin necesidad de conocerlos, como los decoradores que veremos un poco más adelante.

0.6.3 Funciones como objetos y funciones lambda

En Python una función es un objeto más: se puede guardar en una variable, pasar como argumento o devolver desde otra función. Fíjate en la diferencia entre `f` (la función) y `f()` (el resultado de llamarla). Cuando creemos hilos y procesos le pasaremos **la función**, sin paréntesis, para que sea el hilo quien la ejecute:

```
def tarea():
    print("trabajando...")

def ejecutar_dos_veces(funcion):
    funcion()
    funcion()

ejecutar_dos_veces(tarea)        # bien: pasamos la función
# ejecutar_dos_veces(tarea())    # MAL: se ejecuta ahora y se pasa None

doble = lambda x: x * 2          # función anónima de una expresión
```

```
print(sorted(["pera", "kiwi", "uva"], key=len))
```

Las **funciones lambda** son funciones anónimas de una sola expresión, útiles para pasar una función corta como argumento, por ejemplo el criterio de ordenación:

```
tareas = [("copia de seguridad", 5), ("alarma", 1), ("informe", 3)]
tareas.sort(key=lambda t: t[1])            # ordenar por prioridad
# equivale a definir: def prioridad(t): return t[1]   y usar key=prioridad
```

> **Cuidado.** Con `multiprocessing` (Tema 1) no se pueden usar funciones lambda ni funciones definidas dentro de otras como función objetivo de un proceso: la función se envía al proceso hijo por su nombre, así que tiene que estar definida a nivel de módulo. Con los hilos no hay esa limitación.

0.7 Anotaciones de tipo

Python es un lenguaje de **tipado dinámico**: el tipo lo tienen los valores, no las variables, y el intérprete no comprueba los tipos antes de ejecutar. Aun así, desde Python 3.5 se pueden **anotar** los tipos esperados de variables, parámetros y valores devueltos. En este curso las usaremos siempre.

0.7.1 Sintaxis

```
edad: int = 20                      # variable
nombres: list[str] = []             # variable de tipo colección

def media(valores: list[float], decimales: int = 2) -> float:
    #      parámetro: tipo          valor por defecto  -> tipo devuelto
    return round(sum(valores) / len(valores), decimales)

def saludar(nombre: str) -> None:    # None: no devuelve nada
    print(f"Hola, {nombre}")
```

0.7.2 Tipos más habituales

Anotación	Significado
`int`, `float`, `str`, `bool`, `bytes`	Tipos básicos.
`list[str]`, `set[int]`	Lista de cadenas, conjunto de enteros.
`dict[str, float]`	Diccionario con claves `str` y valores `float`.
`tuple[int, str]`	Tupla de exactamente dos elementos: un entero y una cadena.
`tuple[int, ...]`	Tupla de enteros de cualquier longitud.
`int | None`	Un entero o `None` (antes se escribía `Optional[int]`).
`int | str`	Un entero o una cadena (antes, `Union[int, str]`).
`Alumno`, `"Alumno"`	Un objeto de una clase. Hasta Python 3.13, entre comillas si la clase todavía no está definida (por ejemplo, dentro de la propia clase); ver la nota sobre la evaluación diferida.
`Any`	Cualquier tipo: desactiva la comprobación (`from typing import Any`).
`Callable[[int, int], int]`	Una función que recibe dos enteros y devuelve un entero (`from collections.abc import Callable`).
`Iterator[str]`	Un iterador o generador que produce cadenas (`collections.abc`).
`Literal["r", "w"]`	Sólo esos valores concretos (`from typing import Literal`).

Se pueden definir **alias** para tipos complejos y escribir funciones **genéricas**, que funcionan con cualquier tipo manteniendo la relación entre parámetros y resultado (sintaxis de Python 3.12):

```
type Notas = dict[str, list[float]]     # alias: nombre del alumno -> sus notas

def mejores(notas: Notas, minimo: float) -> list[str]:
    return [n for n, lista in notas.items() if min(lista) >= minimo]

def primero[T](elementos: list[T]) -> T:   # T: "el tipo que sea"
    return elementos[0]

x = primero([3, 4])          # el editor sabe que x es int
s = primero(["a", "b"])      # y que s es str
```

0.7.3 Más ejemplos

Los tipos `Literal`, `Any` y `TypedDict` se importan del módulo `typing`; `Callable` e `Iterable`, de `collections.abc`.

```
# Ejemplos de anotaciones
from collections.abc import Callable, Iterable
from typing import Literal, TypedDict

# Uniones y valores opcionales
def buscar(usuarios: dict[str, int], nombre: str) -> int | None:
    return usuarios.get(nombre)

def parsear(valor: str | bytes) -> str:
```

```
    return valor.decode() if isinstance(valor, bytes) else valor

# Valores concretos permitidos
Nivel = Literal["DEBUG", "INFO", "WARNING", "ERROR"]

def registrar(mensaje: str, nivel: Nivel = "INFO") -> None:
    print(f"[{nivel}] {mensaje}")

# Funciones como parámetros: Callable[[argumentos], retorno]
def aplicar(f: Callable[[int], int], datos: Iterable[int]) -> list[int]:
    return [f(x) for x in datos]

# Alias de tipo (Python 3.12+): un nombre para un tipo complejo
type Direccion = tuple[str, int]

def conectar(destino: Direccion) -> None:
    host, port = destino
    print(f"conectando a {host}:{port}")

# Diccionarios con claves fijas
class Peticion(TypedDict):
    metodo: str
    ruta: str
    cabeceras: dict[str, str]

def describir(p: Peticion) -> str:
    return f"{p['metodo']} {p['ruta']}"
```

Las clases también pueden ser genéricas. Esta pila sólo admite elementos del tipo indicado al crearla:

```
class Pila[T]:
    def __init__(self) -> None:
        self._datos: list[T] = []

    def apilar(self, x: T) -> None:
        self._datos.append(x)

    def desapilar(self) -> T:
        return self._datos.pop()

p: Pila[int] = Pila()      # una pila de enteros
p.apilar(3)
p.apilar("tres")           # mypy: error, la pila es de enteros
```

La librería estándar está llena de clases genéricas: `list[str]` o `dict[str, int]` lo son, y `queue.Queue[str]` será una cola de cadenas en el Tema 2. En el Tema 4 definiremos un modelo genérico `Pagina[T]` para las respuestas paginadas de una API.

0.7.4 ¿Para qué sirven si no se comprueban?

Las anotaciones **no cambian la ejecución**: si se pasa un tipo distinto del anotado, Python no protesta (el error, si lo hay, aparecerá más tarde, cuando se use el valor). Su utilidad está en otra parte:

- **Documentación**: la firma de la función dice qué recibe y qué devuelve.

- **Ayuda del editor**: autocompletado y avisos mientras se escribe.
- **Comprobación estática**: herramientas como `mypy` (o la integrada en el editor) analizan el código *sin ejecutarlo* y detectan errores de tipos.
- **Bibliotecas que las leen en tiempo de ejecución**: las *dataclasses* generan el constructor a partir de ellas, Pydantic las usa para validar y convertir datos y FastAPI, para saber de dónde leer cada parámetro de una petición (Tema 4). Se guardan en el atributo `__annotations__` y se obtienen con `typing.get_type_hints()`.

```python
# tipos.py
def media(valores: list[float], decimales: int = 2) -> float:
    return round(sum(valores) / len(valores), decimales)

def buscar(nombre: str, agenda: dict[str, str]) -> str | None:
    return agenda.get(nombre)

nota = media([7, 8, 9], decimales="2")              # error de tipo
telefono = buscar("Ana", {"Ana": "600111222"})
print(telefono.upper())                              # ¿y si es None?
```

```
$ pip install mypy
$ mypy tipos.py
tipos.py:7: error: Argument "decimales" to "media" has incompatible type "str"; expected
    ↪ "int"
tipos.py:9: error: Item "None" of "str | None" has no attribute "upper"
Found 2 errors in 1 file (checked 1 source file)
```

`mypy` ha encontrado los dos errores sin ejecutar el programa. El segundo es especialmente valioso: obliga a tratar el caso en que `buscar` no encuentra nada, un error muy frecuente. Se corrige comprobándolo antes (`if telefono is not None:`); a partir de ahí, `mypy` sabe que `telefono` es un `str` (*estrechamiento de tipos*).

Con `mypy --strict`, además, exige que todas las funciones estén anotadas. En VS Code puedes activar la comprobación de Pylance en *Settings* → *Python › Analysis: Type Checking Mode* (`basic` o `strict`).

0.7.5 `Annotated`: añadir información al tipo

`Annotated[tipo, información]` es el tipo indicado acompañado de datos extra que el comprobador ignora pero que otras librerías leen. Lo veremos constantemente con FastAPI y Pydantic:

```python
from typing import Annotated
from pydantic import Field

Edad = Annotated[int, Field(ge=16, le=99)]       # un int entre 16 y 99
# En FastAPI:  limite: Annotated[int, Query(ge=1, le=100)] = 20
```

Evaluación diferida de las anotaciones. Hasta Python 3.13, las anotaciones se evaluaban al definir la función, por lo que para referirse a una clase que todavía no estaba definida (por ejemplo, la propia clase dentro de sus métodos) había que escribirla entre comillas: `def transferir(self, otra: "Cuenta")`. Desde Python 3.14 (PEP 649) las anota-

ciones se evalúan sólo cuando alguien las consulta, y las comillas ya no son necesarias. Verás ambas formas en el código que encuentres.

0.7.6 Buenas prácticas

- Anota siempre los **parámetros y el valor de retorno** de las funciones y métodos. Las variables locales normalmente no hace falta: el comprobador deduce su tipo (`n = 0` es un `int`).
- Usa la sintaxis moderna: `list[int]` en lugar de `List[int]`, `int | None` en lugar de `Optional[int]`.
- En los parámetros, pide el tipo más general que te sirva (`Iterable[int]` si sólo vas a recorrerlo); en el retorno, devuelve el más concreto (`list[int]`).
- Evita `Any`: es «apagar» la comprobación.
- Pasa un comprobador de tipos antes de entregar una práctica, igual que pasarías las pruebas.

0.8 Decoradores

Un **decorador** es una función que recibe otra función y devuelve una versión modificada de ella, que hace algo más (antes, después o en lugar de la original) sin tocar su código. Se aplica escribiendo `@nombre` justo encima de la definición. Aparecerán enseguida en estos apuntes: `@contextmanager` en los gestores de contexto y `@property`, `@classmethod` y `@dataclass` en la programación orientada a objetos. Escribir uno sencillo ayuda a entender cómo funcionan:

```
def anunciar(funcion):
    def envoltorio(*args, **kwargs):
        print(f"Empieza {funcion.__name__}")
        resultado = funcion(*args, **kwargs)      # se llama a la función original
        print(f"Termina {funcion.__name__}")
        return resultado
    return envoltorio

@anunciar
def sumar(a: int, b: int) -> int:
    return a + b

print(sumar(2, 3))
# Empieza sumar
# Termina sumar
# 5
```

Paso a paso:

1. `anunciar` recibe una función (`funcion`) y define dentro otra, `envoltorio`, que la "envuelve": muestra un mensaje, llama a la original con los mismos argumentos (`*args`, `**kwargs`, apartado de funciones), muestra otro mensaje y devuelve el resultado.
2. `anunciar` devuelve `envoltorio`, **sin llamarla**.
3. Escribir `@anunciar` encima de `def sumar` es sólo una forma abreviada de escribir, después de definirla:

```
sumar = anunciar(sumar)     # "sumar" pasa a ser el envoltorio
```

4. Por eso, al llamar a `sumar(2, 3)`, en realidad se ejecuta `envoltorio(2, 3)`, que a su vez llama a la `sumar` original.

Los decoradores se usan para añadir a muchas funciones el mismo comportamiento: registrar llamadas, medir tiempos, comprobar permisos, reintentar operaciones que fallan... FastAPI los usa para asociar cada función a una ruta (`@app.get("/libros")`, Tema 4) y en el Tema 5 escribiremos uno que comprueba los permisos del usuario. Por ejemplo, para medir cuánto tarda una función:

```
import time

def cronometrar(funcion):
    def envoltorio(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = funcion(*args, **kwargs)
        print(f"{funcion.__name__} tardó {time.perf_counter() - inicio:.3f} s")
        return resultado
    return envoltorio

@cronometrar
def contar(n: int) -> int:
    return sum(range(n))

contar(1_000_000)          # contar tardó 0.022 s
```

Nota. Como la función decorada es en realidad el envoltorio, pierde su nombre y su documentación: `sumar.__name__` vale `"envoltorio"`. Para conservarlos, en el código profesional se añade `@functools.wraps(funcion)` justo encima de `def envoltorio`. No cambia nada más del funcionamiento.

0.9 Módulos y paquetes

Cada fichero `.py` es un **módulo**. Una carpeta con módulos es un **paquete**. Para usar el código de otro módulo se importa:

```
import json                          # módulo completo
from pathlib import Path             # un nombre concreto
from datetime import date as Fecha   # con alias
```

Un módulo puede ejecutarse como programa o importarse desde otro. Para que cierto código se ejecute sólo en el primer caso se usa este patrón, que veremos en todos los ejemplos:

```
def main() -> None:
    print("Programa principal")

if __name__ == "__main__":
    main()
```

Una estructura típica de proyecto del curso será:

```
proyecto/
├── .venv/
├── requirements.txt
├── tests/
│   └── test_protocolo.py
└── chat/                <- paquete
    ├── __init__.py
    ├── protocolo.py
    ├── servidor.py
    └── cliente.py
```

Y se ejecuta desde la carpeta `proyecto` con `python -m chat.servidor` y `python -m chat.cliente`.

Imprescindible en el Tema 1. En Windows y macOS, `multiprocessing` crea los procesos hijos *importando* tu fichero. Si el código que crea procesos no está protegido por `if __name__ == "__main__":`, cada hijo volvería a crear hijos y el programa fallaría. Acostúmbrate a usar siempre este patrón.

0.10 Excepciones

Cuando algo va mal (un fichero no existe, una conexión falla, un dato no tiene el formato esperado) Python **lanza una excepción**. Si nadie la captura, el programa termina mostrando la traza del error. Este mecanismo es central en el módulo: en un programa de red, que el servidor no responda o que el cliente cierre la conexión a mitad de un mensaje no son fallos raros, sino situaciones normales que el programa debe prever.

```python
def leer_entero(texto: str) -> int | None:
    try:
        valor = int(texto)
    except ValueError as e:
        print(f"'{texto}' no es un número: {e}")
        return None
    else:
        # se ejecuta sólo si no hubo excepción
        return valor
    finally:
        # se ejecuta SIEMPRE, haya excepción o no
        print("Conversión terminada")
```

Algunas excepciones habituales:

`ValueError`	Valor con tipo correcto pero contenido inválido (`int("abc")`).
`TypeError`	Operación con un tipo inadecuado (`"a" + 1`).
`KeyError` / `IndexError`	Clave de diccionario o índice de lista inexistente.
`FileNotFoundError`	El fichero no existe (subclase de `OSError`).
`PermissionError`	Sin permisos sobre un fichero (subclase de `OSError`).
`ConnectionRefusedError`	No hay ningún servidor escuchando en ese puerto (Tema 3).
`ConnectionError`	La conexión se ha cortado o el otro extremo la ha cerrado.
`TimeoutError`	Una operación de red ha superado el tiempo máximo de espera.

Podemos lanzar excepciones con `raise` y definir las nuestras heredando de `Exception`:

```
class ProtocoloError(Exception):
    """El mensaje recibido no respeta el protocolo."""

def leer_orden(linea: str) -> tuple[str, list[str]]:
    partes = linea.split()
    if not partes:
        raise ProtocoloError("Mensaje vacío")
    return partes[0].upper(), partes[1:]
```

Cuidado. Evita el `except:` vacío o `except Exception:` sin más, porque ocultan errores de programación. Captura sólo las excepciones que sabes tratar y deja que el resto se propague.

0.10.1 La sentencia `assert`

`assert` comprueba que una condición se cumple y, si no es así, lanza un `AssertionError`. Se puede añadir un mensaje tras una coma:

```
def media(valores: list[float]) -> float:
    resultado = sum(valores) / len(valores)
    assert min(valores) <= resultado <= max(valores), "media fuera de rango"
    return resultado

assert media([4, 6]) == 5
assert media([4, 6]) == 6, "la media debería ser 6"
# AssertionError: la media debería ser 6
```

Su utilidad es expresar **suposiciones del programador**: condiciones que, si el programa es correcto, siempre deben cumplirse. Si alguna falla, el error está en el código, y `assert` lo detecta en el punto exacto donde aparece en lugar de dejar que se propague y provoque un fallo difícil de rastrear más adelante. Por eso es la herramienta básica para escribir **pruebas**: cada prueba ejecuta un fragmento de código y comprueba con `assert` que el resultado es el esperado. El Anexo B explica cómo organizar y ejecutar pruebas con `pytest`.

Cuidado. `assert` **no sirve para validar datos** que vienen de fuera del programa (lo que escribe el usuario, el contenido de un fichero, la respuesta de un servidor). Si se ejecuta Python con la opción de optimización (`python -O programa.py`), todas las sentencias `assert` se eliminan y la comprobación desaparece. Para validar datos externos se usa `if` y se lanza la excepción adecuada:

```
if edad < 0:
    raise ValueError("La edad no puede ser negativa")    # correcto
assert edad >= 0, "La edad no puede ser negativa"       # incorrecto para datos
    ↪ externos
```

0.11 Gestores de contexto: la sentencia `with`

Muchos recursos (ficheros, conexiones de red, cerrojos) deben **liberarse siempre**, incluso si se produce un error. La sentencia `with` lo garantiza:

```
with open("datos.txt", "w", encoding="utf-8") as f:
    f.write("Hola\n")
# aquí el fichero ya está cerrado, aunque write() hubiera fallado
```

En este módulo usaremos `with` con cerrojos (`with cerrojo:`, Tema 2), *sockets* (`with socket.create_connection(...) as s:`, Tema 3), conjuntos de procesos (`with Pool() as pool:`, Tema 1) y conexiones seguras (Tema 5).

Un **gestor de contexto** es cualquier objeto que tenga los métodos `__enter__` y `__exit__`, y por tanto puede usarse en una sentencia `with`. Al entrar en el bloque, `with` llama a `__enter__`, cuyo resultado se asigna a la variable que sigue a `as`. Al salir, llama siempre a `__exit__`, tanto si el bloque termina bien como si lanza una excepción; ahí es donde se liberan los recursos.

También se pueden crear gestores de contexto de forma sencilla con el decorador `contextlib.contextmanager`:

```
from contextlib import contextmanager
import time

@contextmanager
def cronometro(etiqueta: str):
    inicio = time.perf_counter()
    try:
        yield                       # aquí se ejecuta el bloque del with
    finally:
        print(f"{etiqueta}: {time.perf_counter() - inicio:.3f} s")

with cronometro("Carga de datos"):
    total = sum(range(10_000_000))
```

0.12 Programación orientada a objetos

0.12.1 Clases y objetos

```
class Alumno:
    centro = "IES Ejemplo"            # atributo de clase (compartido)

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre          # atributos de instancia
        self.edad = edad
        self.notas: list[float] = []

    def anadir_nota(self, nota: float) -> None:
        self.notas.append(nota)

    def media(self) -> float:
        return sum(self.notas) / len(self.notas) if self.notas else 0.0

    def __str__(self) -> str:         # representación para el usuario
```

```
        return f"{self.nombre} ({self.edad} años)"

    def __repr__(self) -> str:        # representación para el programador
        return f"Alumno(nombre={self.nombre!r}, edad={self.edad})"

a = Alumno("Ana", 20)
a.anadir_nota(8)
print(a, a.media())
```

- `__init__` es el inicializador (equivalente al constructor).
- `self` es la referencia al propio objeto y se declara explícitamente como primer parámetro.
- No existen `private` ni `protected`: la visibilidad se indica con guiones bajos en el nombre (lo vemos más abajo).
- Los métodos con doble guion bajo (`__str__`, `__eq__`, `__len__`...) se llaman **métodos especiales** y permiten que nuestros objetos se integren con el lenguaje.

0.12.2 Atributos de clase y atributos de instancia

- Los **atributos de instancia** se crean asignando `self.nombre = valor`, normalmente en `__init__`. Cada objeto tiene los suyos.
- Los **atributos de clase** se definen en el cuerpo de la clase, fuera de los métodos. Existe **una sola copia, compartida por todos los objetos**. Equivalen a los atributos `static` de Java y se usan para constantes, valores por defecto o contadores comunes.

```
class Conexion:
    puerto_defecto = 5000          # atributo de CLASE: uno para todas
    abiertas = 0                   # contador compartido

    def __init__(self, host: str, puerto: int | None = None):
        self.host = host                                # atributo de INSTANCIA
        self.puerto = puerto or Conexion.puerto_defecto
        Conexion.abiertas += 1     # se modifica a través de la CLASE

a = Conexion("192.168.1.10")
b = Conexion("192.168.1.20", 8080)
print(a.puerto, b.puerto)          # 5000 8080
print(Conexion.abiertas, a.abiertas, b.abiertas)   # 2 2 2

Conexion.puerto_defecto = 6000     # cambia para todos los objetos
print(a.puerto_defecto, b.puerto_defecto)          # 6000 6000

a.puerto_defecto = 7000            # ¡crea un atributo de instancia en 'a'!
print(a.puerto_defecto, b.puerto_defecto, Conexion.puerto_defecto)  # 7000 6000 6000
print(vars(a))                     # {'host': ..., 'puerto': 5000, 'puerto_defecto':
    ↪ 7000}
```

Cuando se lee `objeto.atributo`, Python lo busca primero en el objeto y, si no lo encuentra, en su clase (y después en las clases padre). Por eso `a.abiertas` funciona aunque `a` no tenga ese atributo. Pero al *asignar* `a.puerto_defecto = 7000` no se modifica el atributo de clase: se crea uno nuevo en el objeto `a` que «tapa» al de la clase. **Para modificar un atributo de clase, usa siempre el nombre de la clase**: `Conexion.abiertas += 1`.

Nunca uses un objeto mutable como atributo de clase.

```
class Grupo:
    alumnos = []                    # MAL: una única lista para TODOS los grupos

    def matricular(self, nombre: str) -> None:
        self.alumnos.append(nombre)

g1, g2 = Grupo(), Grupo()
g1.matricular("Ana")
print(g2.alumnos)                   # ['Ana']  ¡también aparece en g2!

class GrupoBien:
    def __init__(self):
        self.alumnos: list[str] = []   # BIEN: una lista por objeto
```

`append` no asigna, modifica la lista existente, que es la de la clase. Además, un atributo de clase compartido es **estado compartido**: si varios hilos lo modifican a la vez tendremos las condiciones de carrera del Tema 2.

0.12.3 Métodos de clase y métodos estáticos

Además de los métodos normales (*de instancia*), que reciben el objeto en `self`, una clase puede tener otros dos tipos de métodos, que se marcan con un decorador (sección 0.8):

- **`@classmethod`**: recibe la **clase** en lugar del objeto, en un primer parámetro que por convención se llama `cls`. Su uso principal son los **constructores alternativos**: formas distintas de crear un objeto (a partir de bytes recibidos por la red, de un JSON, de un fichero...).
- **`@staticmethod`**: no recibe ni el objeto ni la clase. Es una función normal que se coloca dentro de la clase porque está relacionada con ella. Equivale a un método `static` de Java.

```
class Mensaje:
    separador = "|"                         # atributo de clase

    def __init__(self, emisor: str, texto: str):
        self.emisor = emisor
        self.texto = texto

    def a_bytes(self) -> bytes:             # método de INSTANCIA: usa self
        return f"{self.emisor}{self.separador}{self.texto}\n".encode("utf-8")

    @classmethod
    def desde_bytes(cls, datos: bytes) -> "Mensaje":   # método de CLASE: usa cls
        emisor, texto = datos.decode("utf-8").rstrip("\n").split(cls.separador, 1)
        return cls(emisor, texto)           # crea un objeto de la clase

    @staticmethod
    def es_valido(datos: bytes) -> bool:    # método ESTÁTICO: ni self ni cls
        return datos.endswith(b"\n") and len(datos) <= 1024

m = Mensaje("ana", "hola")
datos = m.a_bytes()                         # b'ana|hola\n'
print(Mensaje.es_valido(datos))             # True
copia = Mensaje.desde_bytes(datos)          # se llama sobre la clase
print(copia.emisor, copia.texto)            # ana hola
```

La ventaja de usar `cls` en lugar de escribir `Mensaje` dentro del método es que funciona también con las clases hijas: si `MensajeUrgente` hereda de `Mensaje`, la llamada `MensajeUrgente.desde_bytes(...)` devuelve un `MensajeUrgente` y usa su propio `separador`.

Tipo	Primer parámetro	Cuándo usarlo
De instancia	`self` (el objeto)	Necesita los datos del objeto. Es el caso habitual.
`@classmethod`	`cls` (la clase)	Constructores alternativos; trabajar con atributos de clase.
`@staticmethod`	ninguno	Funciones auxiliares relacionadas con la clase que no usan ni el objeto ni la clase.

Los encontraremos en las bibliotecas que usaremos. Por ejemplo, en los modelos de Pydantic, `model_validate_json` es un método de clase que construye un objeto a partir de un JSON, y los validadores de Pydantic se declaran con `@classmethod`.

0.12.4 Visibilidad: el guion bajo

Python no tiene modificadores de acceso como `private`, `protected` o `public`. La visibilidad de atributos y métodos se indica con **convenciones en el nombre**:

Nombre	Equivale a	Significado
`nombre`	público	Forma parte de la interfaz de la clase; se puede usar desde fuera.
`_nombre`	protegido / interno	Convenio: "es un detalle interno, no lo uses desde fuera". Python no lo impide, pero los editores avisan y `from modulo import *` no importa los nombres que empiezan por _.
`__nombre`	privado	Python **renombra** el atributo a `_Clase__nombre` (*name mangling*). Así se evita que una subclase lo sobrescriba por accidente; no es una protección real.
`__nombre__`	—	Métodos especiales del lenguaje (`__init__`, `__str__`...). No hay que inventar nombres nuevos con este formato.

```python
class Cuenta:
    def __init__(self, titular: str, saldo: float, pin: str) -> None:
        self.titular = titular        # público
        self._saldo = saldo           # interno: úsese a través de métodos
        self.__pin = pin              # "privado": se guarda como _Cuenta__pin

    def comprobar_pin(self, pin: str) -> bool:     # método público
        return self.__pin == pin                   # dentro de la clase, normal

    def _registrar(self, texto: str) -> None:      # método interno
        print(f"[{self.titular}] {texto}")

c = Cuenta("Ana", 100.0, "1234")
print(c.titular)          # Ana
```

```
print(c._saldo)           # 100.0 (funciona, pero no se debe hacer)
print(c.__pin)            # AttributeError: 'Cuenta' object has no attribute '__pin'
print(c._Cuenta__pin)     # 1234: el nombre real; la "privacidad" es un convenio
```

El *name mangling* **no es un mecanismo de seguridad** (como se ve, `c._Cuenta__pin` sigue siendo accesible): su objetivo es evitar choques de nombres en la herencia. Si una clase padre y una hija usan el mismo atributo `__id`, cada una tiene el suyo:

```
class Base:
    def __init__(self):
        self.__id = "base"          # se guarda como _Base__id

class Hija(Base):
    def __init__(self):
        super().__init__()
        self.__id = "hija"          # se guarda como _Hija__id: no pisa al padre

h = Hija()
print(vars(h))                      # {'_Base__id': 'base', '_Hija__id': 'hija'}
```

Nota. La filosofía de Python se resume en la frase "*we are all consenting adults here*": el lenguaje confía en que el programador respete los convenios. Para controlar de verdad cómo se lee o modifica un atributo se usan las **propiedades** del apartado siguiente.

¿Uno o dos guiones bajos? En la práctica, la mayoría del código Python usa **un solo guion bajo** para todo lo que es de uso interno, y reserva los dos para evitar conflictos en clases pensadas para heredarse. En lugar de escribir métodos `get_saldo()` y `set_saldo()` como en Java, se expone el atributo interno a través de una **propiedad** (`@property`), que se usa como si fuera un atributo (`c.saldo`) pero ejecuta un método; si no se define su *setter*, es de sólo lectura.

0.12.5 Herencia

En los temas 1 y 2 crearemos procesos e hilos heredando de las clases `Process` y `Thread` y sobrescribiendo su método `run()`. El mecanismo es el mismo que en este ejemplo:

```
class Persona:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def presentarse(self) -> str:
        return f"Soy {self.nombre}"

class Profesor(Persona):
    def __init__(self, nombre: str, departamento: str) -> None:
        super().__init__(nombre)
        self.departamento = departamento

    def presentarse(self) -> str:      # sobrescritura
        return f"{super().presentarse()} y trabajo en {self.departamento}"
```

0.12.6 Propiedades

En lugar de escribir *getters* y *setters* como en Java, Python permite acceder a los atributos directamente y, si más adelante hace falta validar, convertirlos en **propiedades** sin cambiar el código que los usa.

```
class Producto:
    def __init__(self, nombre: str, precio: float) -> None:
        self.nombre = nombre
        self.precio = precio           # pasa por el setter

    @property
    def precio(self) -> float:
        return self._precio

    @precio.setter
    def precio(self, valor: float) -> None:
        if valor < 0:
            raise ValueError("El precio no puede ser negativo")
        self._precio = valor

p = Producto("Teclado", 25.0)
p.precio = 30            # llama al setter
p.precio = -1            # ValueError
```

Fíjate en que en el código aparecen dos nombres distintos, y es intencionado:

- `precio` es la **propiedad**: la interfaz pública de la clase. Toda lectura (`p.precio`) pasa por el *getter* y toda asignación (`p.precio = 30`) pasa por el *setter*.
- `_precio` es el **atributo interno** donde se guarda realmente el valor. Lleva guion bajo porque es un detalle de implementación (ver el apartado de visibilidad).

Por eso la línea `self.precio = precio` del constructor **no crea un atributo** `precio`: como `precio` es una propiedad definida en la clase, esa asignación llama al *setter*, que valida el valor y lo guarda en `self._precio`. La ventaja es que la validación se aplica también al crear el objeto:

```
p = Producto("Teclado", 25.0)
print(vars(p))           # {'nombre': 'Teclado', '_precio': 25.0}
Producto("Ratón", -5)         # ValueError: El precio no puede ser negativo
```

> **Cuidado.** Dentro del *getter* y del *setter* hay que usar el atributo interno (`self._precio`), nunca la propiedad. Si en el *setter* se escribiera `self.precio = valor`, esa asignación volvería a llamar al *setter*, que se llamaría a sí mismo una y otra vez hasta provocar un `RecursionError`.

0.12.7 Dataclasses

Para clases cuyo propósito principal es **guardar datos** (lo que en Java serían los POJO o los *records*), el decorador `@dataclass` del módulo `dataclasses` genera automáticamente `__init__`, `__repr__` y `__eq__`. Las usaremos para representar los mensajes, tareas y resultados que se intercambian procesos, hilos y programas.

```
from dataclasses import dataclass, field, asdict
```

```python
from datetime import date

@dataclass
class Libro:
    isbn: str
    titulo: str
    autor: str
    anio: int
    disponible: bool = True
    etiquetas: list[str] = field(default_factory=list)

l = Libro("978-84-00000-00-0", "El Quijote", "Cervantes", 1605)
print(l)              # Libro(isbn='978-...', titulo='El Quijote', ...)
print(asdict(l))      # convierte a diccionario (útil para JSON)

@dataclass(frozen=True)     # inmutable
class Punto:
    x: float
    y: float
```

La función `field`

En el ejemplo, el valor por defecto del campo `etiquetas` no es una lista vacía, sino una llamada a `field(default_factory=list)`. La función `field` del módulo `dataclasses` permite configurar un campo concreto cuando no basta con escribir su tipo y un valor por defecto.

Su uso más habitual es dar como valor por defecto una **colección vacía**. En Python, los valores por defecto se crean **una sola vez**, al definir la clase o la función, no cada vez que se usan. Si ese valor es una lista, todos los objetos comparten la misma lista, y lo que añade uno lo ven todos. Con una clase normal es un error clásico y silencioso:

```python
class LibroNormal:
    def __init__(self, titulo: str, etiquetas: list[str] = []):   # ¡una sola lista!
        self.titulo = titulo
        self.etiquetas = etiquetas

x, y = LibroNormal("Nada"), LibroNormal("La colmena")
x.etiquetas.append("novela")
print(y.etiquetas)        # ['novela']  <- y también la tiene
```

Las *dataclasses* impiden este error: si se escribe `etiquetas: list[str] = []`, la clase ni siquiera se crea y se produce `ValueError: mutable default <class 'list'> for field etiquetas is not allowed: use default_factory`. La solución es `default_factory`, que recibe una **función sin argumentos** y la llama **cada vez que se crea un objeto**:

```python
a, b = Libro("1", "Nada", "Laforet", 1945), Libro("2", "La colmena", "Cela", 1951)
a.etiquetas.append("novela")
print(b.etiquetas)                  # []: cada libro tiene su propia lista
print(a.etiquetas is b.etiquetas)   # False
```

Para diccionarios y conjuntos se hace lo mismo, pasando `dict` o `set` como `default_factory`. Con valores inmutables (números, cadenas, `None`, tuplas) no hace falta: `disponible: bool = True` es correcto.

Las clases inmutables (`frozen=True`, como `Punto` en el ejemplo) son especialmente útiles en

programas concurrentes: si un objeto no se puede modificar, varios hilos pueden compartirlo sin ninguna sincronización.

`field` admite otras opciones útiles:

`field(default_factory=f)`	El valor por defecto se obtiene llamando a `f()` para cada objeto.
`field(repr=False)`	El campo no aparece al imprimir el objeto (útil para contraseñas o datos internos).
`field(compare=False)`	El campo no se tiene en cuenta al comparar objetos con ==.
`field(init=False)`	El campo no es un parámetro del constructor (se calcula después, por ejemplo en `__post_init__`).

0.12.8 Clases de datos con validación: Pydantic

Las *dataclasses* de la biblioteca estándar **no comprueban** que los valores coincidan con las anotaciones: `Libro("978-...", "Nada", "Laforet", "1945")` crea un libro cuyo año es la cadena `"1945"`. Cuando los datos llegan de fuera del programa (un fichero de configuración, un mensaje recibido por un *socket*, el cuerpo de una petición a una API), conviene validarlos. Para eso se usa la biblioteca Pydantic, que ofrece dos opciones con la misma sintaxis basada en anotaciones:

```
$ pip install pydantic
```

Dataclasses de Pydantic. Se usan exactamente igual que las de la biblioteca estándar, cambiando sólo el `import`, pero validan y convierten los datos al crear el objeto:

```
from pydantic import ValidationError
from pydantic.dataclasses import dataclass    # en lugar de "from dataclasses"

@dataclass
class Libro:
    titulo: str
    anio: int

print(Libro("Nada", "1945"))      # Libro(titulo='Nada', anio=1945): convierte "1945"
try:
    Libro("Nada", "mil")
except ValidationError as e:
    print(e.errors()[0]["msg"])   # Input should be a valid integer...
```

`BaseModel`. Es la forma principal de usar Pydantic y la que usa FastAPI (Tema 4). Además de validar, permite añadir restricciones a cada campo, escribir validadores propios y convertir los objetos desde y hacia diccionarios y JSON directamente.

```
# Modelo de datos con BaseModel
from datetime import date

from pydantic import BaseModel, Field, ValidationError, field_validator

class Alumno(BaseModel):
```

```
    nombre: str = Field(min_length=2, max_length=50)
    edad: int = Field(ge=16, le=99)
    correo: str
    nacimiento: date | None = None
    modulos: list[str] = []          # en Pydantic esto es seguro: se copia

    @field_validator("correo")
    @classmethod
    def correo_del_centro(cls, valor: str) -> str:
        if not valor.endswith("@educa.jcyl.es"):
            raise ValueError("debe ser un correo del centro")
        return valor.lower()

# Validar datos que llegan de fuera (por ejemplo, el JSON de una petición)
texto = '{"nombre": "Ana", "edad": "19", "correo": "ANA@educa.jcyl.es", "nacimiento":
    ↪ "2007-03-15"}'
ana = Alumno.model_validate_json(texto)
print(ana.nacimiento.year)          # 2007: la cadena se ha convertido a date
print(ana.model_dump())             # objeto -> dict
print(ana.model_dump_json())        # objeto -> JSON
print(ana.model_copy(update={"edad": 20}).edad)   # copia modificada

try:
    Alumno(nombre="A", edad=12, correo="a@gmail.com")
except ValidationError as e:
    print(e.error_count(), "errores")              # 3 errores
    for error in e.errors():
        print(" -", error["loc"][0], ":", error["msg"])
```

```
2007
{'nombre': 'Ana', 'edad': 19, 'correo': 'ana@educa.jcyl.es', 'nacimiento': datetime.date
    ↪ (2007, 3, 15), 'modulos': []}
{"nombre":"Ana","edad":19,"correo":"ana@educa.jcyl.es","nacimiento":"2007-03-15","
    ↪ modulos":[]}
20
3 errores
 - nombre : String should have at least 2 characters
 - edad : Input should be greater than or equal to 16
 - correo : Value error, debe ser un correo del centro
```

Fíjate en que Pydantic informa de **todos** los errores a la vez, no sólo del primero, e indica en qué campo está cada uno. El decorador `@field_validator` define una validación propia; los decoradores se explican en la sección 0.8.

> **Nota.** Por defecto, Pydantic valida al *crear* el objeto, pero no cuando después se asigna un atributo (`l.anio = "abc"` no da error). Si se necesita, se activa con `model_config = ConfigDict(validate_assignment=True)`.

0.12.9 Comparación de las clases para almacenar datos

	Clase normal	**`@dataclass`**	**`@dataclass` de Pydantic**	**`BaseModel`**
Origen	Lenguaje	Biblioteca estándar	Pydantic	Pydantic
`__init__`, `__repr__`, `__eq__`	Se escriben a mano	Generados	Generados	Generados
Comprueba los tipos al crear	No (salvo que se programe)	No	Sí	Sí
Convierte tipos (`"1945"` → `1945`)	No	No	Sí	Sí
Restricciones por campo (mínimo, longitud, patrón)	Con propiedades	No	Sí (`Field`)	Sí (`Field`)
Conversión a `dict` / JSON	A mano	`asdict()` / a mano	Con `TypeAdapter`	`model_dump()` `model_dump_json()`
Lectura desde JSON	A mano	A mano	Con `TypeAdapter`	`model_validate_json()`
Generación de JSON Schema	No	No	Sí	`model_json_schema()`
Inmutabilidad	A mano	`frozen=True`	`frozen=True`	`frozen=True` en la configuración
Coste	Mínimo	Mínimo	Validar tiene un coste	Validar tiene un coste
Cuándo usarla	Objetos con comportamiento y estado propio (un servicio, un repositorio)	Datos internos del programa, ya fiables	Igual que `@dataclass`, pero con datos de origen externo	Datos que entran o salen del programa: ficheros, API, configuración

Existen otras opciones: `typing.NamedTuple` (una tupla inmutable con nombres de campo) y, cuando no se necesita una clase, un simple `dict`, que es como llegan los datos de un JSON.

> **Regla práctica.** Valida en la **frontera** del programa. Los datos que vienen del exterior (peticiones HTTP, mensajes de red, ficheros, variables de entorno) se convierten en modelos de `BaseModel` nada más llegar; dentro del programa, donde ya sabes que los datos son correctos, una `dataclass` normal es más ligera. Esta idea, «no confíes en la entrada», es uno de los principios de programación segura del Tema 5.

0.13 Iteradores y generadores

Un objeto es **iterable** si se puede recorrer con `for`. Los ficheros, las líneas que llegan por un *socket* (Tema 3) o los resultados de `Pool.imap` (Tema 1) son iterables: los datos se van obteniendo *a medida que se piden*, sin cargarlo todo en memoria.

Una función que contiene `yield` es un **generador**: cada vez que se le pide un valor, se ejecuta hasta el siguiente `yield` y se detiene.

```python
def lineas_no_vacias(ruta: str):
    with open(ruta, encoding="utf-8") as f:
        for linea in f:
            linea = linea.strip()
            if linea:
                yield linea

for l in lineas_no_vacias("datos.txt"):
    print(l)
```

Las **expresiones generadoras** son como las comprensiones de listas pero con paréntesis, y no construyen la lista en memoria:

```python
total = sum(len(l) for l in lineas_no_vacias("datos.txt"))
```

0.14 Enumeraciones

Una **enumeración** define un conjunto cerrado de valores con nombre. Es más segura que usar cadenas o números sueltos, porque un error de escritura se detecta en lugar de pasar inadvertido.

```python
from enum import Enum

class Estado(Enum):
    PENDIENTE = "P"
    ENVIADO = "E"
    ENTREGADO = "D"

e = Estado("E")          # Estado.ENVIADO (a partir del valor guardado)
print(e.name, e.value)   # ENVIADO E
```

0.15 Texto y bytes

Este es probablemente el concepto más importante del tema para la segunda mitad del curso. En Python hay dos tipos distintos:

- `str`: **texto** (secuencia de caracteres Unicode).
- `bytes`: **datos binarios** (secuencia de valores 0–255).

Por la red, en los ficheros binarios y en la criptografía sólo viajan **bytes**. Para pasar de uno a otro hay que *codificar* y *decodificar* indicando una codificación, normalmente UTF-8:

```python
texto = "¡Hola, Peñafiel!"
datos = texto.encode("utf-8")    # str -> bytes
print(datos)        # b'\xc2\xa1Hola, Pe\xc3\xb1afiel!'
print(len(texto), len(datos))    # 16 18  (ñ y ¡ ocupan 2 bytes)
print(datos.decode("utf-8"))     # bytes -> str

literal = b"GET / HTTP/1.1\r\n"  # literal de bytes (sólo ASCII)
print(literal.hex())             # representación hexadecimal
```

Cuidado. Un error típico en el Tema 3 es intentar enviar un `str` por un socket (`sock.sendall("hola")`), lo que lanza `TypeError`. Hay que enviar `"hola".encode()`, y decodificar al recibir.

0.15.1 JSON

Para enviar estructuras de datos (diccionarios, listas...) entre programas usaremos JSON, que se convierte fácilmente a texto y de ahí a bytes:

```
import json

pedido = {"id": 7, "productos": ["teclado", "ratón"], "urgente": True}
texto = json.dumps(pedido, ensure_ascii=False)   # dict -> str
recuperado = json.loads(texto)                   # str  -> dict

with open("pedido.json", "w", encoding="utf-8") as f:
    json.dump(pedido, f, indent=2, ensure_ascii=False)
```

0.16 Fechas y horas

Casi todos los programas del módulo manejan fechas y horas: la marca de tiempo de cada línea de un registro, la cabecera `Date` de una respuesta HTTP, la fecha de caducidad de un *token* o de un certificado, el tiempo que tarda una tarea... Y cada protocolo y formato las representa de forma distinta. Merece la pena conocer bien el módulo `datetime` de la biblioteca estándar.

0.16.1 Los tipos de `datetime`

Tipo	Representa	Ejemplo
`date`	Una fecha (año, mes, día).	`2026-10-19`
`time`	Una hora del día (hora, minuto, segundo, microsegundo).	`09:30:00`
`datetime`	Una fecha con su hora: un instante concreto.	`2026-10-19 09:30:00`
`timedelta`	Una **duración**: la diferencia entre dos fechas u horas.	`67 days, 0:00:00`

Todos ellos son **inmutables**: las operaciones no modifican el objeto, sino que devuelven uno nuevo.

0.16.2 Crear fechas y acceder a sus partes

```
from datetime import date, time, datetime, timedelta

hoy = date.today()                       # fecha actual
ahora = datetime.now()                   # fecha y hora actuales

d = date(2026, 10, 19)                   # año, mes, día
t = time(9, 30)                          # hora, minuto (segundos = 0)
dt = datetime(2026, 10, 19, 9, 30, 15)   # año, mes, día, hora, minuto, segundo
```

```
print(datetime.combine(d, t))            # 2026-10-19 09:30:00

print(d.year, d.month, d.day)            # 2026 10 19
print(d.weekday())                       # 0 -> lunes (0 = lunes ... 6 = domingo)
print(d.isoweekday())                    # 1 -> lunes (1 = lunes ... 7 = domingo)
print(dt.date(), dt.time())              # 2026-10-19 09:30:15
print(dt.hour, dt.minute, dt.second)     # 9 30 15

print(dt.replace(day=1))                 # 2026-10-01 09:30:15 (nuevo objeto)
print(dt.replace(hour=0, minute=0, second=0))   # 2026-10-19 00:00:00
```

Crear una fecha imposible lanza `ValueError` (por ejemplo, `date(2026, 2, 31)`), lo que resulta muy útil para validar datos.

0.16.3 Operaciones: duraciones y comparaciones

Sumar o restar un `timedelta` a una fecha da otra fecha; restar dos fechas da un `timedelta`:

```
d = date(2026, 10, 19)
print(d + timedelta(days=30))            # 2026-11-18
print(d - timedelta(weeks=2))            # 2026-10-05

hasta_navidad = date(2026, 12, 25) - d
print(hasta_navidad)                     # 67 days, 0:00:00
print(hasta_navidad.days)                # 67

jornada = datetime(2026, 10, 19, 14, 0) - datetime(2026, 10, 19, 9, 30)
print(jornada)                           # 4:30:00
print(jornada.total_seconds() / 3600)    # 4.5 (horas)

print(date(2026, 1, 1) < d)              # True: las fechas se comparan
print(max(date(2025, 5, 1), d))          # 2026-10-19
```

Cuidado con los meses y los años. `timedelta` admite días, semanas, horas, minutos y segundos, pero **no meses ni años**, porque no duran siempre lo mismo. Tampoco conviene calcular la edad dividiendo los días entre 365, porque los años bisiestos acumulan error justo alrededor del cumpleaños:

```
nacimiento, hoy = date(2008, 10, 20), date(2026, 10, 19)
print((hoy - nacimiento).days // 365)    # 18  ¡incorrecto: cumple 18 mañana!

def edad(nacimiento: date, hoy: date) -> int:
    ya_cumplio = (hoy.month, hoy.day) >= (nacimiento.month, nacimiento.day)
    return hoy.year - nacimiento.year - (0 if ya_cumplio else 1)

print(edad(nacimiento, hoy))             # 17  correcto
```

0.16.4 De fecha a texto y de texto a fecha

Para mostrar una fecha con un formato concreto se usa el método `strftime` (*string format time*); para interpretar un texto como fecha, `datetime.strptime` (*string parse time*). Ambos usan los mismos códigos de formato:

Código	Significado	Ejemplo
`%d` / `%m` / `%Y`	Día / mes (2 dígitos) / año (4 dígitos)	`19, 10, 2026`
`%y`	Año con 2 dígitos	`26`
`%H` / `%M` / `%S`	Hora (00–23) / minutos / segundos	`09, 30, 15`
`%f`	Microsegundos	`000000`
`%A` / `%a`	Nombre del día / abreviado	`Monday, Mon`
`%B` / `%b`	Nombre del mes / abreviado	`October, Oct`
`%j`	Día del año (001–366)	`292`
`%z` / `%Z`	Desfase horario / nombre de la zona	`+0200, CEST`

```
dt = datetime(2026, 10, 19, 9, 30, 15)
print(dt.strftime("%d/%m/%Y %H:%M"))             # 19/10/2026 09:30
print(f"{dt:%d/%m/%Y}")                          # 19/10/2026 (lo mismo en una f-string
    ↪ )

leida = datetime.strptime("19/10/2026 09:30", "%d/%m/%Y %H:%M")
print(leida)                                     # 2026-10-19 09:30:00

try:
    datetime.strptime("31/02/2026", "%d/%m/%Y")
except ValueError as e:
    print("Fecha no válida:", e)                 # day is out of range for month
```

Nota. Los nombres de días y meses (`%A`, `%B`) salen en inglés salvo que se cambie la configuración regional con el módulo `locale`, cuyo resultado depende del sistema operativo. Para programas que deben funcionar igual en cualquier equipo, es más fiable usar listas propias de nombres en español.

0.16.5 ISO 8601: el formato para guardar e intercambiar fechas

El estándar **ISO 8601** escribe las fechas de mayor a menor: `2026-10-19` y `2026-10-19T09:30:15`. Es el formato que debes usar siempre que guardes o intercambies fechas como texto (ficheros CSV, JSON, XML, APIs), porque no es ambiguo (¿`05/04` es 5 de abril o 4 de mayo?), lo entienden todos los lenguajes y gestores y, además, al ordenar las cadenas alfabéticamente quedan en orden cronológico.

```
print(d.isoformat())                             # 2026-10-19
print(dt.isoformat())                            # 2026-10-19T09:30:15
print(date.fromisoformat("2026-10-19"))          # objeto date
print(datetime.fromisoformat("2026-10-19T09:30:00"))
```

0.16.6 Zonas horarias

Un `datetime` puede ser de dos tipos:

- **Ingenuo** (*naive*): no sabe a qué zona horaria pertenece. `datetime.now()` y `datetime(2026, 10, 19, 9, 30)` son ingenuos. Las 09:30, pero ¿de dónde?

- **Consciente** (*aware*): incluye su zona horaria en el atributo `tzinfo`, y por tanto representa un instante exacto.

Mientras un programa sólo trabaje en una zona, las fechas ingenuas son suficientes. Pero en cuanto los datos se comparten entre equipos, servidores o países (una base de datos en la nube, una API), hay que usar fechas conscientes. Las zonas horarias se obtienen con el módulo `zoneinfo`, que además aplica automáticamente los cambios de horario de verano e invierno:

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

madrid = ZoneInfo("Europe/Madrid")
reunion = datetime(2026, 10, 19, 9, 30, tzinfo=madrid)
print(reunion.isoformat())             # 2026-10-19T09:30:00+02:00 (horario de verano)
print(datetime(2026, 12, 19, 9, 30, tzinfo=madrid).isoformat())
                                       # 2026-12-19T09:30:00+01:00 (horario de invierno)

print(reunion.astimezone(timezone.utc))                    # 2026-10-19 07:30:00+00:00
print(reunion.astimezone(ZoneInfo("America/Mexico_City")))   # 2026-10-19 01:30:00-06:00

ahora_utc = datetime.now(timezone.utc)   # instante actual en UTC (consciente)
```

Cuidado.

- No se pueden comparar ni restar fechas ingenuas y conscientes: `reunion < datetime(2026, 10, 19, 9, 30)` lanza `TypeError: can't compare offset-naive and offset-aware datetimes`.
- La práctica recomendada es **guardar siempre en UTC** y convertir a la hora local sólo al mostrar los datos al usuario.
- En Windows, `zoneinfo` puede necesitar el paquete `tzdata` (`pip install tzdata`).

0.16.7 Medir duraciones

Para medir cuánto tarda una operación (algo que haremos constantemente al comparar versiones secuenciales, con procesos y con hilos) no se usa `datetime.now()`, sino `time.perf_counter()` del módulo `time`: devuelve un número de segundos con la máxima precisión disponible y no le afectan los cambios de la hora del sistema. Sólo tiene sentido la diferencia entre dos llamadas:

```
import time

inicio = time.perf_counter()
sum(range(1_000_000))
print(f"{time.perf_counter() - inicio:.3f} s")      # 0.015 s
```

0.16.8 Fechas en los protocolos y formatos del curso

Dónde	Cómo se representan las fechas
`logging` (apartado 0.18)	Cada mensaje lleva su fecha y hora local (`%(asctime)s`), con el formato que se indique en `datefmt`.
JSON y APIs REST (Tema 4)	No hay tipo fecha: se envían como **texto en ISO 8601**. Pydantic y FastAPI los convierten automáticamente en `date` y `datetime`.
Cabeceras HTTP (Tema 4)	Formato propio de la RFC 9110, siempre en GMT: `Mon, 19 Oct 2026 09:30:00 GMT`.
Tokens JWT (Tema 5)	La caducidad (`exp`) es una **marca de tiempo UNIX**: segundos transcurridos desde el 1 de enero de 1970 en UTC.
Certificados X.509 (Tema 5)	Periodo de validez en UTC; `cryptography` lo devuelve como `datetime` consciente (`not_valid_after_utc`).
SQLite (Tema 4)	No tiene tipo fecha: se guardan como texto ISO 8601.

```python
from datetime import datetime, timedelta, timezone
from email.utils import format_datetime, parsedate_to_datetime

ahora = datetime(2026, 10, 19, 9, 30, tzinfo=timezone.utc)

# Cabecera Date de HTTP
print(format_datetime(ahora, usegmt=True))          # Mon, 19 Oct 2026 09:30:00 GMT
print(parsedate_to_datetime("Mon, 19 Oct 2026 09:30:00 GMT"))  # 2026-10-19
    ↪ 09:30:00+00:00

# Marca de tiempo UNIX
marca = int(ahora.timestamp())
print(marca)                                        # 1792402200
print(datetime.fromtimestamp(marca, timezone.utc))  # 2026-10-19 09:30:00+00:00
caduca = ahora + timedelta(minutes=30)              # campo "exp" de un JWT
print(int(caduca.timestamp()))                      # 1792404000
```

0.17 Argumentos de línea de comandos

Muchos de los programas del curso serán **utilidades de terminal**: un servidor que se arranca en un puerto, un cliente que se conecta a una dirección, una herramienta que procesa ficheros en paralelo... Lo habitual es que reciban sus datos como **argumentos** al ejecutarlos:

```
$ python hashes.py descargas -a sha512 -o informe.json -p 8 --sobrescribir
```

0.17.1 `sys.argv`

Python guarda los argumentos en la lista `sys.argv`: el primer elemento es el nombre del programa y el resto, las palabras escritas detrás, siempre como cadenas.

```python
import sys
print(sys.argv)
```

```
# ['hashes.py', 'descargas', '-a', 'sha512', '-o', 'informe.json', '-p', '8', '--
    ↪ sobrescribir']
```

Procesar esta lista a mano es sencillo para un argumento, pero en cuanto hay opciones, valores por defecto, argumentos que pueden aparecer en cualquier orden o errores de escritura, se vuelve largo y propenso a fallos. Para eso existe el módulo `argparse` de la biblioteca estándar: describimos qué argumentos acepta el programa y él se encarga de leerlos, convertirlos, validarlos, mostrar errores claros y generar la ayuda.

0.17.2 Un ejemplo completo

Este programa es el esqueleto de la herramienta del proyecto del Tema 1, que calcula en paralelo la huella de todos los ficheros de un directorio:

```
# hashes.py
import argparse
from pathlib import Path

def crear_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="hashes",
        description="Calcula la huella de todos los ficheros de un directorio.")
    parser.add_argument("directorio", type=Path, help="directorio que se recorre")
    parser.add_argument("-a", "--algoritmo", choices=["sha256", "sha512", "blake2b"],
                        default="sha256", help="función resumen (por defecto: %(default)
    ↪ s)")
    parser.add_argument("-o", "--salida", type=Path, metavar="FICHERO",
                        help="guarda el informe en un fichero JSON")
    parser.add_argument("-p", "--procesos", type=int, default=4,
                        help="número de procesos trabajadores (por defecto: %(default)s)
    ↪ ")
    parser.add_argument("--sobrescribir", action="store_true",
                        help="sobrescribe el fichero de salida si ya existe")
    parser.add_argument("-v", "--verbose", action="count", default=0,
                        help="muestra más información (-v, -vv)")
    return parser

def main(argv: list[str] | None = None) -> int:
    parser = crear_parser()
    args = parser.parse_args(argv)          # None -> usa sys.argv
    if not args.directorio.is_dir():
        parser.error(f"no existe el directorio {args.directorio}")
    if args.procesos < 1:
        parser.error("el número de procesos debe ser al menos 1")
    if args.salida and args.salida.exists() and not args.sobrescribir:
        parser.error(f"{args.salida} ya existe (usa --sobrescribir)")
    if args.verbose >= 1:
        print(f"Recorriendo {args.directorio} con {args.procesos} procesos ({args.
    ↪ algoritmo})")
    # ... aquí iría el cálculo en paralelo (proyecto del Tema 1) ...
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

`parse_args()` devuelve un objeto con un atributo por argumento (`args.directorio`,

`args.procesos`...), ya convertido al tipo indicado. Los guiones del nombre largo se convierten en guiones bajos (`--max-clientes` → `args.max_clientes`).

0.17.3 Ayuda y errores automáticos

Sin escribir ni una línea más, el programa responde a `-h` o `--help`:

```
$ python hashes.py -h
usage: hashes [-h] [-a {sha256,sha512,blake2b}] [-o FICHERO] [-p PROCESOS]
              [--sobrescribir] [-v]
              directorio

Calcula la huella de todos los ficheros de un directorio.

positional arguments:
  directorio            directorio que se recorre

options:
  -h, --help            show this help message and exit
  -a, --algoritmo {sha256,sha512,blake2b}
                        función resumen (por defecto: sha256)
  -o, --salida FICHERO  guarda el informe en un fichero JSON
  -p, --procesos PROCESOS
                        número de procesos trabajadores (por defecto: 4)
  --sobrescribir        sobrescribe el fichero de salida si ya existe
  -v, --verbose         muestra más información (-v, -vv)
```

Y si el usuario se equivoca, muestra el uso correcto y un mensaje de error, y termina con el **código de salida 2** (el convenio para "uso incorrecto"):

```
$ python hashes.py
hashes: error: the following arguments are required: directorio
$ python hashes.py . -a md5
hashes: error: argument -a/--algoritmo: invalid choice: 'md5' (choose from 'sha256', '
    ↪ sha512', 'blake2b')
$ python hashes.py . -p dos
hashes: error: argument -p/--procesos: invalid int value: 'dos'
$ python hashes.py noexiste
hashes: error: no existe el directorio noexiste
```

(Antes de cada error se muestra también la línea *usage*, que aquí se ha omitido.) El último error lo genera nuestro propio código con `parser.error()`, que produce el mismo formato y el mismo código de salida. Los textos fijos (*usage*, *options*...) aparecen en inglés, y su formato exacto puede variar ligeramente entre versiones de Python.

0.17.4 Tipos de argumentos y opciones de `add_argument`

- **Posicionales** (`"entrada"`): obligatorios, se identifican por su posición.
- **Opcionales** (`"-o", "--salida"`): empiezan por guiones, pueden ir en cualquier orden y tener forma corta y larga. Son opcionales salvo que se indique `required=True`.

Parámetro	Para qué sirve
`help`	Texto que aparece en la ayuda. `%(default)s` se sustituye por el valor por defecto.
`type`	Función que convierte el texto: `int`, `float`, `Path`… Si falla, se muestra un error.
`default`	Valor si el argumento no aparece (si no se indica, `None`).
`required`	Hace obligatorio un argumento opcional.
`choices`	Lista de valores permitidos.
`metavar`	Nombre del valor en la ayuda (`FICHERO` en lugar de `SALIDA`).
`dest`	Nombre del atributo en `args`, si no se quiere el deducido.
`action="store_true"`	Interruptor sin valor: `True` si aparece, `False` si no.
`action="count"`	Cuenta las repeticiones: `-v` = 1, `-vv` = 2.
`action="append"`	Opción repetible: acumula los valores en una lista.
`nargs`	Número de valores: `"?"` (0 o 1), `"*"` (0 o más), `"+"` (1 o más) o un número.

```
parser = argparse.ArgumentParser(prog="descargar")
parser.add_argument("urls", nargs="+", help="una o más URL")
parser.add_argument("-H", "--cabecera", action="append", default=[],
                    help="cabecera HTTP extra (se puede repetir)")

grupo = parser.add_mutually_exclusive_group()     # opciones incompatibles entre sí
grupo.add_argument("-q", "--silencioso", action="store_true")
grupo.add_argument("-v", "--verbose", action="store_true")

args = parser.parse_args(["https://a.es", "https://b.es",
                          "-H", "Accept: text/html", "-H", "User-Agent: dam", "-q"])
print(args)
# Namespace(urls=['https://a.es', 'https://b.es'],
#           cabecera=['Accept: text/html', 'User-Agent: dam'], silencioso=True, verbose=
    ↪ False)
# descargar https://a.es -q -v  ->  error: argument -v/--verbose: not allowed with
    ↪ argument -q/--silencioso
```

0.17.5 Subcomandos

Cuando un programa hace varias cosas distintas, lo más claro es organizarlo en **subcomandos**, como `git add` y `git commit` o `pip install` y `pip list`. Cada subcomando tiene sus propios argumentos. Por ejemplo, un mismo programa de chat puede arrancarse como servidor o como cliente:

```
# chat.py
import argparse

def crear_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="chat", description="Chat en red local")
    parser.add_argument("-p", "--puerto", type=int, default=5000, help="puerto TCP")
    sub = parser.add_subparsers(dest="orden", required=True, metavar="ORDEN")

    servidor = sub.add_parser("servidor", help="arranca el servidor")
```

```
    servidor.add_argument("--host", default="0.0.0.0", help="interfaz de escucha")
    servidor.add_argument("--max-clientes", type=int, default=20)

    cliente = sub.add_parser("cliente", help="se conecta a un servidor")
    cliente.add_argument("host", help="dirección del servidor")
    cliente.add_argument("--nick", required=True, help="nombre en el chat")
    return parser

def main(argv: list[str] | None = None) -> int:
    args = crear_parser().parse_args(argv)
    match args.orden:
        case "servidor":
            print(f"Servidor en {args.host}:{args.puerto} (máx. {args.max_clientes}
    ↪ clientes)")
        case "cliente":
            print(f"Conectando a {args.host}:{args.puerto} como {args.nick}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

```
$ python chat.py servidor --max-clientes 5
Servidor en 0.0.0.0:5000 (máx. 5 clientes)
$ python chat.py -p 6000 cliente 192.168.1.20 --nick ana
Conectando a 192.168.1.20:6000 como ana
$ python chat.py cliente -h
usage: chat cliente [-h] --nick NICK host
...
```

Fíjate en que las opciones globales (`-p`) van antes del subcomando, y las de cada subcomando, detrás.

0.17.6 Estructura recomendada y pruebas

En los ejemplos anteriores, el programa está organizado en una función `main(argv=None)` que devuelve el código de salida:

- Al ejecutarlo desde la terminal, `argv` vale `None` y `parse_args` lee `sys.argv`.
- En una prueba, se le pasa la lista de argumentos directamente, sin necesidad de lanzar un proceso.
- `raise SystemExit(main())` hace que el programa termine con el código que devuelve `main`: 0 si todo ha ido bien y otro valor si ha habido un error, lo que permite usarlo en *scripts* y tareas programadas.

```
# test_hashes.py
import pytest
from hashes import main

def test_algoritmo_no_valido(tmp_path, capsys):
    with pytest.raises(SystemExit) as e:      # argparse termina el programa
        main([str(tmp_path), "-a", "md5"])
    assert e.value.code == 2
    assert "invalid choice" in capsys.readouterr().err
```

```python
def test_ejecucion_correcta(tmp_path, capsys):
    (tmp_path / "a.txt").write_text("hola", encoding="utf-8")
    assert main([str(tmp_path), "-p", "2", "-v"]) == 0
    assert "2 procesos" in capsys.readouterr().out
```

El Anexo B explica las pruebas con `pytest` y las *fixtures* `tmp_path` y `capsys`.

0.18 Registro de eventos y depuración

Un **registro** (*log*) es la lista de eventos que un programa va anotando mientras se ejecuta: que ha arrancado, que se ha conectado un cliente, que ha fallado una operación... En un servidor que funciona día y noche sin que nadie lo mire, el registro es la única forma de saber qué ha pasado; en un programa con varios hilos o procesos, es la herramienta de depuración más útil, porque el depurador altera el orden de ejecución.

¿Por qué no basta con `print`?

- `print` no indica **cuándo** ocurrió algo, **dónde** (módulo, función, línea) ni **quién** (hilo, proceso).
- No distingue la **gravedad**: un mensaje de depuración y un error grave se ven igual.
- No se puede **redirigir** (a un fichero, a la consola, a un servidor de registros) ni **filtrar** sin cambiar el código.
- Para «quitar los mensajes» hay que borrarlos o comentarlos. Con `logging` basta con cambiar el nivel.

0.18.1 Niveles

Cada mensaje tiene un nivel de gravedad. Se registran sólo los mensajes cuyo nivel es igual o superior al configurado:

Nivel	Valor	Cuándo usarlo
`DEBUG`	10	Detalles para depurar: valores de variables, mensajes recibidos byte a byte. Desactivado en producción.
`INFO`	20	Funcionamiento normal: servidor iniciado, cliente conectado, tarea completada.
`WARNING`	30	Algo inesperado que no impide seguir: reintento de conexión, configuración por defecto, disco casi lleno. **Es el nivel por defecto.**
`ERROR`	40	Una operación ha fallado: no se pudo atender una petición, fichero ilegible.
`CRITICAL`	50	El programa no puede continuar: no se puede abrir el puerto, falta la base de datos.

0.18.2 Primer uso: `basicConfig`

```python
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s [%(levelname)s] (%(threadName)s) %(message)s",
    datefmt="%H:%M:%S",
```

```
    # filename="app.log", encoding="utf-8",   # descomentar para ir a fichero
)
log = logging.getLogger(__name__)

log.debug("detalle para depurar")
log.info("servidor iniciado en el puerto %d", 5000)
log.warning("cliente desconectado bruscamente")
log.error("no se pudo abrir el fichero")
log.critical("no hay espacio en disco")
```

```
10:15:02 [DEBUG] (MainThread) detalle para depurar
10:15:02 [INFO] (MainThread) servidor iniciado en el puerto 5000
10:15:02 [WARNING] (MainThread) cliente desconectado bruscamente
10:15:02 [ERROR] (MainThread) no se pudo abrir el fichero
10:15:02 [CRITICAL] (MainThread) no hay espacio en disco
```

`basicConfig` sólo tiene efecto la primera vez que se llama, y debe hacerse una única vez, al principio del programa principal (nunca en un módulo que vaya a ser importado).

0.18.3 El formato del mensaje

La cadena `format` admite estos atributos, entre otros:

`%(asctime)s`	Fecha y hora (su formato se cambia con `datefmt`).
`%(levelname)s`	Nivel del mensaje. `%(levelname)-8s` lo alinea.
`%(name)s`	Nombre del *logger* que generó el mensaje.
`%(message)s`	El mensaje.
`%(module)s,%(funcName)s,%(lineno)d`	Módulo, función y línea desde donde se registró.
`%(threadName)s,%(thread)d`	Nombre e identificador del hilo.
`%(processName)s,%(process)d`	Nombre y PID del proceso.

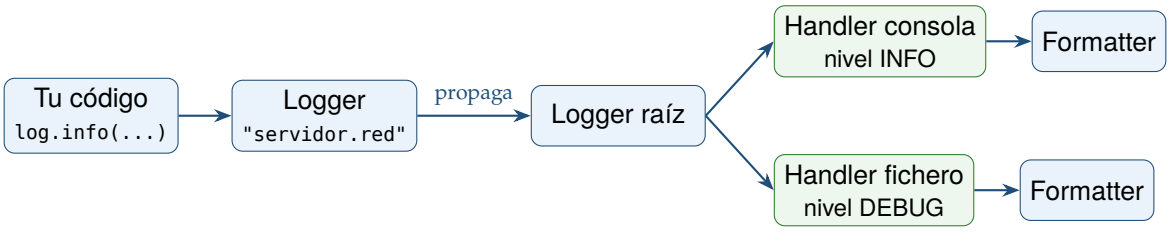
Pasa los datos como argumentos, no en una f-string. Escribe `log.debug("recibido %r de %s", datos, cliente)` y no `log.debug(f"recibido {datos!r} de {cliente}")`. Con la primera forma, el texto sólo se construye si el mensaje se va a registrar (si el nivel es `INFO`, los `debug` no cuestan nada), y las herramientas de análisis pueden agrupar los mensajes iguales. Además, `%r` muestra los `\n` y los bytes tal cual, algo muy útil al depurar protocolos de red.

0.18.4 Registrar excepciones

Dentro de un bloque `except`, `log.exception()` registra el mensaje con nivel `ERROR` **y la traza completa** de la excepción, que es justo lo que necesitaremos para encontrar el fallo:

```
try:
    media = sum(notas) / len(notas)
except ZeroDivisionError:
    log.exception("fallo al calcular la media")   # incluye la traza
    media = 0.0
# Con otro nivel:  log.warning("...", exc_info=True)
```

0.18.5 Cómo está organizado `logging`



Logger
: Objeto con el que el código registra los mensajes. Se obtiene con `logging.getLogger(nombre ↪ )`. Los nombres forman una **jerarquía** separada por puntos: `"servidor.red"` es hijo de `"servidor"`, que es hijo del *logger raíz*. Un mensaje **se propaga** hacia arriba, de modo que basta con configurar la raíz (o `"servidor"`) para recibir los mensajes de todos sus hijos.

Handler
: Decide **a dónde** va el mensaje: consola (`StreamHandler`), fichero (`FileHandler`), fichero con rotación (`RotatingFileHandler`, `TimedRotatingFileHandler`), correo (`SMTPHandler`), red (`SocketHandler`, `HTTPHandler`), una cola (`QueueHandler`)... Un logger puede tener varios, y cada uno su propio nivel.

Formatter
: Decide **cómo** se escribe el mensaje.

Filter
: Permite descartar o enriquecer mensajes con criterios propios.

> **Un logger por módulo.** La convención es crear en cada fichero un logger con el nombre del módulo: `log = logging.getLogger(__name__)`. Así cada mensaje indica de qué módulo procede y se puede ajustar el nivel de cada parte del programa por separado. Los módulos sólo *usan* su logger; la *configuración* (niveles, handlers, formato) se hace una sola vez en el programa principal.

0.18.6 Varios destinos y rotación de ficheros

Un caso muy habitual en un servidor: en pantalla, sólo lo importante; en un fichero, todo. Para que el fichero no crezca sin límite, se usa un `RotatingFileHandler`: cuando llega a un tamaño, lo renombra (`app.log.1`, `app.log.2`...) y empieza uno nuevo, conservando un número máximo de copias.

```
# registro.py: configuración reutilizable
import logging
import sys
from logging.handlers import RotatingFileHandler

def configurar_logging(fichero: str = "app.log") -> None:
    formato = logging.Formatter(
        "%(asctime)s %(levelname)-8s %(name)s [%(threadName)s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    consola = logging.StreamHandler(sys.stdout)
    consola.setLevel(logging.INFO)              # en pantalla, sólo lo importante
    consola.setFormatter(formato)
```

```
    archivo = RotatingFileHandler(fichero, maxBytes=1_000_000, backupCount=5,
                                  encoding="utf-8")
    archivo.setLevel(logging.DEBUG)              # en el fichero, todo
    archivo.setFormatter(formato)

    raiz = logging.getLogger()                   # logger raíz
    raiz.setLevel(logging.DEBUG)
    raiz.addHandler(consola)
    raiz.addHandler(archivo)
    logging.getLogger("urllib3").setLevel(logging.WARNING)   # menos ruido

if __name__ == "__main__":
    configurar_logging()
    log = logging.getLogger("servidor")
    log.debug("sólo en el fichero")
    log.info("en pantalla y en el fichero")
```

La última línea de `configurar_logging` muestra otra ventaja de la jerarquía: las librerías externas también usan `logging`, y podemos subir el nivel de una concreta (aquí, la que usa `requests` por debajo) para que no llene el registro de mensajes de depuración.

0.18.7 Configuración con un diccionario

En aplicaciones medianas, la configuración se describe como datos en un diccionario, que se aplica con la función `dictConfig` del módulo `logging.config`. El diccionario puede cargarse de un fichero JSON o TOML, de modo que se puede cambiar el registro sin tocar el código:

```
import logging.config

CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "simple": {"format": "%(levelname)s %(name)s: %(message)s"},
        "detallado": {"format": "%(asctime)s %(levelname)s %(name)s "
                                "%(funcName)s:%(lineno)d %(message)s"},
    },
    "handlers": {
        "consola": {"class": "logging.StreamHandler", "level": "INFO",
                    "formatter": "simple"},
        "archivo": {"class": "logging.handlers.RotatingFileHandler",
                    "filename": "app.log", "maxBytes": 1_000_000,
                    "backupCount": 3, "encoding": "utf-8",
                    "formatter": "detallado"},
    },
    "loggers": {
        "servidor": {"level": "DEBUG", "handlers": ["consola", "archivo"]},
    },
}

logging.config.dictConfig(CONFIG)
logging.getLogger("servidor.red").info("se propaga al logger 'servidor'")
```

0.18.8 Logging con hilos y procesos

- **Hilos**: `logging` es seguro con hilos. Varios hilos pueden registrar a la vez en el mismo logger y los mensajes no se mezclan. Añade `%(threadName)s` al formato y da nombres útiles a los hilos.
- **Procesos**: varios procesos escribiendo **en el mismo fichero** sí pueden mezclar o perder líneas, y cada proceso hijo tiene su propia configuración. La solución recomendada es que los hijos envíen sus mensajes a una cola (`QueueHandler`) y que un único oyente en el proceso principal (`QueueListener`) los escriba.

```python
# Registro centralizado desde varios procesos
import logging
import logging.handlers
import multiprocessing as mp

def trabajador(cola, n: int) -> None:
    # En el hijo: todo lo que se registre se envía a la cola
    raiz = logging.getLogger()
    raiz.handlers.clear()
    raiz.addHandler(logging.handlers.QueueHandler(cola))
    raiz.setLevel(logging.DEBUG)
    logging.getLogger("calculo").info("procesando el bloque %d", n)

if __name__ == "__main__":
    cola = mp.Queue()
    destino = logging.FileHandler("procesos.log", encoding="utf-8")
    destino.setFormatter(logging.Formatter(
        "%(asctime)s %(processName)s (%(process)d) %(levelname)s %(message)s"))
    # El oyente, en el proceso principal, es el único que escribe en el fichero
    oyente = logging.handlers.QueueListener(cola, destino)
    oyente.start()

    procesos = [mp.Process(target=trabajador, args=(cola, i)) for i in range(4)]
    for p in procesos: p.start()
    for p in procesos: p.join()
    oyente.stop()
```

Entenderás del todo este ejemplo al terminar el Tema 1.

0.18.9 Buenas prácticas

- En servidores y programas concurrentes, `logging` siempre; `print` sólo para la salida que el usuario debe ver.
- Un logger por módulo con `__name__`; configuración en un único sitio.
- Elige bien el nivel: si todo es `ERROR`, nada lo es.
- Incluye el **contexto** necesario para entender el mensaje sin el código delante: qué cliente, qué fichero, qué identificador. `"Error"` no sirve; `"No se pudo leer pedidos.csv: permiso denegado"` sí.
- **Nunca registres datos sensibles**: contraseñas, tokens, claves, números de tarjeta o datos personales (lo veremos en el Tema 5).
- Usa `log.exception()` en los `except` en lugar de tragarte las excepciones en silencio.

- En sistemas grandes, los registros se escriben en formato estructurado (por ejemplo, una línea JSON por evento) y se centralizan en herramientas como Grafana Loki o Elasticsearch para buscar en ellos.

0.18.10 El depurador

VS Code permite poner **puntos de ruptura** haciendo clic a la izquierda del número de línea y ejecutar con `F5`. Puedes inspeccionar variables, avanzar línea a línea y, muy importante en este módulo, ver la lista de hilos activos en el panel *Call Stack*. Desde el código también puedes detener la ejecución con `breakpoint()`.

El depurador desde la terminal: `pdb`

Si no tienes un editor a mano (por ejemplo, en un servidor), el depurador de la biblioteca estándar, `pdb`, se usa desde la terminal. Se activa ejecutando el programa con `python -m pdb programa.py` o, mejor, escribiendo `breakpoint()` en la línea donde quieras detenerte. En la consola del depurador se usan estas órdenes:

Orden	Qué hace
`n` (*next*)	Ejecuta la línea actual sin entrar en las funciones.
`s` (*step*)	Ejecuta la línea actual entrando en las funciones.
`c` (*continue*)	Continúa hasta el siguiente punto de ruptura.
`p expresión`	Muestra el valor de una expresión (`p datos`, `p len(cola)`).
`l` (*list*)	Muestra el código alrededor de la línea actual.
`w` (*where*)	Muestra la pila de llamadas.
`q` (*quit*)	Termina el programa.

Depurar programas concurrentes. Un depurador detiene el programa, y eso cambia el orden en que se intercalan los hilos y procesos: un error de concurrencia puede desaparecer precisamente al depurarlo. Por eso, en los temas 1 y 2, la herramienta principal será el **registro de eventos** con el nombre del hilo o del proceso en cada mensaje. Si un programa se queda colgado (por ejemplo, por un interbloqueo), el módulo `faulthandler` muestra en qué línea está cada hilo (Tema 2). Las pruebas automáticas y la documentación del código se explican en el Anexo B.

0.19 Ejercicios

Ejercicio 0.1 — Calificaciones

Dificultad: ★

Escribe un programa que pida por teclado notas numéricas hasta que el usuario introduzca una línea vacía. Si una entrada no es un número o no está entre 0 y 10, muestra un aviso y sigue pidiendo. Al terminar, muestra cuántas notas se han introducido, la media, la máxima, la mínima y cuántas hay de cada calificación (Suspenso, Aprobado, Notable, Sobresaliente) usando un diccionario.

Ejercicio 0.2 — Comprensiones

Dificultad: ★

Dada la lista `palabras = ["python", "Java", "sql", "XML", "json", "Kotlin"]`, obtén usando comprensiones:

a) Una lista con las palabras en minúsculas.

b) Un diccionario *palabra* → *longitud*.

c) Un conjunto con las letras iniciales (en mayúsculas).

d) La lista de palabras con más de cuatro letras, ordenada alfabéticamente sin distinguir mayúsculas.

Ejercicio 0.3 — Bytes

Dificultad: ★

Escribe una función que reciba una cadena y muestre, para cada carácter, su representación en bytes UTF-8 en hexadecimal. Pruébala con `"año"`, `"€"` y un emoji. ¿Cuántos bytes ocupa cada uno?

Ejercicio 0.4 — Intérprete de órdenes

Dificultad: ★★

Usando `match`, programa un bucle que lea órdenes del teclado hasta que se escriba `SALIR`. Debe soportar `SUMA a b`, `MAYUS texto...`, `INVERTIR texto...` y `HORA`. Las órdenes incorrectas deben capturarse con excepciones y mostrar un mensaje de error, sin que el programa termine.

Ejercicio 0.5 — Funciones y excepciones

Dificultad: ★★

Implementa una función `validar_dni(dni: str) -> str` que reciba un DNI (8 dígitos y una letra), compruebe que el formato es correcto y que la letra coincide con la calculada (`"TRWAGMYFPDXBNJZSQVHLCKE"[numero % 23]`). Si es válido, lo devuelve normalizado en mayúsculas; si no, lanza una excepción propia `DNIInvalido` con un mensaje que explique el problema. Escribe un pequeño programa que pruebe la función con varios DNI correctos e incorrectos.

Ejercicio 0.6 — Clases

Dificultad: ★★

Crea una clase `CuentaBancaria` con titular, IBAN y saldo. El saldo debe ser una propiedad de sólo lectura; las operaciones `ingresar` y `retirar` lanzarán `ValueError` si la cantidad no es positiva o si no hay saldo suficiente. Añade un historial de movimientos (fecha, tipo, cantidad) e implementa `__str__` y `__len__` (número de movimientos). Crea después una subclase `CuentaAhorro` con un tipo de interés y un método `aplicar_intereses()`.

Ejercicio 0.7 — Atributos y visibilidad

Dificultad: ★

Crea una clase `Pedido` con un atributo de clase que cuente los pedidos creados y genere un identificador único para cada uno, un atributo interno `_lineas` (lista de productos y pre-

cios) y una propiedad de sólo lectura `total`. Añade los métodos públicos `añadir_linea()` y `quitar_linea()`, que usen un método interno `_comprobar_precio()`. Explica en un comentario qué ocurriría si `_lineas` fuera un atributo de clase.

Ejercicio 0.8 — Dataclasses y ordenación

Dificultad: ★★

Define una *dataclass* `Pelicula` (título, director, año, duración en minutos, géneros). Crea una lista con al menos ocho películas y escribe funciones que:

a) Devuelvan las películas de un género dado, ordenadas por año.

b) Calculen la duración media por director (diccionario).

c) Devuelvan, mediante un generador, las películas que duran más de un valor dado.

Ejercicio 0.9 — Configuración validada

Dificultad: ★★

Un servidor lee su configuración de un fichero `config.json` con el host, el puerto (1024–65535), el número máximo de clientes (1–1000), el nivel de registro (sólo `DEBUG`, `INFO`, `WARNING` o `ERROR`) y una lista de direcciones IP bloqueadas. Define el modelo con `BaseModel` (usa `Literal` y `Field`) y escribe una función `cargar_configuracion(ruta) -> Configuracion` que muestre todos los errores de forma clara si el fichero no es válido. Anota todo el código y comprueba que `mypy --strict` no da ningún aviso. Repite el modelo con una `dataclass` normal y compara qué errores se detectan en cada caso.

Ejercicio 0.10 — Fechas

Dificultad: ★★

Escribe un módulo `fechas_util.py` con estas funciones y pruébalas con `pytest`:

a) `edad(nacimiento, hoy=None) -> int`, correcta también el día anterior y el día del cumpleaños, y para nacidos un 29 de febrero.

b) `leer_fecha(texto) -> date`, que acepte los formatos `19/10/2026`, `19-10-2026` y `2026-10-19` y lance `ValueError` con un mensaje claro si el texto no es una fecha válida.

c) `dias_laborables(inicio, fin) -> int`, que cuente los días de lunes a viernes entre dos fechas (ambas incluidas).

d) `a_utc(texto, zona="Europe/Madrid") -> datetime`, que reciba una fecha y hora local como `"25/03/2026 10:00"` y devuelva el instante equivalente en UTC. Comprueba el resultado antes y después del cambio de horario de marzo.

Ejercicio 0.11 — Utilidad de línea de comandos

Dificultad: ★★

Usando la clase `Pelicula` del ejercicio *Dataclasses y ordenación*, escribe un programa `peliculas.py` con `argparse` y tres subcomandos: `listar` (con opciones `--genero`, `--desde AÑO` y `--orden` con los valores `titulo`, `anio` o `duracion`), `buscar TEXTO` (busca en el título sin distinguir mayúsculas) y `estadisticas` (número de películas y duración media por director). Añade una opción global `-v` que se pueda repetir para mostrar más detalle. Or-

ganízalo con una función `main(argv=None)` y escribe al menos tres pruebas con `pytest`, una de ellas para un argumento no válido.

Ejercicio 0.12 — Registro de un programa

Dificultad: ★★

Crea un módulo `registro.py` con una función `configurar_logging(nivel_consola, fichero)` que envíe a la consola los mensajes del nivel indicado y a un fichero con rotación (máximo 100 KiB y 3 copias) todos los mensajes, con fecha, nivel, módulo, función y línea. Úsalo desde un programa con dos módulos que registren mensajes de todos los niveles y una excepción con `log.exception`. Escribe un bucle que genere mensajes suficientes para comprobar que la rotación funciona y verifica que cambiando sólo el nivel de la consola desaparecen los mensajes de depuración.

Ejercicio 0.13 — Gestor de contexto y decorador

Dificultad: ★★★

a) Escribe un decorador `@reintentar(veces=3)` que vuelva a ejecutar la función decorada si lanza una excepción, hasta el número de veces indicado, y relance la última excepción si todos los intentos fallan. (Pista: un decorador con parámetros es una función que devuelve un decorador.)

b) Escribe un gestor de contexto `directorio_temporal()` que cree una carpeta temporal, la devuelva para usarla dentro del `with` y la borre al salir. Puedes usar `tempfile.mkdtemp` y `shutil.rmtree`.

Proyecto: Gestor de tareas en línea de comandos

Desarrolla una aplicación de consola para gestionar una lista de tareas pendientes que se guarde en un fichero JSON. La reutilizaremos como base de varias prácticas del curso (por ejemplo, para ofrecerla como servicio web en el Tema 4).

Requisitos:

1. Un paquete `tareas` con, al menos, los módulos `modelo.py`, `almacen.py` y `main.py`, ejecutable con `python -m tareas.main`.
2. En `modelo.py`, una *dataclass* `Tarea` (identificador, título, prioridad, fecha de creación, fecha límite opcional, estado y etiquetas). La prioridad y el estado serán enumeraciones (`Enum`).
3. En `almacen.py`, una clase `Almacen` que lea y escriba el fichero con `with` y `json` (fechas en ISO 8601) y ofrezca métodos para añadir, buscar por identificador, buscar por texto, completar, borrar y listar ordenando por cualquier campo. Si el fichero no existe se crea; si está corrupto, se informa con un error claro.
4. Excepciones propias: `TareaNoEncontrada` y `AlmacenCorrupto`.
5. En `main.py`, subcomandos de `argparse`: `añadir`, `listar` (con filtros por prioridad, estado y etiqueta), `completar ID`, `borrar ID` y `vencidas` (tareas pendientes cuya fecha límite ya ha pasado). Opción global `--fichero` y `-v` repetible.
6. Registro de todas las operaciones con `logging` en un fichero `tareas.log` con rotación; por pantalla, sólo los avisos y errores (o todo, con `-v`).

7. El programa nunca termina con una traza de error: los errores previstos se muestran al usuario con un mensaje claro y un código de salida distinto de 0.
8. Anotaciones de tipo en todo el código (`mypy --strict` sin errores), *docstrings* y al menos cinco pruebas con `pytest` (Anexo B).

Entrega: el código del paquete, las pruebas y un fichero `README.md` con instrucciones para ejecutarlo.
Ampliación: añade una orden `exportar` que genere un informe en Markdown con las tareas agrupadas por prioridad.

TEMA 1

Programación multiproceso

En este tema aprenderás a…

- Distinguir entre programa, proceso y servicio, y conocer cómo gestiona el sistema operativo los procesos (estados y planificación).
- Diferenciar la programación concurrente, paralela y distribuida, y los procesos de los hilos.
- Monitorizar y gestionar procesos desde el sistema operativo y desde Python con `psutil`.
- Lanzar programas externos con `subprocess` y comunicarte con ellos.
- Crear procesos con `multiprocessing`, compartir información entre ellos y sincronizarlos.
- Repartir un cálculo entre varios procesos con `Pool` y `ProcessPoolExecutor` y recoger sus resultados.
- Depurar y documentar aplicaciones compuestas por varios procesos.

Resultado de aprendizaje asociado: **RA1**.

1.1 Ejecutables, procesos y servicios

Programa o ejecutable
Fichero almacenado en disco que contiene instrucciones (código máquina, o código intermedio que interpreta otro programa, como un `.py` o un `.jar`). Es algo **estático**: no hace nada por sí mismo.

Proceso
Un programa **en ejecución**. Además del código, incluye todo lo necesario para ejecutarlo: su propio espacio de memoria (código, datos, pila y *heap*), los valores de los registros del procesador, los ficheros abiertos, etc. Un mismo programa puede dar lugar a varios procesos simultáneos (por ejemplo, dos ventanas independientes de un editor).

Servicio (o demonio)
Proceso que se ejecuta en segundo plano, normalmente sin interfaz, y que suele arrancar con el sistema para ofrecer una funcionalidad de forma continua: un servidor web, una base de datos, el servicio de impresión...En Linux se gestionan con `systemd` (`systemctl status ssh`) y en Windows desde la consola *Servicios* (`services.msc`).

1.1.1 El bloque de control de proceso

Para gestionar cada proceso, el sistema operativo mantiene una estructura llamada **PCB** (*Process Control Block*) con:

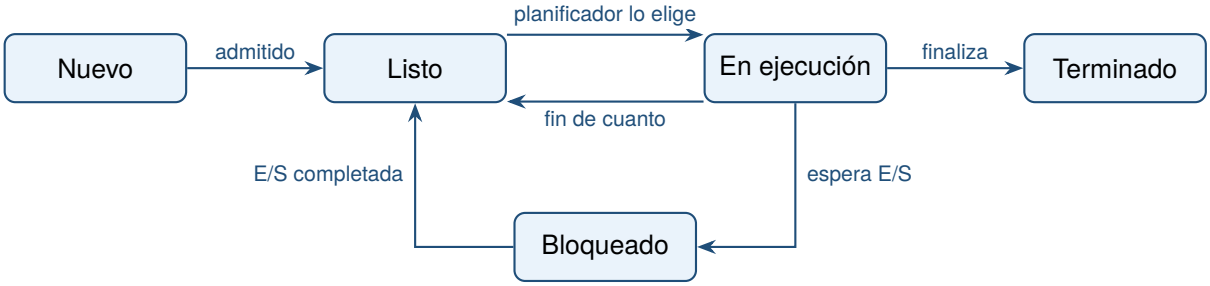
- **Identificador** (PID) y el del proceso padre (PPID).

- **Estado** actual (listo, en ejecución, bloqueado…).
- **Contador de programa** y resto de registros del procesador.
- Información de **planificación**: prioridad, tiempo consumido.
- Información de **memoria** asignada.
- Recursos: ficheros abiertos, conexiones, usuario propietario…

Los procesos forman un **árbol**: todo proceso es creado por otro (su padre). En Linux puedes verlo con `pstree`. Cuando un programa Python crea un proceso, el intérprete es el padre y el nuevo proceso el hijo.

1.2 Estados de un proceso y planificación

Un procesador (un núcleo) sólo puede ejecutar **una instrucción de un proceso a la vez**. Sin embargo, en tu ordenador hay cientos de procesos «funcionando a la vez». El responsable de repartir el procesador entre todos ellos es el **planificador** (*scheduler*) del sistema operativo. Para ello, cada proceso pasa por distintos estados:



- **Nuevo**: el proceso se está creando.
- **Listo**: puede ejecutarse; espera a que le asignen un núcleo.
- **En ejecución**: está usando el procesador.
- **Bloqueado**: espera a que ocurra algo (leer del disco, recibir datos de la red, que el usuario pulse una tecla…). Aunque le den el procesador, no podría avanzar.
- **Terminado**: ha finalizado; el SO libera sus recursos.

Cuando el planificador quita el procesador a un proceso para dárselo a otro se produce un **cambio de contexto**: el SO guarda el estado (registros, contador de programa) del proceso saliente en su PCB y carga el del entrante. Es una operación costosa, por lo que no conviene que ocurra demasiado a menudo.

1.2.1 Algoritmos de planificación

Algoritmo	Funcionamiento
FCFS (*First Come, First Served*)	El primero que llega es el primero que se ejecuta, hasta que termina o se bloquea. Sencillo, pero un proceso largo retrasa a todos.
SJF (*Shortest Job First*)	Se elige el proceso con menor tiempo de ejecución estimado. Minimiza el tiempo medio de espera, pero puede provocar *inanición* de los procesos largos.
Round Robin	Cada proceso recibe un *cuanto* de tiempo (pocos milisegundos); al agotarlo vuelve al final de la cola de listos. Es la base de los sistemas de tiempo compartido.
Por prioridades	Se ejecuta el proceso de mayor prioridad. Para evitar la inanición se aplica *envejecimiento*: la prioridad aumenta con el tiempo de espera.
Colas multinivel	Varias colas con distintas prioridades y algoritmos. Es lo que usan, con variantes, Linux, Windows y macOS.

El planificador de Linux. Desde la versión 6.6 del núcleo, Linux usa el planificador EEVDF (*Earliest Eligible Virtual Deadline First*), sucesor del CFS. La prioridad de un proceso normal se expresa con el valor **nice**, de -20 (máxima prioridad) a 19 (mínima). Puedes lanzar un programa con baja prioridad con `nice -n 10 python script.py`.

1.3 Hilos

Un **hilo** (*thread*) es una secuencia de ejecución *dentro* de un proceso. Todo proceso tiene al menos un hilo (el hilo principal), y puede crear más. Los hilos de un mismo proceso **comparten la memoria** y los recursos del proceso, pero cada uno tiene su propia pila y sus propios registros.

	Procesos	Hilos
Memoria	Cada uno la suya, aislada	Compartida dentro del proceso
Creación	Costosa (copiar o crear espacio de memoria)	Ligera
Comunicación	Mediante mecanismos del SO (tuberías, colas, sockets…)	Directa, a través de variables compartidas
Robustez	Si uno falla, los demás siguen	Un fallo grave puede tumbar todo el proceso
Sincronización	Necesaria sólo para recursos compartidos explícitamente	Imprescindible casi siempre

Los hilos se estudian a fondo en el Tema 2.

1.4 Programación concurrente, paralela y distribuida

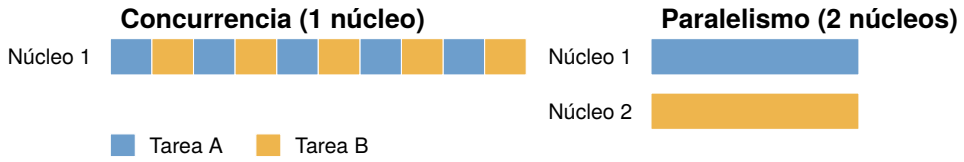
1.4.1 Programación concurrente

Un programa es **concurrente** cuando está formado por varias tareas que *progresan* durante el mismo intervalo de tiempo, sin que ninguna tenga que esperar a que termine otra. No implica que se ejecuten en el mismo instante: en un único núcleo, el SO las alterna tan rápido que parece que van a la vez.

Ámbitos de aplicación típicos:

- **Interfaces gráficas**: la interfaz debe seguir respondiendo mientras se descarga un fichero o se hace un cálculo largo.
- **Servidores**: atender a muchos clientes simultáneamente.
- **Cálculo intensivo**: repartir un problema grande entre varios núcleos para terminar antes.
- **Sistemas de control y tiempo real**: leer sensores mientras se actúa sobre motores.
- **Videojuegos**: física, sonido, red y renderizado en paralelo.

1.4.2 Concurrencia frente a paralelismo



- **Concurrencia**: las tareas se *entrelazan*. Es una forma de *estructurar* el programa.
- **Paralelismo**: las tareas se ejecutan *literalmente al mismo tiempo* en núcleos o procesadores distintos. Requiere hardware con varios núcleos.

1.4.3 Programación paralela y distribuida

	Paralela	**Distribuida**
Hardware	Un ordenador con varios núcleos/procesadores	Varios ordenadores conectados por red
Memoria	Normalmente compartida	Cada nodo tiene la suya
Comunicación	Memoria compartida, colas, tuberías	Paso de mensajes por la red (sockets, HTTP, colas de mensajes)
Ventajas	Comunicación muy rápida; más sencilla de programar	Escalabilidad casi ilimitada; tolerancia a fallos; los nodos pueden estar en lugares distintos
Inconvenientes	Limitada por el número de núcleos de la máquina	Latencia de red; fallos parciales; mucho más compleja de programar y depurar
Ejemplos	Renderizar un vídeo con todos los núcleos	Una red de microservicios, un clúster de cálculo, BitTorrent, la web

La ley de Amdahl. Si sólo una fracción p de un programa se puede paralelizar, la mejora

máxima con n núcleos es

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

Por ejemplo, si el 90 % es paralelizable ($p = 0{,}9$), con infinitos núcleos el programa nunca irá más de 10 veces más rápido. La parte secuencial acaba siendo el cuello de botella.

1.5 Gestión de procesos y herramientas de monitorización

1.5.1 Desde el sistema operativo

Linux	Windows	Para qué sirve
`ps aux`, `ps -ef`	`tasklist`, `Get-Process`	Listar procesos
`top`, `htop`	Administrador de tareas, `resmon`	Monitorizar CPU y memoria en tiempo real
`pstree`	Process Explorer	Ver el árbol de procesos
`kill PID`, `kill -9 PID`	`taskkill /PID n /F`, `Stop-Process`	Terminar un proceso
`nice`, `renice`	`start /low`, Administrador de tareas	Cambiar la prioridad
`systemctl`	`sc`, `Get-Service`	Gestionar servicios

Señales en Linux. `kill` no «mata» directamente: envía una **señal**. `SIGTERM` (15, la que se envía por defecto) pide al proceso que termine y le permite cerrar ordenadamente; `SIGKILL` (9) lo elimina sin posibilidad de reacción; `SIGINT` (2) es la que envía `Ctrl+C`.

1.5.2 Desde Python: `os` y `psutil`

El módulo estándar `os` da información básica del proceso actual. Para monitorizar el sistema usaremos la librería `psutil` (`pip install psutil`), que funciona igual en Linux, Windows y macOS:

```python
import os
import psutil

print("Mi PID:", os.getpid(), "- PID de mi padre:", os.getppid())
print("Núcleos lógicos:", os.cpu_count())

yo = psutil.Process()                    # el proceso actual
print(yo.name(), yo.status(), yo.memory_info().rss // 1024, "KiB")

# Los 5 procesos que más memoria usan
procesos = []
for p in psutil.process_iter(["pid", "name", "memory_percent"]):
    procesos.append(p.info)
procesos.sort(key=lambda d: d["memory_percent"] or 0, reverse=True)
for info in procesos[:5]:
    print(f"{info['pid']:>7}  {info['name']:<25} {info['memory_percent']:.1f} %")

print("Uso de CPU por núcleo:", psutil.cpu_percent(interval=1, percpu=True))
```

1.6 Lanzar programas externos: `subprocess`

La forma más directa de crear un proceso es pedir al SO que ejecute *otro programa*. En Python se hace con el módulo `subprocess`.

1.6.1 `subprocess.run`: ejecutar y esperar

```
import subprocess
import sys

# Ejecuta otro intérprete de Python que imprime su versión
resultado = subprocess.run(
    [sys.executable, "--version"],   # orden y argumentos, como lista
    capture_output=True,             # capturar stdout y stderr
    text=True,                       # devolver str en lugar de bytes
)
print("Código de salida:", resultado.returncode)   # 0 = todo bien
print("Salida:", resultado.stdout.strip())

# Con check=True, un código de salida distinto de 0 lanza una excepción
try:
    subprocess.run([sys.executable, "-c", "import sys; sys.exit(3)"],
                   check=True)
except subprocess.CalledProcessError as e:
    print("El proceso falló con código", e.returncode)
```

Seguridad: no uses `shell=True` con datos externos. Puedes pasar la orden como una cadena con `shell=True` (`run("ls -l", shell=True)`), pero entonces la cadena la interpreta un *shell*. Si una parte procede del usuario, este podría inyectar órdenes (`fichero.txt; rm -rf ~`). Pasa siempre la orden como **lista** de argumentos. Volveremos sobre ello en el Tema 5.

1.6.2 Pasar datos al proceso hijo

Un proceso hijo puede recibir información de su padre por los **argumentos**, por las **variables de entorno** y por su **entrada estándar**:

```
import os, subprocess, sys

codigo_hijo = (
    "import os, sys\n"
    "datos = sys.stdin.read()\n"
    "print(f'Arg: {sys.argv[1]} | Env: {os.environ[\"MODO\"]} | "
    "Recibí {len(datos.split())} palabras')\n"
)
entorno = os.environ | {"MODO": "pruebas"}     # copia del entorno + MODO

r = subprocess.run([sys.executable, "-c", codigo_hijo, "hola"],
                   input="uno dos tres", text=True,
                   capture_output=True, env=entorno)
print(r.stdout)
```

```
Arg: hola | Env: pruebas | Recibí 3 palabras
```

1.6.3 `Popen`: procesos que se ejecutan en paralelo

`run` espera a que el hijo termine. Si queremos que el padre siga trabajando mientras el hijo se ejecuta, usamos `Popen` directamente:

```
import subprocess, sys, time

hijos = []
for segundos in (3, 1, 2):
    p = subprocess.Popen([sys.executable, "-c",
                          f"import time; time.sleep({segundos})"])
    hijos.append((p, segundos))
    print(f"Lanzado hijo PID {p.pid} ({segundos} s)")

inicio = time.perf_counter()
for p, segundos in hijos:
    p.wait()                          # espera a que termine ESTE hijo
    print(f"Hijo {p.pid} terminó con código {p.returncode}")
print(f"Tiempo total: {time.perf_counter() - inicio:.1f} s")   # ~3 s, no 6
```

También podemos comunicarnos con el hijo mientras se ejecuta a través de **tuberías** (*pipes*): con `stdout=subprocess.PIPE` la salida del hijo queda disponible en `p.stdout` para que el padre la lea, y con `p.communicate(entrada)` se envían datos y se recoge la respuesta.

1.7 El módulo `multiprocessing`

`subprocess` lanza *otro programa*. Cuando lo que queremos es ejecutar **una función de nuestro propio programa** en otro proceso, usamos `multiprocessing`. Su API es casi idéntica a la de los hilos (`threading`), lo que facilitará el Tema 2.

1.7.1 ¿Por qué procesos y no hilos? El GIL

La versión *normal* del intérprete oficial (CPython) tiene un cerrojo global, el **GIL** (*Global Interpreter Lock*), que impide que dos hilos del mismo proceso ejecuten código Python a la vez. Consecuencias:

- Para tareas **limitadas por E/S** (esperar a la red, al disco), los hilos funcionan perfectamente: mientras uno espera, el GIL se libera y otro avanza.
- Para tareas **limitadas por CPU** (cálculos), los hilos *no* aprovechan varios núcleos. Hay que usar **procesos**: cada uno tiene su propio intérprete y su propio GIL.

Con la **compilación sin GIL** (`python3.14t`, `python3.15t`), presentada en el apartado 0.1.2 (su instalación se explica en el Apéndice A), los hilos también se reparten entre los núcleos para hacer cálculos; lo comprobaremos con un experimento en el Tema 2. Aun así, los procesos siguen siendo la herramienta adecuada en muchos casos:

- **Aislamiento y robustez**: si un proceso falla o se cuelga, los demás siguen; si falla un hilo de forma grave, cae todo el programa.

- **Seguridad**: un proceso puede ejecutarse con otro usuario o con menos privilegios.
- **Compatibilidad**: funcionan igual en cualquier versión de Python y con cualquier librería, incluidas las que no admiten la versión sin GIL.
- **Ejecutar otros programas**, escritos en cualquier lenguaje (`subprocess`).
- Es el primer paso hacia la **programación distribuida**: procesos que, en lugar de compartir memoria, se envían mensajes.

1.7.2 Crear un proceso con `Process`

```python
# Creación de procesos con una función objetivo
import os
import time
from multiprocessing import Process

def trabajar(nombre: str, segundos: float) -> None:
    print(f"[{nombre}] PID={os.getpid()} padre={os.getppid()}")
    time.sleep(segundos)
    print(f"[{nombre}] terminado")

if __name__ == "__main__":
    p1 = Process(target=trabajar, args=("A", 2))
    p2 = Process(target=trabajar, args=("B", 1), name="proceso-B")

    p1.start()            # crea el proceso y empieza a ejecutar 'trabajar'
    p2.start()
    print(f"Padre {os.getpid()}: hijos lanzados ({p1.pid}, {p2.pid})")

    p1.join()             # espera a que p1 termine
    p2.join()
    print("Códigos de salida:", p1.exitcode, p2.exitcode)
```

```
Padre 4120: hijos lanzados (4121, 4122)
[A] PID=4121 padre=4120
[B] PID=4122 padre=4120
[B] terminado
[A] terminado
Códigos de salida: 0 0
```

Nota. El orden de los mensajes puede variar de una ejecución a otra: depende del planificador del SO. Esto es lo normal en programación concurrente y es lo primero que hay que asumir: **no se puede suponer ningún orden** entre procesos salvo que lo forcemos con mecanismos de sincronización.

Métodos y atributos más útiles de `Process`:

`start()`	Lanza el proceso. Sólo se puede llamar una vez.
`join(timeout=None)`	Bloquea al padre hasta que el hijo termine (o pase el `timeout`).
`is_alive()`	Indica si el proceso sigue en ejecución.
`terminate()` / `kill()`	Envía `SIGTERM` / `SIGKILL` al hijo. Úsalo con cuidado: el hijo no libera sus recursos.
`pid`, `name`	Identificador y nombre.
`exitcode`	`None` si no ha terminado; 0 si terminó bien; > 0 si terminó con `sys.exit(n)` o por una excepción (1); < 0 si lo terminó la señal $-n$.
`daemon`	Si es `True`, el proceso se termina automáticamente al terminar el padre.

1.7.3 Pasar argumentos al proceso: `args` y `kwargs`

El constructor de `Process` tiene esta forma:

```
Process(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)
```

Los argumentos de la función objetivo se indican con dos parámetros:

- `args`: una **tupla** con los argumentos *posicionales*, en el mismo orden en que los espera la función.
- `kwargs`: un **diccionario** con los argumentos *con nombre* (*keyword arguments*): cada clave es el nombre de un parámetro de la función y cada valor, lo que recibe.

Al llamar a `start()`, el hijo ejecuta `target(*args, **kwargs)`: la tupla se desempaqueta en argumentos posicionales y el diccionario en argumentos con nombre. Se pueden usar los dos a la vez, y los parámetros con valor por defecto se pueden omitir, igual que en una llamada normal:

```python
# Paso de argumentos posicionales y con nombre
from multiprocessing import Process

def saludar(nombre: str, veces: int = 1, *, signo: str = "!") -> None:
    for _ in range(veces):
        print(f"Hola, {nombre}{signo}")

if __name__ == "__main__":
    procesos = [
        Process(target=saludar, args=("Ana",)),                  # saludar("Ana")
        Process(target=saludar, args=("Luis", 2)),               # saludar("Luis", 2)
        Process(target=saludar, kwargs={"nombre": "Eva", "signo": "?"}),
        Process(target=saludar, args=("Raúl",), kwargs={"veces": 2, "signo": "."}),
    ]
    for p in procesos:
        p.start()
    for p in procesos:
        p.join()
```

```
Hola, Ana!
Hola, Luis!
Hola, Luis!
Hola, Eva?
Hola, Raúl.
Hola, Raúl.
```

El tercer proceso ejecuta `saludar(nombre="Eva", signo="?")` y el cuarto, `saludar("Raúl", veces=2, signo=".")`. Como `signo` va detrás de `*` en la definición de la función, sólo se puede pasar por nombre, es decir, dentro de `kwargs`. Recuerda que el orden de las líneas puede cambiar de una ejecución a otra.

Una tupla de un solo elemento lleva coma. `args=("Ana")` no es una tupla, sino la cadena `"Ana"` entre paréntesis. Como una cadena también se puede desempaquetar, el hijo ejecuta `saludar("A", "n", "a")` y falla. El error no aparece en el padre, sino en el hijo, que termina con `exitcode` 1:

```
TypeError: saludar() takes 1 positional argument but 3 were given
```

Lo correcto es `args=("Ana",)`. Lo mismo ocurre con `Thread`.

Los argumentos **se copian** en el proceso hijo: se serializan con `pickle` y el hijo reconstruye su propia copia. Si el hijo modifica una lista que ha recibido como argumento, el padre no ve el cambio (lo veremos en el apartado 1.7.5). Para devolver información hay que usar los mecanismos de comunicación del apartado 1.8.

1.7.4 Crear un proceso heredando de `Process`

La alternativa es crear una subclase y sobrescribir `run()`, que es el método que se ejecuta en el nuevo proceso. Es útil cuando el proceso tiene estado o varios métodos auxiliares.

En este caso los argumentos se pasan de otra forma. El método `run()` que trae `Process` es, en esencia, `self._target(*self._args, **self._kwargs)`: es el que usa `target`, `args` y `kwargs`. Al sobrescribirlo, esos parámetros dejan de tener efecto, y como `run()` **no recibe parámetros**, los datos llegan así:

1. El **constructor de nuestra clase** (`__init__`) recibe los argumentos que necesite el proceso, con los parámetros posicionales o con nombre que queramos.
2. Llama a `super().__init__()`, pasándole si hace falta los parámetros propios de `Process` (`name`, `daemon`).
3. Guarda los argumentos como **atributos** de `self`.
4. `run()` lee esos atributos.

```
from multiprocessing import Process

class Contador(Process):
    def __init__(self, hasta: int, *, paso: int = 1, nombre: str | None = None):
        super().__init__(name=nombre)    # ¡imprescindible y lo primero!
        self.hasta = hasta               # los argumentos se guardan como atributos
        self.paso = paso
        self.total = 0

    def run(self) -> None:               # sin parámetros: lee los datos de self
        self.total = sum(range(0, self.hasta, self.paso))
        print(f"{self.name}: suma hasta {self.hasta} de {self.paso} en {self.paso}"
              f" = {self.total}")

if __name__ == "__main__":
    procesos = [Contador(10), Contador(1_000, paso=2),
                Contador(1_000_000, nombre="grande")]
```

```
    for p in procesos:
        p.start()                          # se llama a start(), NUNCA a run()
    for p in procesos:
        p.join()
    print("El padre ve:", [p.total for p in procesos])
```

```
Contador-1: suma hasta 10 de 1 en 1 = 45
Contador-2: suma hasta 1000 de 2 en 2 = 249500
grande: suma hasta 1000000 de 1 en 1 = 499999500000
El padre ve: [0, 0, 0]
```

Fíjate en la última línea. El objeto `Contador` se crea en el padre (allí se ejecuta `__init__`); al llamar a `start()` se copia en el hijo, y `run()` se ejecuta sobre esa copia. Por eso el valor que `run()` guarda en `self.total` no llega al padre, que sigue viendo 0. Con hilos sí llegaría, porque comparten memoria (Tema 2). Por el mismo motivo, todos los atributos que se guardan en `__init__` deben poderse serializar con `pickle`.

`start()` y no `run()`. Si llamas a `p.run()` directamente, el código se ejecuta *en el proceso padre*, como una función normal: no se crea ningún proceso.

No olvides `super().__init__()`. Si tu `__init__` no llama al constructor de `Process`, el objeto no queda bien inicializado y `start()` falla con un error poco claro (`AttributeError: 'Contador' object has no attribute '_closed'`). Llámalo al principio, antes de usar `self`.

1.7.5 Cómo se crea el proceso hijo

`multiprocessing` puede crear el hijo de tres maneras (*métodos de inicio*):

- `spawn`: arranca un intérprete nuevo que *importa* tu módulo y ejecuta la función. Es el método por defecto en Windows y macOS. Por eso es obligatorio el `if __name__ == "__main__"`.
- `fork`: duplica el proceso padre (sólo en sistemas Unix). Rápido, pero puede dar problemas si el padre tiene hilos.
- `forkserver`: un proceso servidor auxiliar crea los hijos. Es el método por defecto en Linux desde Python 3.14 (antes era `fork`).

Lo importante es la consecuencia práctica: **los procesos no comparten memoria**. Si el hijo modifica una variable global, el padre no ve el cambio:

```
from multiprocessing import Process

datos = []

def anadir():
    datos.append("desde el hijo")
    print("Hijo ve:", datos)

if __name__ == "__main__":
    p = Process(target=anadir)
    p.start(); p.join()
```

```
    print("Padre ve:", datos)      # Padre ve: []
```

Para compartir información necesitamos los mecanismos de comunicación entre procesos (IPC) de la siguiente sección. Además, todo lo que se envía de un proceso a otro (argumentos, resultados, mensajes) se **serializa** con `pickle`; por eso las funciones objetivo deben estar definidas a nivel de módulo (no valen `lambda` ni funciones anidadas con `spawn`).

1.8 Comunicación entre procesos

Como los procesos no comparten memoria, `multiprocessing` ofrece objetos especiales para intercambiar información. Todos se crean en el padre y se pasan a los hijos como un argumento más (en `args`, `kwargs` o en el constructor de la subclase):

- `Event`: una señal de sí o no ("empieza", "para").
- `Queue`: mensajes entre cualquier número de procesos.
- `Pipe`: mensajes entre exactamente dos procesos.
- `Value` y `Array`: números en memoria compartida.
- `Manager`: listas, diccionarios y otros objetos compartidos.

1.8.1 Eventos: `Event`

Un evento es el mecanismo de comunicación más sencillo: una **bandera** compartida que sólo puede estar activada o desactivada. Unos procesos esperan a que se active y otro la activa. No transmite datos, sólo un aviso: "ya puedes empezar", "hay que terminar"...

`Event()`	Crea el evento, desactivado.
`set()`	Activa la bandera y despierta a todos los procesos que esperan.
`clear()`	La vuelve a desactivar.
`is_set()`	Devuelve `True` si está activada. No bloquea.
`wait(timeout=None)`	Bloquea hasta que se active (o hasta que pasen `timeout` segundos). Devuelve `True` si se activó y `False` si se agotó el tiempo.

Un uso típico es avisar a los hijos de que deben terminar:

```
# Un evento para avisar a los hijos de que deben terminar
import time
from multiprocessing import Event, Process

def vigilar(parar, nombre: str) -> None:
    vueltas = 0
    while not parar.is_set():           # ¿se ha activado la señal?
        vueltas += 1
        parar.wait(0.5)                 # espera 0,5 s o hasta que se active
    print(f"{nombre}: aviso recibido tras {vueltas} vueltas")

if __name__ == "__main__":
    parar = Event()                     # empieza desactivado
    hijos = [Process(target=vigilar, args=(parar, f"vigilante-{i}"))
             for i in range(3)]
```

```
    for h in hijos:
        h.start()
    time.sleep(2)
    print("Padre: ¡parad!")
    parar.set()                          # activa la señal para todos
    for h in hijos:
        h.join()
```

```
Padre: ¡parad!
vigilante-1: aviso recibido tras 4 vueltas
vigilante-0: aviso recibido tras 4 vueltas
vigilante-2: aviso recibido tras 4 vueltas
```

`wait(0.5)` mejor que `time.sleep(0.5)`. Las dos esperan medio segundo, pero `wait()` se despierta en cuanto se activa el evento, sin esperar a que acabe la pausa: los hijos reaccionan al aviso inmediatamente.

1.8.2 Colas: `Queue`

Una cola es un canal FIFO seguro que pueden usar varios procesos a la vez, tanto para escribir (`put`) como para leer (`get`). La lectura es **bloqueante**: si la cola está vacía, `get()` espera.

`Queue(maxsize=0)`	Crea la cola. Con `maxsize` mayor que 0 tiene capacidad limitada.
`put(obj, timeout=None)`	Añade un objeto al final. Si la cola está llena, espera a que haya sitio.
`get(timeout=None)`	Saca el primer objeto. Si la cola está vacía, espera. Con `timeout`, lanza `queue.Empty` si no llega nada a tiempo.
`put_nowait(obj)`, `get_nowait()`	Como `put` y `get`, pero sin esperar: lanzan `queue.Full` o `queue.Empty`.
`empty()`, `qsize()`	Si está vacía y cuántos elementos tiene. Son aproximados, porque otro proceso puede cambiar la cola justo después; `qsize()` no funciona en macOS.
`close()`	Indica que este proceso ya no va a añadir más datos.

Es la base del patrón **productor–consumidor**:

```
# Productor y consumidor comunicados por una cola
import random
import time
from multiprocessing import Process, Queue

FIN = None                                  # marca de fin ("píldora venenosa")

def productor(cola: Queue, n: int) -> None:
    for i in range(n):
        dato = random.randint(1, 100)
        cola.put(dato)
        print(f"Productor: {dato}")
        time.sleep(random.random() / 2)
```

```
    cola.put(FIN)

def consumidor(cola: Queue) -> None:
    total = 0
    while (dato := cola.get()) is not FIN:  # get() bloquea si está vacía
        total += dato
        print(f"   Consumidor: recibido {dato} (acumulado {total})")
    print("Consumidor: fin")

if __name__ == "__main__":
    cola = Queue()
    prod = Process(target=productor, args=(cola, 5))
    cons = Process(target=consumidor, args=(cola,))
    prod.start(); cons.start()
    prod.join(); cons.join()
```

El operador morsa `:=`. `while (dato := cola.get()) is not FIN:` asigna y compara a la vez. Es equivalente a leer antes del bucle y al final de cada iteración.

Para no quedarse esperando para siempre, se usa `get()` con `timeout`. La excepción `Empty` está en el módulo `queue`:

```
import queue                        # sólo para la excepción queue.Empty

try:
    dato = cola.get(timeout=2)
except queue.Empty:
    print("No ha llegado nada en 2 segundos")
```

No uses `empty()` para saber si has terminado. Que la cola esté vacía no significa que el productor haya acabado: puede que aún no haya enviado el siguiente dato. Para indicar el final, el productor envía una marca de fin, como `FIN` en el ejemplo.

1.8.3 Tuberías: `Pipe`

`Pipe()` devuelve los dos extremos de un canal entre **dos** procesos. Por defecto es bidireccional: cada extremo puede enviar y recibir.

`Pipe(duplex=True)`	Devuelve la tupla `(c1, c2)` con los dos extremos. Con `duplex=False`, `c1` sólo recibe y `c2` sólo envía.
`send(obj)`	Envía un objeto al otro extremo.
`recv()`	Recibe el siguiente objeto. Si no hay ninguno, espera. Lanza `EOFError` si no queda nada por leer y el otro extremo está cerrado.
`poll(timeout=0)`	Devuelve `True` si hay algo para leer, esperando como mucho `timeout` segundos (`None`: sin límite).
`close()`	Cierra este extremo.

```
from multiprocessing import Process, Pipe

def hijo(conexion):
```

```
    while (mensaje := conexion.recv()) != "FIN":
        conexion.send(mensaje.upper())
    conexion.close()

if __name__ == "__main__":
    extremo_padre, extremo_hijo = Pipe()
    p = Process(target=hijo, args=(extremo_hijo,))
    p.start()
    for palabra in ["hola", "proceso", "hijo"]:
        extremo_padre.send(palabra)
        print(extremo_padre.recv())     # HOLA, PROCESO, HIJO
    extremo_padre.send("FIN")
    p.join()
```

En lugar de una marca de fin, el emisor puede cerrar su extremo: el receptor lo detecta porque `recv()` lanza `EOFError`. Para que funcione, **todos** los procesos que tienen una copia del extremo de escritura deben cerrarla, también el padre:

```
# Tubería de un solo sentido: el hijo envía y el padre recibe
from multiprocessing import Process, Pipe

def productor(salida) -> None:
    for i in range(3):
        salida.send(i)
    salida.close()                      # fin de los datos

if __name__ == "__main__":
    lectura, escritura = Pipe(duplex=False)
    p = Process(target=productor, args=(escritura,))
    p.start()
    escritura.close()                   # el padre no escribe: cierra su copia
    try:
        while True:
            print("Recibido:", lectura.recv())
    except EOFError:
        print("El otro extremo se ha cerrado")
    p.join()
```

```
Recibido: 0
Recibido: 1
Recibido: 2
El otro extremo se ha cerrado
```

Una tubería, dos procesos. Si dos procesos leen (o escriben) a la vez en el *mismo* extremo, los datos se pueden corromper. Cuando haya más de un productor o de un consumidor, usa una `Queue`.

1.8.4 Memoria compartida: `Value` y `Array`

Para compartir datos numéricos sencillos, `multiprocessing` puede crear variables en un segmento de memoria compartida. Hay que indicar el tipo con un código de `ctypes`: `'i'` entero, `'d'` real, etc.

`Value(tipo, valor)`	Crea un número compartido: `Value("i", 0)`, `Value("d", 1.5)`.
`v.value`	Lee o modifica el número: `v.value = 10`.
`Array(tipo, n)`	Crea un vector de `n` elementos a 0. También admite una secuencia con los valores iniciales: `Array("d", [1.5, 2.5])`. Su tamaño es fijo.
`a[i]`, `a[i:j]`, `len(a)`	Se usa como una lista: acceso por índice, porciones, longitud y recorrido con `for`. `list(a)` lo copia en una lista normal.
`get_lock()`	Devuelve el cerrojo propio del objeto, para modificarlo sin que otro proceso intervenga (ver el aviso).

```python
from multiprocessing import Process, Value, Array

def rellenar(vector, indice: int, hechos) -> None:
    vector[indice] = indice * indice
    with hechos.get_lock():                # += no es atómico (ver el aviso)
        hechos.value += 1

if __name__ == "__main__":
    cuadrados = Array("i", 5)              # 5 enteros inicializados a 0
    hechos = Value("i", 0)                 # un entero que vale 0
    hijos = [Process(target=rellenar, args=(cuadrados, i, hechos))
             for i in range(5)]
    for h in hijos:
        h.start()
    for h in hijos:
        h.join()
    print(list(cuadrados), "- tareas hechas:", hechos.value)
    # [0, 1, 4, 9, 16] - tareas hechas: 5
```

`v.value += 1` no es una operación atómica. Son tres pasos: leer el valor, sumarle uno y escribirlo. Si dos procesos leen el mismo valor antes de que ninguno escriba, se pierde un incremento: es una **condición de carrera**. Por eso la modificación se hace dentro de `with hechos.get_lock():`, que garantiza que sólo un proceso la hace en cada momento. En cambio, cada hijo escribe en una posición distinta de `cuadrados`, así que ahí no hay conflicto. Estudiaremos este problema en el apartado 2.6 del Tema 2.

1.8.5 `Manager`: estructuras compartidas de alto nivel

Un `Manager` lanza un proceso servidor que guarda objetos Python (listas, diccionarios...) a los que los demás procesos acceden a través de *proxies*. Es más lento que `Value`/`Array` pero mucho más flexible.

`Manager()`	Lanza el proceso servidor. Con `with` se detiene al salir del bloque; si no, hay que llamar a `shutdown()`.
`gestor.list()`	Crea una lista compartida, que se usa como una normal: `append()`, `l[i]`, `len()`, `for`...
`gestor.dict()`	Crea un diccionario compartido: `d[clave]`, `get()`, `keys()`, `items()`...
`gestor.Namespace()`	Crea un objeto al que se le pueden añadir atributos: `ns.total = 0`.
`gestor.Queue()`...	También crea `Queue`, `Event`, `Value`, `Array` y `Lock` accesibles mediante *proxy*. Son necesarios con `Pool` (apartado 1.9), al que no se le pueden pasar los de `multiprocessing`.

```python
from multiprocessing import Process, Manager

def registrar(resultados, nombres, nombre: str) -> None:
    resultados[nombre] = len(nombre)
    nombres.append(nombre)

if __name__ == "__main__":
    with Manager() as gestor:
        resultados = gestor.dict()         # diccionario compartido
        nombres = gestor.list()            # lista compartida
        hijos = [Process(target=registrar, args=(resultados, nombres, n))
                 for n in ("ana", "roberto", "eva")]
        for h in hijos:
            h.start()
        for h in hijos:
            h.join()
        print(dict(resultados))
        print(list(nombres))
```

```
{'ana': 3, 'roberto': 7, 'eva': 3}
['ana', 'roberto', 'eva']
```

El orden de las claves y de la lista depende del orden en que los hijos escriben, que puede cambiar en cada ejecución.

Los objetos anidados no se actualizan solos. El *proxy* sólo detecta los cambios hechos directamente sobre él. Si el diccionario compartido `d` contiene una lista, `d["notas"].append(7)` modifica una *copia* de la lista y el cambio se pierde. Hay que sacar el valor, modificarlo y volver a asignarlo:

```python
notas = d["notas"]
notas.append(7)
d["notas"] = notas                    # ahora sí se guarda en el servidor
```

Sincronización entre procesos. Cuando varios procesos acceden a la vez a un recurso compartido (un `Value`, un fichero, la consola) aparecen condiciones de carrera, como la del aviso de `Value`. Para evitarlas, `multiprocessing` ofrece los mismos mecanismos de sincronización que `threading` (`Lock`, `RLock`, `Semaphore`, `Condition`, `Barrier` y el `Event` que ya conoces), con los mismos métodos. Los estudiaremos con hilos en el apartado 2.7 del Tema 2: todo lo que veas allí sirve para procesos importando la clase de `multiprocessing`

y pasando el objeto al hijo como argumento.

1.9 Obtener resultados: `Pool` y `ProcessPoolExecutor`

`Process` no devuelve el valor de la función que ejecuta. Podríamos usar una cola para recogerlo, pero lo habitual es usar un **conjunto** (**pool**) **de procesos**: un número fijo de procesos trabajadores que ejecutan tareas y devuelven sus resultados al padre.

1.9.1 `multiprocessing.Pool`

```
# Repartir un cálculo entre varios procesos
import time
from multiprocessing import Pool

def es_primo(n: int) -> bool:
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

def contar_primos(rango: tuple[int, int]) -> int:
    inicio, fin = rango
    return sum(1 for n in range(inicio, fin) if es_primo(n))

if __name__ == "__main__":
    LIMITE, TROZOS = 1_000_000, 8
    paso = LIMITE // TROZOS
    rangos = [(i, i + paso) for i in range(0, LIMITE, paso)]

    t = time.perf_counter()
    secuencial = sum(contar_primos(r) for r in rangos)
    t_sec = time.perf_counter() - t

    t = time.perf_counter()
    with Pool() as pool:                    # tantos procesos como núcleos
        parciales = pool.map(contar_primos, rangos)
    t_par = time.perf_counter() - t

    print(f"Secuencial: {secuencial} primos en {t_sec:.2f} s")
    print(f"Paralelo:   {sum(parciales)} primos en {t_par:.2f} s")
    print(f"Aceleración: x{t_sec / t_par:.1f}")
```

Métodos principales de `Pool`:

- `map(f, iterable)`: aplica `f` a cada elemento y devuelve la lista de resultados **en orden**. Bloquea hasta terminar.
- `starmap(f, iterable)`: igual, pero cada elemento es una tupla de argumentos.
- `imap_unordered(f, iterable)`: devuelve los resultados según van terminando.

- `apply_async(f, args)`: lanza una tarea y devuelve un objeto `AsyncResult`; su método `get()` espera y devuelve el valor (o relanza la excepción que se produjo en el hijo).

1.9.2 `concurrent.futures`

La librería estándar ofrece una interfaz más moderna y uniforme, que funciona igual con procesos (`ProcessPoolExecutor`) y con hilos (`ThreadPoolExecutor`). Cada tarea enviada devuelve un **futuro**: un objeto que representa un resultado que estará disponible más adelante.

```
from concurrent.futures import ProcessPoolExecutor, as_completed

def factorizar(n: int) -> list[int]:
    factores, d = [], 2
    while d * d <= n:
        while n % d == 0:
            factores.append(d)
            n //= d
        d += 1
    if n > 1:
        factores.append(n)
    return factores

if __name__ == "__main__":
    numeros = [600851475143, 9999999967, 1234567890123, 97]
    with ProcessPoolExecutor(max_workers=4) as ejecutor:
        futuros = {ejecutor.submit(factorizar, n): n for n in numeros}
        for futuro in as_completed(futuros):        # según van acabando
            n = futuros[futuro]
            try:
                print(n, "=", futuro.result())
            except Exception as e:
                print(n, "produjo un error:", e)
```

1.10 Depuración y buenas prácticas

- Identifica cada mensaje con el proceso que lo genera. Con `logging` basta con añadir `%(processName)s` y `%(process)d` al formato.
- Las excepciones de un hijo creado con `Process` **no** llegan al padre: se imprimen en el hijo y el `exitcode` será 1. Comprueba siempre el `exitcode` o usa `Pool`/`Executor`, que sí propagan la excepción al llamar a `get()`/`result()`.
- Haz siempre `join()` de los procesos creados. Un proceso terminado del que el padre no ha recogido el estado queda como **zombi** en la tabla de procesos.
- Evita compartir estado: es preferible que los procesos se comuniquen enviándose mensajes (colas) que modificando memoria común.
- El depurador de VS Code puede seguir procesos hijos (opción `"subProcess": true` en `launch.json`).

Resumen del tema.

- Un **proceso** es un programa en ejecución con su propia memoria. El SO alterna entre

ellos según su **estado** y un algoritmo de **planificación**.

- **Concurrencia** es estructurar el programa en tareas que progresan a la vez; **paralelismo** es ejecutarlas a la vez en varios núcleos; **distribuida**, en varias máquinas.
- `subprocess` lanza otros programas; `multiprocessing` ejecuta funciones Python en procesos nuevos (y evita el GIL).
- Los argumentos se pasan con `args` (tupla) y `kwargs` (diccionario), o al constructor de la subclase de `Process`, que los guarda como atributos para `run()`.
- Los procesos no comparten memoria: se comunican con `Event`, `Queue`, `Pipe`, `Value`/`Array` o `Manager`.
- El acceso concurrente a datos compartidos provoca **condiciones de carrera**; se evitan con `get_lock()` y con los mecanismos de sincronización del Tema 2.
- `Pool` y `ProcessPoolExecutor` reparten tareas y recogen sus resultados.

1.11 Ejercicios

Ejercicio 1.1 — Planificación a mano

Dificultad: ★

Cinco procesos llegan en el instante 0 con estas duraciones (ms): P1=8, P2=4, P3=9, P4=5, P5=2. Dibuja el diagrama de Gantt y calcula el tiempo medio de espera con FCFS, SJF y Round Robin con cuanto de 3 ms. ¿Qué algoritmo es más justo? ¿Cuál da mejor tiempo medio?

Ejercicio 1.2 — Monitor de procesos

Dificultad: ★

Usando `psutil`, escribe un programa que cada 2 segundos limpie la pantalla y muestre los 10 procesos que más CPU consumen (PID, nombre, usuario, % CPU, memoria en MiB), además del uso total de CPU y memoria del sistema. Debe terminar limpiamente con `Ctrl+C` (captura `KeyboardInterrupt`).

Ejercicio 1.3 — Lanzador de órdenes

Dificultad: ★

Escribe un programa que lea de un fichero `ordenes.txt` una orden por línea (por ejemplo, `python --version`, `ping -c 2 localhost`), las ejecute **todas a la vez** con `Popen`, y muestre para cada una su código de salida y su salida estándar. Usa `shlex.split()` para trocear cada línea de forma segura.

Ejercicio 1.4 — Carrera de procesos

Dificultad: ★★

Crea una clase `Corredor` que herede de `Process`. Cada corredor avanza una distancia aleatoria cada décima de segundo hasta llegar a 100 m. Al llegar, escribe su nombre en una `Queue` compartida. El proceso principal lanza 5 corredores y muestra la clasificación final en orden de llegada.

Ejercicio 1.5 — Banco con condiciones de carrera

Dificultad: ★★

Simula una cuenta bancaria compartida (`Value('d', 1000)`). Lanza 4 procesos que hagan 1000 ingresos de 1 € y otros 4 que hagan 1000 retiradas de 1 €. Comprueba que, sin sincronización, el saldo final no es 1000. Corrígelo con `get_lock()` y explica en un comentario por qué funciona tu solución.

Ejercicio 1.6 — Medida de aceleración

Dificultad: ★★

Toma el programa de recuento de primos con `Pool` y ejecútalo con 1, 2, 4, 8 y 16 procesos (`Pool(processes=n)`). Anota los tiempos en una tabla, calcula la aceleración respecto a 1 proceso y explica los resultados en función del número de núcleos de tu equipo y de la ley de Amdahl.

Ejercicio 1.7 — Tubería entre programas

Dificultad: ★★★

Escribe dos scripts: `generador.py`, que escribe por su salida estándar 1000 números aleatorios (uno por línea), y `estadistica.py`, que los lee de su entrada estándar y muestra la media, la mediana y la desviación típica. Un tercer script, `orquestador.py`, debe lanzar ambos con `Popen` conectando la salida del primero con la entrada del segundo (como hace `|` en el shell) y mostrar el resultado.

Proyecto: Procesador paralelo de ficheros

Una empresa necesita calcular la huella `SHA-256` de miles de ficheros para detectar duplicados. Desarrolla una herramienta de línea de órdenes que lo haga aprovechando todos los núcleos.

Requisitos:

1. Recibirá (`argparse`) un directorio, el número de procesos trabajadores y, opcionalmente, una extensión para filtrar.
2. Recorrerá el directorio recursivamente (`pathlib.Path.rglob`) y repartirá los ficheros entre los trabajadores con un `ProcessPoolExecutor`. Cada trabajador calcula el hash leyendo el fichero por bloques (`hashlib.sha256`, bloques de 64 KiB).
3. Mostrará una barra de progreso sencilla según van terminando las tareas (`as_completed`).
4. Los ficheros ilegibles (permisos, ficheros borrados) no deben detener el programa: se registran como error con `logging`.
5. Al terminar, generará un informe JSON con los grupos de ficheros duplicados, el espacio que se ahorraría eliminándolos y el tiempo empleado.
6. Incluirá una opción `--secuencial` para comparar tiempos y una breve memoria (`README.md`) con las mediciones realizadas.

Ampliación: añade un proceso monitor independiente que, cada segundo, muestre con `psutil` el uso de CPU y memoria de cada trabajador.

TEMA 2

Programación multihilo

En este tema aprenderás a…

- Identificar las situaciones en las que conviene usar hilos y entender el efecto del GIL.
- Crear, iniciar, detener de forma cooperativa y esperar hilos con `threading`.
- Conocer los estados de un hilo, su contexto de ejecución y cómo se controlan las prioridades en Python.
- Detectar los problemas de compartir información: condiciones de carrera, interbloqueos e inanición.
- Sincronizar hilos con cerrojos, semáforos, eventos, condiciones, barreras y colas.
- Usar conjuntos de hilos (`ThreadPoolExecutor`) y depurar programas multihilo.

Resultado de aprendizaje asociado: **RA2**.

2.1 ¿Cuándo usar hilos?

En el tema anterior vimos que un hilo es una secuencia de ejecución dentro de un proceso y que todos los hilos de un proceso comparten su memoria. Crear un hilo es mucho más barato que crear un proceso, y comunicarse entre hilos es tan sencillo como leer y escribir variables. Son la herramienta adecuada cuando:

- El programa pasa mucho tiempo **esperando**: descargas de red, consultas a una base de datos, lectura de ficheros, peticiones a una API. Mientras un hilo espera, otros pueden trabajar.
- Hay que mantener una **interfaz gráfica receptiva**: el hilo de la interfaz no debe bloquearse nunca; el trabajo pesado se hace en hilos secundarios.
- Un **servidor** debe atender a varios clientes a la vez (lo haremos en el Tema 3).
- Hay tareas periódicas o de fondo: guardar automáticamente, comprobar actualizaciones, enviar latidos (*heartbeats*) a un servidor.

Hilos en Python y cálculo intensivo. Recuerda el GIL (Tema 1): en la versión *normal* de Python, los hilos **no aceleran los cálculos** que usan intensivamente la CPU, porque sólo uno ejecuta código Python en cada instante. Con la versión **sin GIL** (`python3.15t`) sí lo hacen. Para esperar a la E/S, los hilos funcionan bien en ambas.

Este pequeño experimento lo demuestra. Simulamos 10 descargas de un segundo cada una con `time.sleep()`, que, como una operación de red, deja libre el GIL mientras espera:

```
import threading
import time
```

```
def descargar(n: int) -> None:
    time.sleep(1)                 # simula esperar a la red
    print(f"descarga {n} completada")

inicio = time.perf_counter()
for i in range(10):
    descargar(i)
print(f"Secuencial: {time.perf_counter() - inicio:.1f} s")     # ~10 s

inicio = time.perf_counter()
hilos = [threading.Thread(target=descargar, args=(i,)) for i in range(10)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"Con hilos:  {time.perf_counter() - inicio:.1f} s")     # ~1 s
```

2.1.1 Experimento: hilos con y sin GIL

Este programa reparte un cálculo (contar números primos) entre 4 hilos y lo compara con hacerlo en un solo hilo. Ejecútalo con las dos versiones del intérprete: `python3.15 gil.py` y `python3.15t gil.py`.

```
# gil.py: cálculo intensivo con hilos
import sys
import sysconfig
import time
from concurrent.futures import ThreadPoolExecutor

def contar_primos(inicio: int, fin: int) -> int:
    total = 0
    for n in range(max(inicio, 2), fin):
        if all(n % d for d in range(2, int(n ** 0.5) + 1)):
            total += 1
    return total

if __name__ == "__main__":
    print(sys.version)
    print("Compilación sin GIL:", sysconfig.get_config_var("Py_GIL_DISABLED") == 1)
    print("GIL activo ahora:   ", sys._is_gil_enabled())

    rangos = [(i, i + 100_000) for i in range(0, 400_000, 100_000)]
    inicio = time.perf_counter()
    secuencial = sum(contar_primos(a, b) for a, b in rangos)
    t_sec = time.perf_counter() - inicio

    inicio = time.perf_counter()
    with ThreadPoolExecutor(max_workers=4) as pool:
        con_hilos = sum(pool.map(lambda r: contar_primos(*r), rangos))
    t_hilos = time.perf_counter() - inicio

    print(f"Secuencial: {t_sec:.2f} s | 4 hilos: {t_hilos:.2f} s "
          f"| aceleración x{t_sec / t_hilos:.1f}")
```

Resultados en un equipo con 2 núcleos:

```
$ python3.13 gil.py
```

```
3.13.16 (main, ...)
Compilación sin GIL: False
GIL activo ahora:    True
Secuencial: 1.18 s | 4 hilos: 1.40 s | aceleración x0.8

$ python3.15t gil.py
3.15.0 free-threading build (main, ...)
Compilación sin GIL: True
GIL activo ahora:    False
Secuencial: 1.13 s | 4 hilos: 0.66 s | aceleración x1.7
```

Con GIL, los hilos no sólo no ayudan: el programa va *más lento* por el coste de alternar entre ellos. Sin GIL, la aceleración se acerca al número de núcleos.

2.2 Contexto de ejecución de los hilos



Cada hilo tiene su propio **contexto de ejecución**: su pila (donde viven las variables locales y los parámetros de las funciones que está ejecutando), su contador de programa y sus registros. Todo lo demás se comparte. En la práctica, en Python:

- Las **variables locales** de la función que ejecuta un hilo son privadas de ese hilo, aunque varios hilos ejecuten la misma función.
- Las **variables globales** y **los objetos** que reciben varios hilos (una lista, un diccionario, un objeto de una clase) son compartidos.
- Si necesitas un dato global pero distinto en cada hilo, existe `threading.local()`.

```python
import threading

datos_hilo = threading.local()       # cada hilo verá sus propios atributos

def tarea(usuario: str) -> None:
    datos_hilo.usuario = usuario      # no interfiere con otros hilos
    procesar()

def procesar() -> None:
    print(f"{threading.current_thread().name} procesa para {datos_hilo.usuario}")

for u in ("ana", "luis", "eva"):
    threading.Thread(target=tarea, args=(u,)).start()
```

2.3 Crear, iniciar y finalizar hilos

El módulo `threading` sigue el mismo esquema que `multiprocessing`.

2.3.1 Con una función objetivo

```
import threading
import time

def contar(nombre: str, hasta: int, pausa: float) -> None:
    for i in range(1, hasta + 1):
        print(f"{nombre}: {i}")
        time.sleep(pausa)

h1 = threading.Thread(target=contar, args=("Rápido", 5, 0.2), name="rapido")
h2 = threading.Thread(target=contar, kwargs={"nombre": "Lento", "hasta": 3,
                                             "pausa": 0.5})
h1.start()
h2.start()
print("Hilos activos:", threading.active_count())   # 3 (incluye el principal)
h1.join()
h2.join()
print("Fin del hilo", threading.current_thread().name)   # MainThread
```

2.3.2 Pasar argumentos al hilo: `args` y `kwargs`

El constructor de `Thread` tiene la misma forma que el de `Process` (apartado 1.7.3):

```
Thread(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)
```

- `args` es una **tupla** con los argumentos posicionales de `target`, en orden. Con un solo argumento, no olvides la coma: `args=("Rápido",)`.
- `kwargs` es un **diccionario** con los argumentos con nombre: las claves son los nombres de los parámetros.

Al llamar a `start()`, el nuevo hilo ejecuta `target(*args, **kwargs)`. En el ejemplo anterior, `h1` ejecuta `contar("Rápido", 5, 0.2)` y `h2`, `contar(nombre="Lento", hasta=3, pausa=0.5)`. Se pueden combinar los dos parámetros, como en una llamada normal.

La gran diferencia con los procesos es que los argumentos **no se copian**: el hilo recibe el *mismo* objeto que tiene el hilo principal. Si es mutable (una lista, un diccionario) y el hilo lo modifica, el cambio se ve desde fuera:

```
# Los hilos reciben el mismo objeto, no una copia
import threading

def anadir(lista: list[str], texto: str, *, veces: int = 1) -> None:
    for _ in range(veces):
        lista.append(texto)

compartida: list[str] = []
hilos = [
    threading.Thread(target=anadir, args=(compartida, "A")),
```

```
    threading.Thread(target=anadir, args=(compartida, "B"), kwargs={"veces": 2}),
]
for h in hilos: h.start()
for h in hilos: h.join()
print(compartida)                    # ['A', 'B', 'B']: el principal ve los cambios
```

Con procesos, la lista del padre seguiría vacía. Esta comodidad tiene un precio: si varios hilos modifican el mismo objeto a la vez pueden aparecer condiciones de carrera (apartado 2.6). El orden de los elementos también puede variar si los hilos se ejecutan a la vez.

2.3.3 Heredando de `Thread`

Igual que con `Process`, se crea una subclase y se sobrescribe `run()`. Al hacerlo, `target`, `args` y `kwargs` dejan de usarse, porque es el `run()` original el que llama a `target(*args, **kwargs)`. Como `run()` no recibe parámetros, los datos se pasan al **constructor** de la subclase, que llama a `super().__init__()` (con `name` o `daemon` si hace falta) y los guarda como atributos; después `run()` los lee de `self`:

```
import random
import threading
import time

class Sensor(threading.Thread):
    """Hilo que simula un sensor tomando lecturas periódicas."""

    def __init__(self, nombre: str, lecturas: int, *, intervalo: float = 0.1):
        super().__init__(name=nombre)       # ¡imprescindible y lo primero!
        self.lecturas = lecturas            # los argumentos se guardan como atributos
        self.intervalo = intervalo
        self.valores: list[float] = []      # resultado accesible tras join()

    def run(self) -> None:                  # sin parámetros: lee los datos de self
        for _ in range(self.lecturas):
            self.valores.append(round(random.uniform(18, 25), 1))
            time.sleep(self.intervalo)

sensores = [Sensor("sensor-0", 5), Sensor("sensor-1", 3, intervalo=0.2)]
for s in sensores: s.start()
for s in sensores: s.join()
for s in sensores:
    print(s.name, s.valores, "media:", round(sum(s.valores) / len(s.valores), 2))
```

```
sensor-0 [21.3, 20.5, 22.5, 18.7, 23.8] media: 21.36
sensor-1 [24.2, 19.9, 20.4] media: 21.5
```

Las lecturas son aleatorias, así que cambian en cada ejecución. Fíjate en que, a diferencia de los procesos, el hilo puede dejar su resultado en un atributo y el principal leerlo directamente tras el `join()`: el objeto `Sensor` que ejecuta `run()` es el mismo que creó el hilo principal, no una copia.

> **No olvides `super().__init__()`.** Sin esa llamada, el hilo no queda inicializado y `start()` lanza `RuntimeError: thread.__init__() not called`.

2.3.4 Información sobre los hilos

`threading.current_thread()`	Objeto `Thread` del hilo que ejecuta esa línea.
`threading.main_thread()`	El hilo principal.
`threading.enumerate()`	Lista de hilos vivos.
`threading.active_count()`	Número de hilos vivos.
`h.name`, `h.ident`, `h.native_id`	Nombre, identificador de Python e identificador del SO (el que verías en `htop`).
`h.is_alive()`	Si el hilo está en ejecución.

2.3.5 Finalizar un hilo

Un hilo termina cuando termina su función `run()` (o la función objetivo), ya sea normalmente o por una excepción no capturada.

> **No se puede «matar» un hilo.** A diferencia de los procesos, **Python no permite terminar un hilo desde fuera**. Es una decisión de diseño: un hilo interrumpido a la fuerza podría dejar un cerrojo adquirido o un fichero a medio escribir. La forma correcta es la **finalización cooperativa**: el hilo comprueba periódicamente una señal y termina por sí mismo.

```
# Finalización cooperativa con Event
import threading
import time

class Vigilante(threading.Thread):
    def __init__(self):
        super().__init__(name="vigilante")
        self._parar = threading.Event()

    def run(self) -> None:
        while not self._parar.is_set():
            print("Comprobando el sistema...")
            self._parar.wait(1)        # espera 1 s o hasta que pidan parar
        print("Vigilante: cerrando recursos y saliendo")

    def detener(self) -> None:
        self._parar.set()

v = Vigilante()
v.start()
time.sleep(3.5)
v.detener()
v.join()
```

2.3.6 Hilos demonio

Un programa Python no termina mientras haya hilos normales vivos. Si un hilo se marca como **demonio** (`daemon=True`), el programa puede terminar aunque ese hilo siga ejecutándose; el hilo se interrumpe bruscamente. Úsalos sólo para tareas de fondo que no dejen nada a medias (por ejemplo, un hilo que muestra un reloj).

```python
import threading, time

def reloj():
    while True:
        print(time.strftime("%H:%M:%S"), end="\r")
        time.sleep(1)

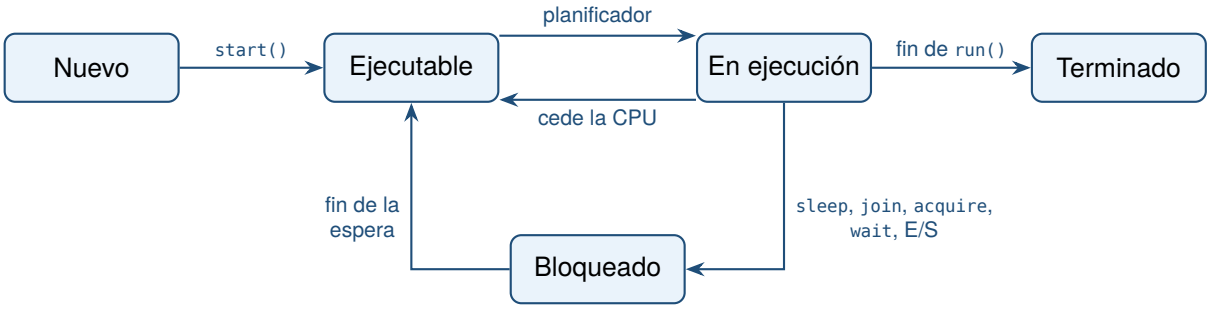
threading.Thread(target=reloj, daemon=True).start()
input("Pulsa Intro para salir\n")      # al salir, el hilo demonio muere
```

2.3.7 Tareas diferidas: `Timer`

`threading.Timer(segundos, funcion)` es un hilo que ejecuta la función tras un retardo; se puede cancelar antes con `cancel()`. Es útil, por ejemplo, para cerrar una sesión por inactividad.

2.4 Estados de un hilo

Los estados de un hilo son análogos a los de un proceso:



- **Nuevo**: el objeto `Thread` existe pero no se ha llamado a `start()`. `is_alive()` devuelve `False`.
- **Ejecutable** y **en ejecución**: tras `start()`. Python no distingue entre ambos; el SO (y el GIL) deciden.
- **Bloqueado**: el hilo espera por `sleep()`, `join()`, al adquirir un cerrojo ocupado, en `Event.wait()`, `Condition.wait()`, `Queue.get()` o una operación de E/S.
- **Terminado**: `run()` ha acabado. `is_alive()` vuelve a ser `False` y el hilo no puede reiniciarse.

Python no ofrece `suspend()`/`resume()` (en Java se eliminaron por peligrosos). Si queremos pausar y reanudar un hilo, lo programamos nosotros:

```python
# Hilo que se puede pausar, reanudar y detener
import threading
import time

class Tarea(threading.Thread):
    def __init__(self):
        super().__init__()
        self._activo = threading.Event()
        self._activo.set()                # empieza en marcha
```

```
        self._parar = threading.Event()

    def run(self) -> None:
        paso = 0
        while not self._parar.is_set():
            self._activo.wait()            # BLOQUEADO mientras esté pausado
            paso += 1
            print("paso", paso)
            time.sleep(0.3)

    def pausar(self):   self._activo.clear()
    def reanudar(self): self._activo.set()
    def detener(self):
        self._parar.set()
        self._activo.set()                 # por si estaba pausado

t = Tarea(); t.start()
time.sleep(1);  t.pausar();   print("-- pausa --")
time.sleep(1);  t.reanudar(); print("-- reanuda --")
time.sleep(1);  t.detener();  t.join()
```

2.5 Prioridades de los hilos

En lenguajes como Java, cada hilo tiene una prioridad (`setPriority()`) que el planificador *puede* tener en cuenta. **Python no permite fijar la prioridad de los hilos**, por dos motivos:

1. En la versión normal, el GIL decide qué hilo ejecuta código Python, y lo hace por turnos (cada 5 ms por defecto, ver `sys.getswitchinterval()`), no por prioridad. En la versión sin GIL lo decide el planificador del SO, pero `threading` no ofrece ninguna forma de indicarle una prioridad.
2. Las prioridades de hilos dependen mucho del SO y, en la práctica, son poco fiables también en otros lenguajes: no garantizan ningún orden.

Lo que sí podemos hacer en Python es:

- Cambiar la prioridad de todo un **proceso**, con `os.nice()` o con `psutil.Process().nice()`.
- Controlar el **orden en que se atienden las tareas** usando una cola de prioridad. Es el enfoque que se usa en aplicaciones reales: unos hilos trabajadores cogen siempre primero la tarea más urgente.

```
# Prioridad de tareas con PriorityQueue
import queue
import threading
import time

tareas: queue.PriorityQueue = queue.PriorityQueue()

def trabajador() -> None:
    while True:
        prioridad, descripcion = tareas.get()    # siempre la menor primero
        if descripcion is None:
            break
        print(f"[{threading.current_thread().name}] p={prioridad}: {descripcion}")
```

```
        time.sleep(0.2)
        tareas.task_done()

for prioridad, texto in [(5, "copia de seguridad"), (1, "alarma de incendio"),
                         (3, "enviar informe"), (1, "caída del servidor")]:
    tareas.put((prioridad, texto))

h = threading.Thread(target=trabajador, name="worker")
h.start()
tareas.join()                    # espera a que se procesen todas
tareas.put((99, None))           # orden de terminar
h.join()
```

> **Actualización del currículo.** El currículo pide establecer y controlar la prioridad de cada uno de los hilos. En Python esto no es posible: el módulo `threading` no ofrece prioridades, por los motivos explicados. El criterio se trabaja con las dos técnicas anteriores (cola de prioridad de tareas y prioridad del proceso) y analizando por qué la biblioteca del lenguaje no ofrece esta posibilidad.

2.6 Compartir información entre hilos y sus problemas

Compartir información entre hilos es trivial: basta con que accedan a la misma variable u objeto. El problema es hacerlo **correctamente**.

2.6.1 Condición de carrera

```
# Dos cajeros retiran dinero de la misma cuenta
import threading
import time

saldo = 100

def retirar(cantidad: int) -> None:
    global saldo
    if saldo >= cantidad:            # 1. comprobar
        time.sleep(0.01)             #    (el hilo pierde la CPU aquí)
        saldo -= cantidad            # 2. actuar
        print(f"{threading.current_thread().name} retira {cantidad}")

cajeros = [threading.Thread(target=retirar, args=(80,), name=f"cajero{i}")
           for i in range(2)]
for c in cajeros: c.start()
for c in cajeros: c.join()
print("Saldo final:", saldo)         # -60: ¡se ha retirado dos veces!
```

Los dos hilos comprueban que hay saldo *antes* de que ninguno lo descuente. Es el patrón «comprobar y luego actuar», típico de las condiciones de carrera. El `sleep` sólo hace que el fallo ocurra siempre; sin él ocurriría de vez en cuando, que es aún peor porque es difícil de reproducir.

2.6.2 Catálogo de problemas de la concurrencia

Condición de carrera
El resultado depende del orden de ejecución de los hilos. Se evita con exclusión mutua.

Interbloqueo (*deadlock*)
Dos o más hilos esperan indefinidamente recursos que tiene otro hilo del grupo. Por ejemplo, A tiene el cerrojo 1 y espera el 2, mientras B tiene el 2 y espera el 1.

Inanición (*starvation*)
Un hilo nunca consigue el recurso porque otros se lo quitan continuamente.

Bloqueo activo (*livelock*)
Los hilos no están bloqueados, pero cambian de estado continuamente en respuesta a los otros sin avanzar (como dos personas que se apartan hacia el mismo lado en un pasillo).

Inversión de prioridad
Un hilo de alta prioridad espera un recurso que tiene uno de baja prioridad, que a su vez no avanza porque otros de prioridad media le quitan la CPU.

2.7 Sincronización de hilos

2.7.1 `Lock`: exclusión mutua

Un cerrojo tiene dos estados: libre u ocupado. `acquire()` lo ocupa (o espera si ya lo está) y `release()` lo libera. Con `with` se garantiza que siempre se libera, incluso si hay una excepción:

```
cerrojo = threading.Lock()

def retirar(cantidad: int) -> None:
    global saldo
    with cerrojo:                    # comprobar y actuar, juntos y en exclusiva
        if saldo >= cantidad:
            time.sleep(0.01)
            saldo -= cantidad
            print(f"{threading.current_thread().name} retira {cantidad}")
        else:
            print(f"{threading.current_thread().name}: saldo insuficiente")
```

Buenas prácticas con cerrojos.

- Las secciones críticas deben ser **lo más cortas posible**: mientras un hilo tiene el cerrojo, los demás esperan.
- Lo ideal es que el cerrojo sea un atributo de la clase que contiene los datos y que sólo sus métodos lo usen (encapsulación).
- `acquire(timeout=2)` devuelve `False` si no consigue el cerrojo a tiempo; útil para detectar interbloqueos.

2.7.2 `RLock`: cerrojo reentrante

Si un hilo que ya tiene un `Lock` intenta adquirirlo otra vez, se bloquea a sí mismo para siempre. Un `RLock` puede ser adquirido varias veces por *el mismo* hilo (hay que liberarlo el

mismo número de veces). Es útil cuando un método protegido llama a otro método protegido del mismo objeto:

```
class Cuenta:
    def __init__(self, saldo: float):
        self.saldo = saldo
        self._cerrojo = threading.RLock()

    def retirar(self, cantidad: float) -> bool:
        with self._cerrojo:
            if self.saldo < cantidad:
                return False
            self.saldo -= cantidad
            return True

    def ingresar(self, cantidad: float) -> None:
        with self._cerrojo:
            self.saldo += cantidad

    def transferir_a(self, otra: "Cuenta", cantidad: float) -> bool:
        with self._cerrojo:                  # 1.ª adquisición
            if self.retirar(cantidad):       # 2.ª adquisición: con Lock se
                otra.ingresar(cantidad)      #   quedaría bloqueado aquí
                return True
            return False
```

2.7.3 `Semaphore`: limitar el acceso

Un semáforo tiene un contador. `acquire()` lo decrementa (esperando si es 0) y `release()` lo incrementa. Permite que hasta n hilos usen un recurso a la vez:

```
import random, threading, time

plazas = threading.Semaphore(3)          # aparcamiento con 3 plazas

def coche(matricula: str) -> None:
    print(f"{matricula} llega")
    with plazas:
        print(f"   {matricula} aparca")
        time.sleep(random.uniform(0.5, 1.5))
        print(f"   {matricula} se va")

for i in range(8):
    threading.Thread(target=coche, args=(f"{1000 + i}-BCD",)).start()
```

`BoundedSemaphore` es igual, pero lanza un error si se hace más `release()` que `acquire()`, lo que ayuda a detectar fallos.

2.7.4 `Event`: avisar a otros hilos

Un evento es una bandera que unos hilos esperan (`wait()`) y otro activa (`set()`). Ya lo hemos usado para detener y pausar hilos. Otro uso típico es la «salida» simultánea:

```
import threading, time
```

```
salida = threading.Event()

def corredor(n: int) -> None:
    salida.wait()                          # todos bloqueados hasta la señal
    print(f"¡Corredor {n} sale!")

for i in range(4):
    threading.Thread(target=corredor, args=(i,)).start()
time.sleep(1); print("Preparados... ¡ya!"); salida.set()
```

2.7.5 `Condition`: esperar a que se cumpla una condición

Una condición combina un cerrojo con una cola de espera. Un hilo que comprueba que no puede continuar llama a `wait()`, que **libera el cerrojo** y lo bloquea; otro hilo, al cambiar el estado, llama a `notify()` o `notify_all()` para despertarlo. Es el mecanismo que hay detrás de las colas.

```
# Productor--consumidor con búfer limitado y Condition
import threading
import time
import random

class Bufer:
    def __init__(self, capacidad: int):
        self._datos: list[int] = []
        self._capacidad = capacidad
        self._cond = threading.Condition()

    def poner(self, x: int) -> None:
        with self._cond:
            while len(self._datos) >= self._capacidad:   # ¡while, no if!
                self._cond.wait()                         # lleno: espera
            self._datos.append(x)
            self._cond.notify_all()                       # avisa a consumidores

    def sacar(self) -> int:
        with self._cond:
            self._cond.wait_for(lambda: len(self._datos) > 0)
            x = self._datos.pop(0)
            self._cond.notify_all()                       # avisa a productores
            return x

bufer = Bufer(capacidad=3)

def productor():
    for i in range(10):
        bufer.poner(i); print(f"P -> {i}")
        time.sleep(random.random() * 0.1)

def consumidor():
    for _ in range(10):
        x = bufer.sacar(); print(f"      C <- {x}")
        time.sleep(random.random() * 0.3)

hilos = [threading.Thread(target=productor), threading.Thread(target=consumidor)]
for h in hilos: h.start()
```

```
for h in hilos: h.join()
```

`wait()` siempre dentro de un `while`. Cuando un hilo despierta de `wait()`, la condición podría volver a ser falsa (otro hilo se le adelantó, o se trata de un «despertar espurio»). Por eso se comprueba en un bucle `while`, o se usa `wait_for(predicado)` que hace el bucle por nosotros.

2.7.6 `Barrier`: punto de encuentro

Una barrera para n hilos bloquea a cada hilo que llama a `wait()` hasta que han llegado los n. Útil en algoritmos por fases (todos terminan la fase 1 antes de que nadie empiece la 2).

```
import random, threading, time

barrera = threading.Barrier(3)

def fase(n: int) -> None:
    time.sleep(random.random())
    print(f"Hilo {n} terminó la fase 1")
    barrera.wait()
    print(f"Hilo {n} empieza la fase 2")

for i in range(3):
    threading.Thread(target=fase, args=(i,)).start()
```

2.7.7 Colas seguras: el módulo `queue`

En la mayoría de los casos, la forma más sencilla y segura de compartir información entre hilos es **no compartir variables, sino enviarse mensajes** a través de una cola. `queue.Queue` ya está sincronizada internamente:

`Queue(maxsize=0)`	FIFO. Con `maxsize` > 0, `put()` bloquea si está llena.
`LifoQueue`	Pila (el último en entrar sale primero).
`PriorityQueue`	Sale primero el elemento menor.
`put(x)`, `get()`	Insertar y extraer (bloqueantes; admiten `timeout`).
`task_done()`, `join()`	Marcar una tarea como terminada y esperar a que se hayan terminado todas.
`shutdown()`	(Python 3.13+) Cierra la cola: los `get()` en espera lanzan `queue.ShutDown`.

2.8 Interbloqueos

```
# Interbloqueo al transferir dinero en sentidos opuestos
import threading, time

cerrojo_a, cerrojo_b = threading.Lock(), threading.Lock()

def transferencia(origen, destino, nombre):
    with origen:
```

```
        time.sleep(0.1)
        print(f"{nombre}: tengo el primero, espero el segundo...")
        with destino:
            print(f"{nombre}: transferencia hecha")

threading.Thread(target=transferencia, args=(cerrojo_a, cerrojo_b, "T1")).start()
threading.Thread(target=transferencia, args=(cerrojo_b, cerrojo_a, "T2")).start()
# El programa se queda colgado para siempre
```

Un interbloqueo sólo puede producirse si se cumplen a la vez las cuatro **condiciones de Coffman**: exclusión mutua, retención y espera (un hilo retiene un recurso mientras espera otro), no expropiación (no se le puede quitar el recurso) y espera circular. Basta con romper una para evitarlo. La técnica más práctica es eliminar la espera circular **adquiriendo siempre los cerrojos en el mismo orden global**:

```
def transferencia(c1, c2, nombre):
    primero, segundo = sorted((c1, c2), key=id)   # orden global fijo
    with primero, segundo:
        print(f"{nombre}: transferencia hecha")
```

2.9 Conjuntos de hilos: `ThreadPoolExecutor`

Crear un hilo por tarea está bien para unas pocas tareas, pero si hay miles conviene reutilizar un número fijo de hilos. `ThreadPoolExecutor` tiene la misma interfaz que `ProcessPoolExecutor` del tema anterior:

```
# Comprobar el estado de varias webs en paralelo
from concurrent.futures import ThreadPoolExecutor, as_completed
import urllib.request
import time

URLS = ["https://www.python.org", "https://docs.python.org",
        "https://pypi.org", "https://www.educa.jcyl.es",
        "https://url-que-no-existe.invalid"]

def comprobar(url: str) -> tuple[str, int, float]:
    inicio = time.perf_counter()
    with urllib.request.urlopen(url, timeout=5) as resp:
        return url, resp.status, time.perf_counter() - inicio

with ThreadPoolExecutor(max_workers=5) as pool:
    futuros = [pool.submit(comprobar, u) for u in URLS]
    for f in as_completed(futuros):
        try:
            url, estado, t = f.result()
            print(f"{estado}  {t:5.2f} s  {url}")
        except Exception as e:
            print(f"ERROR  {e}")
```

> **¿Y `asyncio`?** Python ofrece un tercer modelo de concurrencia: la **programación asíncrona** con `async/await`. Un único hilo alterna entre miles de tareas que esperan E/S, sin el coste de crear hilos ni los problemas de sincronización. Es la base de los servidores web

modernos. Lo veremos aplicado a redes en el Tema 3.

2.10 Depuración de programas multihilo

- Usa `logging` con `%(threadName)s` en el formato y da nombres significativos a los hilos (`name="productor-1"`).
- Las excepciones no capturadas en un hilo se imprimen pero no detienen el programa. Puedes interceptarlas todas asignando una función a `threading.excepthook`.
- Si el programa se queda colgado (sospecha de interbloqueo), el módulo `faulthandler` muestra dónde está cada hilo: `faulthandler.dump_traceback_later(10)` vuelca las pilas a los 10 segundos.
- En VS Code, el panel *Call Stack* muestra todos los hilos; puedes pausar uno concreto e inspeccionar sus variables locales.
- Los errores de concurrencia son **no deterministas**: que un programa funcione diez veces no demuestra que sea correcto. Añadir `time.sleep()` en puntos clave ayuda a provocar los fallos.

Resumen del tema.

- Los hilos comparten la memoria del proceso; cada uno tiene su pila y su contexto. Son ideales para tareas de E/S.
- Se crean con `Thread(target=...)` o heredando de `Thread`; se lanzan con `start()` y se espera su fin con `join()`. Se finalizan de forma cooperativa (`Event`).
- Python no tiene prioridades de hilo: se usa `PriorityQueue` para priorizar tareas y `nice` para procesos.
- Compartir datos sin sincronizar provoca condiciones de carrera; un mal uso de la sincronización provoca interbloqueos e inanición.
- Herramientas: `Lock`, `RLock`, `Semaphore`, `Event`, `Condition`, `Barrier`, `queue.Queue` y `ThreadPoolExecutor`.

2.11 Ejercicios

Ejercicio 2.1 — Hilos con nombre

Dificultad: ★

Crea 5 hilos, cada uno con un nombre distinto, que muestren 5 veces un mensaje con su nombre, su `native_id` y el número de iteración, con una pausa aleatoria entre mensajes. El hilo principal debe esperar a todos y mostrar cuánto tiempo ha tardado en total. Usa `logging`.

Ejercicio 2.2 — Estados de un hilo

Dificultad: ★

Escribe un programa que cree un hilo y muestre el valor de `is_alive()` y el contenido de `threading.enumerate()` en cada uno de estos momentos: antes de `start()`, mientras está

dormido, mientras espera un `Event` y después de `join()`. Explica en comentarios en qué estado del diagrama se encuentra en cada caso.

Ejercicio 2.3 — Cronómetro con pausa

Dificultad: ★

Programa un cronómetro de consola en un hilo que muestre los segundos transcurridos. El hilo principal lee del teclado: `p` pausa, `r` reanuda, `s` detiene el cronómetro y termina el programa.

Ejercicio 2.4 — Contador seguro

Dificultad: ★★

Crea una clase `ContadorSeguro` con los métodos `incrementar()`, `decrementar()` y `valor()` protegidos internamente con un cerrojo. Lanza 10 hilos que incrementen 10 000 veces y 5 que decrementen 10 000 veces y comprueba que el resultado es siempre 50 000. Mide cuánto tarda con y sin cerrojo y explica la diferencia.

Ejercicio 2.5 — Sala de exámenes

Dificultad: ★★

En una sala caben 4 alumnos a la vez. Llegan 15 alumnos en momentos aleatorios; cada uno tarda entre 1 y 3 segundos en hacer el examen. Usa un semáforo para controlar el aforo, muestra en cada momento cuántos alumnos hay dentro y, con una barrera, haz que el profesor (otro hilo) espere a que todos hayan terminado para publicar las notas.

Ejercicio 2.6 — La cena de los filósofos

Dificultad: ★★★

Cinco filósofos se sientan alrededor de una mesa con un tenedor entre cada dos. Cada filósofo piensa y come alternativamente, y para comer necesita los dos tenedores que tiene a los lados. Implementa el problema con un hilo por filósofo y un `Lock` por tenedor. Primero, una versión ingenua que provoque interbloqueo; después, dos soluciones distintas (por ejemplo, orden global de adquisición y un camarero implementado con un semáforo de 4). Comprueba que ningún filósofo sufre inanición contando cuántas veces come cada uno.

Ejercicio 2.7 — Prioridad de tareas

Dificultad: ★★

Amplía el ejemplo de `PriorityQueue` con 3 hilos trabajadores y un hilo generador que crea tareas con prioridades aleatorias. Para evitar la inanición de las tareas de baja prioridad, implementa *envejecimiento*: cada segundo, un hilo de mantenimiento mejora la prioridad de las tareas que llevan más de 3 segundos esperando.

Proyecto: Gestor de descargas con interfaz gráfica

Desarrolla con `tkinter` un gestor de descargas que permita descargar varios ficheros a la vez sin que la interfaz se congele.
Requisitos:

1. Un campo de texto para introducir una URL y un botón *Añadir*. Cada descarga aparece en una lista con su nombre, una barra de progreso (`ttk.Progressbar`), la velocidad y su estado (en cola, descargando, pausada, completada, error, cancelada).
2. Cada descarga se ejecuta en su propio hilo, leyendo la respuesta de `urllib.request.urlopen` por bloques y escribiendo en disco.
3. Como máximo habrá N descargas simultáneas (configurable), controlado con un semáforo; el resto queda *en cola*.
4. Botones para pausar, reanudar y cancelar cada descarga, implementados con `Event` (finalización cooperativa; al cancelar se borra el fichero parcial).
5. **Tkinter no es seguro con hilos**: los hilos no pueden tocar la interfaz. Deben enviar sus actualizaciones a una `queue.Queue`, que el hilo de la interfaz consulta periódicamente con `root.after()`.
6. Registro con `logging` de todas las operaciones, indicando el hilo.

Ampliación: permite priorizar descargas (las de mayor prioridad salen antes de la cola) y guarda la lista de descargas pendientes al cerrar la aplicación para retomarlas al abrirla.

TEMA 3

Programación de comunicaciones en red

En este tema aprenderás a…

- Reconocer los escenarios que requieren comunicación en red y los modelos de comunicación (cliente–servidor, P2P, publicador–suscriptor…).
- Distinguir TCP de UDP y los distintos tipos de *sockets*.
- Programar clientes y servidores TCP y UDP con el módulo `socket`.
- Diseñar protocolos de aplicación y resolver el problema de las fronteras de los mensajes.
- Atender a varios clientes a la vez con hilos, `socketserver` y `asyncio`.
- Depurar aplicaciones en red con `netcat` y Wireshark.

Resultado de aprendizaje asociado: **RA3**.

3.1 Comunicación entre aplicaciones

Casi ninguna aplicación actual funciona aislada. Una app de mensajería envía y recibe mensajes de un servidor; un videojuego sincroniza la posición de los jugadores; una app móvil consulta una API; un sensor envía lecturas a una plataforma en la nube; un sistema de pagos se comunica con el banco. Todos estos **escenarios** requieren que dos o más programas, normalmente en máquinas distintas, intercambien datos a través de una red.

3.1.1 Modelos de comunicación

Cliente–servidor
Un **servidor** ofrece un servicio y espera peticiones; los **clientes** toman la iniciativa y le hacen peticiones. Es el modelo de la web, el correo o las bases de datos. Centralizado y fácil de gestionar, pero el servidor es un punto único de fallo y de carga.

Arquitecturas de n capas
Variante del anterior en la que el servidor se divide en capas (presentación, lógica de negocio, datos), cada una de las cuales puede ser cliente de la siguiente.

Entre iguales (P2P)
Todos los nodos son a la vez clientes y servidores. No hay punto central: BitTorrent, algunas criptomonedas, llamadas de vídeo directas (WebRTC).

Publicador–suscriptor
Los emisores publican mensajes en *temas* de un intermediario (*broker*) y los receptores se suscriben a los temas que les interesan, sin conocerse entre sí. Muy usado en IoT (MQTT) y en sistemas con colas de mensajes (RabbitMQ, Kafka).

Llamada a procedimiento remoto (RPC)
El cliente invoca una función que se ejecuta en otra máquina como si fuera local (gRPC, XML-RPC, JSON-RPC).

Microservicios
La aplicación se divide en servicios pequeños e independientes que se comunican por la red, normalmente con HTTP/REST o mensajes.

3.2 Repaso de redes

Capa TCP/IP	Función	Ejemplos
Aplicación	Protocolos de cada servicio	HTTP, SMTP, DNS, SSH, **el protocolo que diseñemos nosotros**
Transporte	Comunicación entre *procesos*, identificados por puertos	**TCP**, **UDP**
Internet	Encaminar paquetes entre *máquinas*	IPv4, IPv6, ICMP
Acceso a la red	Transmisión física en la red local	Ethernet, Wi-Fi

- La **dirección IP** identifica una máquina (más exactamente, una interfaz de red). `127.0.0.1` (o `localhost`) es la propia máquina: la usaremos para hacer pruebas.
- El **puerto** (0–65535) identifica un *proceso* dentro de la máquina. Los puertos 0–1023 están reservados a servicios conocidos (80 HTTP, 443 HTTPS, 22 SSH…) y requieren privilegios; para nuestras prácticas usaremos puertos por encima de 1024.
- Una comunicación queda identificada por el par `(IP origen, puerto origen)` – `(IP destino, puerto destino)`.

3.2.1 TCP frente a UDP

	TCP	UDP
Conexión	Orientado a conexión (hay que establecerla)	Sin conexión
Fiabilidad	Garantiza entrega, orden y ausencia de duplicados	Los datagramas pueden perderse, duplicarse o llegar desordenados
Datos	**Flujo** continuo de bytes, sin fronteras entre mensajes	**Datagramas** independientes, con fronteras
Velocidad	Más sobrecarga	Más ligero y con menos latencia
Usos	Web, correo, transferencia de ficheros, bases de datos, chats	DNS, streaming, voz/vídeo, juegos en tiempo real, descubrimiento de servicios

3.3 Sockets

Un **socket** es un extremo de una comunicación entre dos procesos a través de la red. Es la interfaz que el sistema operativo ofrece a los programas para usar los protocolos de transporte: el programa escribe y lee datos del socket y el SO se encarga de que lleguen al otro extremo. La API de sockets nació en BSD Unix en los años 80 y es prácticamente idéntica en todos los lenguajes y sistemas.

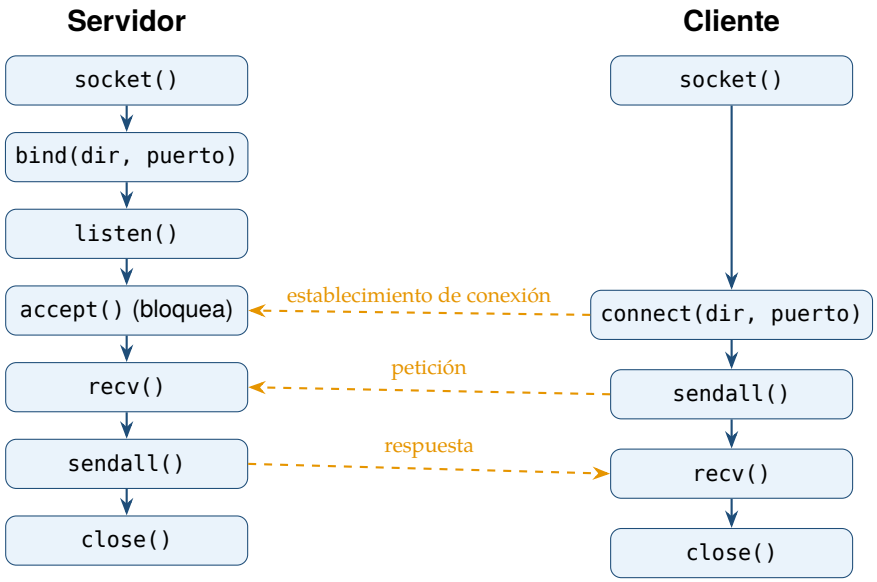
Un socket se caracteriza por:

- Su **familia** de direcciones: `AF_INET` (IPv4), `AF_INET6` (IPv6) o `AF_UNIX` (comunicación local entre procesos de la misma máquina, a través de un fichero especial).
- Su **tipo**: `SOCK_STREAM` (**socket de flujo**, usa TCP), `SOCK_DGRAM` (**socket de datagramas**, usa UDP) o `SOCK_RAW` (acceso directo a IP; requiere privilegios y se usa en herramientas de red como `ping`).
- La dirección y el puerto a los que está asociado.

En Python, todo esto está en el módulo estándar `socket`. Por encima de él, la librería estándar ofrece `socketserver` (para simplificar servidores), `selectors` (para atender varios sockets con un solo hilo) y `asyncio` (programación asíncrona).

3.4 Sockets TCP

3.4.1 Secuencia de llamadas



`bind((host, puerto))`
Asocia el socket a una dirección y puerto locales. Con `host "127.0.0.1"` sólo se aceptan conexiones desde la propia máquina; con `"0.0.0.0"` (o `""`) desde cualquier interfaz de red.

`listen(n)`
Convierte el socket en un **socket de escucha**. `n` es el número de conexiones pendientes que el SO mantiene en cola.

`accept()`
Bloquea hasta que llega un cliente y devuelve **un socket nuevo** para hablar con ese cliente, junto con su dirección. El socket de escucha sigue disponible para aceptar a otros clientes.

`connect((host, puerto))`
Establece la conexión con el servidor (el famoso *three-way handshake* de TCP).

`sendall(datos)` y `recv(n)`
Envían y reciben **bytes**. `recv(n)` devuelve *como mucho* n bytes, los que haya disponibles; si el otro extremo ha cerrado la conexión, devuelve `b""`.

`close()`
Cierra el socket. Con `with` se cierra automáticamente.

3.4.2 Primer ejemplo: servidor de eco

```python
# servidor_eco.py
import socket

HOST, PUERTO = "127.0.0.1", 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
    # Permite reutilizar el puerto nada más cerrar el servidor
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    servidor.bind((HOST, PUERTO))
    servidor.listen()
    print(f"Servidor escuchando en {HOST}:{PUERTO}")

    while True:                                  # atiende clientes de uno en uno
        conexion, direccion = servidor.accept()
        with conexion:
            print("Conectado:", direccion)
            while datos := conexion.recv(1024):  # b"" => el cliente cerró
                conexion.sendall(datos.upper())
            print("Desconectado:", direccion)
```

```python
# cliente_eco.py
import socket

HOST, PUERTO = "127.0.0.1", 5000

with socket.create_connection((HOST, PUERTO), timeout=5) as s:
    while (linea := input("> ")) != "":
        s.sendall(linea.encode("utf-8"))
        respuesta = s.recv(1024)
        print("Servidor:", respuesta.decode("utf-8"))
```

`socket.create_connection` crea el socket y conecta en un solo paso, y funciona tanto con IPv4 como con IPv6. Ejecuta el servidor en un terminal y el cliente en otro. Fíjate en que el servidor atiende a los clientes *de uno en uno*: un segundo cliente se queda esperando hasta que el primero termina. Lo resolveremos con hilos.

> **¿Por qué «adiós» vuelve como «ADIóS»?** El servidor trabaja con **bytes**, y `bytes.upper()` sólo convierte letras ASCII; la «ó» son dos bytes que no cambian. Para tratar el texto habría que decodificarlo, pero ¡cuidado!: un `recv()` podría cortar un carácter de dos bytes por la mitad y `decode()` fallaría. Es otra cara del problema que se explica a continuación.

> **Probar sin escribir el cliente: `netcat`.** Puedes conectarte a cualquier servidor TCP con `nc 127.0.0.1 5000` (Linux y macOS) y escribir a mano lo que se envía. En Windows puedes usar `ncat` (incluido con Nmap) o el cliente de Telnet. Es la herramienta básica para depurar servidores.

3.4.3 El problema de las fronteras de mensaje

El ejemplo anterior tiene un error que sólo aparece en condiciones reales. TCP transmite un **flujo de bytes**, no mensajes: si el cliente hace dos `sendall()` seguidos, el servidor puede recibir ambos en un único `recv()`; y un mensaje largo puede llegar troceado en varios `recv()`. **Nunca debemos suponer que un `recv()` corresponde a un mensaje completo.**

Para separar los mensajes, el protocolo de aplicación debe definir un **enmarcado** (*framing*). Las dos técnicas habituales son:

1. **Delimitador**: cada mensaje termina con un carácter especial, normalmente el salto de línea `\n` (así funcionan HTTP, SMTP o Redis). El receptor acumula bytes hasta encontrarlo.
2. **Prefijo de longitud**: cada mensaje va precedido de su tamaño en un número fijo de bytes. Es ideal para datos binarios.

Mensajes por líneas con `makefile`

El método `makefile()` envuelve el socket en un objeto tipo fichero, que ya sabe leer «hasta el siguiente salto de línea» acumulando lo necesario:

```
import socket

with socket.create_connection(("127.0.0.1", 5000)) as s:
    f = s.makefile("rw", encoding="utf-8", newline="\n")
    f.write("HOLA servidor\n")
    f.flush()                       # ¡importante! envía lo acumulado
    respuesta = f.readline()        # lee hasta '\n' (o "" si se cerró)
    print(respuesta.rstrip("\n"))
```

Mensajes con prefijo de longitud

```
# mensajes.py: enviar y recibir objetos JSON
import json
import socket
import struct

CABECERA = struct.Struct("!I")      # entero sin signo de 4 bytes, orden de red

def recibir_exacto(sock: socket.socket, n: int) -> bytes:
    """Recibe exactamente n bytes o lanza ConnectionError."""
    datos = bytearray()
    while len(datos) < n:
        trozo = sock.recv(n - len(datos))
        if not trozo:
            raise ConnectionError("conexión cerrada por el otro extremo")
        datos.extend(trozo)
    return bytes(datos)

def enviar_mensaje(sock: socket.socket, objeto) -> None:
    cuerpo = json.dumps(objeto).encode("utf-8")
    sock.sendall(CABECERA.pack(len(cuerpo)) + cuerpo)

def recibir_mensaje(sock: socket.socket):
```

```
    (longitud,) = CABECERA.unpack(recibir_exacto(sock, CABECERA.size))
    return json.loads(recibir_exacto(sock, longitud).decode("utf-8"))
```

Con este módulo, cliente y servidor intercambian diccionarios Python sin preocuparse de cómo viajan: `enviar_mensaje(s, {"orden": "SUMA", "args": [3, 4]})`.

3.4.4 Timeouts y errores

Por defecto las operaciones de un socket bloquean indefinidamente. En un programa real hay que poner un límite y tratar los errores de red:

```
try:
    with socket.create_connection(("192.168.1.50", 5000), timeout=3) as s:
        s.settimeout(10)              # máximo 10 s esperando cada recv()
        s.sendall(b"PING\n")
        print(s.recv(1024))
except ConnectionRefusedError:
    print("No hay ningún servidor escuchando en ese puerto")
except TimeoutError:
    print("El servidor no responde")
except OSError as e:
    print("Error de red:", e)
```

3.5 Sockets UDP

Con UDP no hay conexión ni `listen`/`accept`: el servidor hace `bind` y recibe datagramas de cualquiera; cada `recvfrom()` devuelve **un datagrama completo** y la dirección de quien lo envió.

```
# Servidor UDP que devuelve la hora
import socket
from datetime import datetime

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind(("0.0.0.0", 6000))
    print("Servidor de hora UDP en el puerto 6000")
    while True:
        datos, origen = s.recvfrom(1024)
        print(f"{origen} pide: {datos.decode()}")
        s.sendto(datetime.now().isoformat().encode(), origen)
```

```
# Cliente UDP
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.settimeout(2)                  # UDP no avisa si el datagrama se pierde
    s.sendto(b"HORA", ("127.0.0.1", 6000))
    try:
        respuesta, _ = s.recvfrom(1024)
        print("Hora del servidor:", respuesta.decode())
    except TimeoutError:
        print("Sin respuesta (¿servidor caído o datagrama perdido?)")
```

3.5.1 Difusión (*broadcast*)

UDP permite enviar un datagrama a todas las máquinas de la red local, lo que se usa para **descubrir servicios** sin conocer su dirección (así encuentran las impresoras o los Chromecast):

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
s.settimeout(2)
s.sendto(b"DESCUBRIR", ("255.255.255.255", 6000))
try:
    while True:
        datos, servidor = s.recvfrom(1024)
        print("Encontrado servidor en", servidor[0], datos.decode())
except TimeoutError:
    pass
finally:
    s.close()
```

3.6 Diseñar un protocolo de aplicación

Antes de programar un cliente y un servidor hay que definir, por escrito, su **protocolo**: qué mensajes existen, en qué orden se envían, su formato y cómo se señalan los errores. Ejemplo para un servidor de calculadora basado en líneas de texto UTF-8:

Petición	Descripción	Respuesta
`SUMA a b`	Suma dos números	`OK resultado`
`MULT a b`	Multiplica dos números	`OK resultado`
`STATS`	Peticiones atendidas por el servidor	`OK n`
`SALIR`	Cierra la conexión	`OK adios`
(otra cosa)		`ERR mensaje`

Al conectarse, el servidor saluda con `OK Calculadora v1`. Cada mensaje termina en `\n`. Este tipo de diseño (orden en mayúsculas, respuesta que empieza por un código de estado) es el mismo que usan protocolos clásicos como SMTP o FTP.

3.7 Servidores con varios clientes simultáneos

3.7.1 Un hilo por cliente

La solución clásica es que el hilo principal sólo acepte conexiones y, por cada cliente, cree un hilo que lo atienda. Implementamos así el protocolo de la calculadora:

```
# servidor_calculadora.py: servidor multihilo
import logging
import socket
import threading

logging.basicConfig(level=logging.INFO,
```

```
                    format="%(asctime)s [%(threadName)s] %(message)s")

HOST, PUERTO = "0.0.0.0", 5001
peticiones = 0
cerrojo = threading.Lock()                # protege el contador compartido

def procesar(linea: str) -> tuple[str, bool]:
    """Devuelve (respuesta, cerrar_conexión)."""
    global peticiones
    with cerrojo:
        peticiones += 1
    match linea.split():
        case ["SUMA", a, b]:
            return f"OK {float(a) + float(b)}", False
        case ["MULT", a, b]:
            return f"OK {float(a) * float(b)}", False
        case ["STATS"]:
            with cerrojo:
                return f"OK {peticiones}", False
        case ["SALIR"]:
            return "OK adios", True
        case _:
            return "ERR orden desconocida", False

def atender(conexion: socket.socket, direccion) -> None:
    logging.info("Cliente conectado %s", direccion)
    with conexion, conexion.makefile("rw", encoding="utf-8") as f:
        f.write("OK Calculadora v1\n"); f.flush()
        for linea in f:                   # termina cuando el cliente cierra
            try:
                respuesta, cerrar = procesar(linea.strip())
            except ValueError:
                respuesta, cerrar = "ERR argumentos no numéricos", False
            f.write(respuesta + "\n"); f.flush()
            if cerrar:
                break
    logging.info("Cliente desconectado %s", direccion)

def main() -> None:
    with socket.create_server((HOST, PUERTO)) as servidor:
        logging.info("Escuchando en %s:%d", HOST, PUERTO)
        while True:
            conexion, direccion = servidor.accept()
            hilo = threading.Thread(target=atender, args=(conexion, direccion),
                                    name=f"cliente-{direccion[1]}", daemon=True)
            hilo.start()

if __name__ == "__main__":
    main()
```

`socket.create_server` hace en una línea el `socket()`, `bind()` y `listen()`, y activa `SO_REUSEADDR`.

> **Estado compartido.** Todos los hilos del servidor comparten la memoria. Cualquier dato común (el contador de peticiones, la lista de usuarios conectados de un chat, una partida) debe protegerse con los mecanismos del Tema 2.

3.7.2 Limitar el número de hilos

Un hilo por cliente no escala a miles de clientes y un atacante podría agotar los recursos abriendo conexiones. Con un `ThreadPoolExecutor` se fija el máximo de clientes atendidos simultáneamente (los demás esperan):

```
from concurrent.futures import ThreadPoolExecutor

with socket.create_server((HOST, PUERTO)) as servidor, \
     ThreadPoolExecutor(max_workers=50) as pool:
    while True:
        conexion, direccion = servidor.accept()
        pool.submit(atender, conexion, direccion)
```

3.7.3 `socketserver`

El módulo `socketserver` encapsula el bucle de aceptación. Basta con escribir una clase manejadora:

```
import socketserver

class ManejadorEco(socketserver.StreamRequestHandler):
    def handle(self) -> None:              # se ejecuta en un hilo por cliente
        for linea in self.rfile:           # self.rfile / self.wfile: bytes
            self.wfile.write(linea.upper())

class Servidor(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True

if __name__ == "__main__":
    with Servidor(("127.0.0.1", 5000), ManejadorEco) as srv:
        srv.serve_forever()
```

3.7.4 Un solo hilo para todos: `asyncio`

La alternativa moderna a los hilos para servidores con muchas conexiones es la programación asíncrona. Una función `async def` es una **corrutina**: cuando llega a un `await` sobre una operación que tendría que esperar (recibir datos, por ejemplo), cede el control al **bucle de eventos**, que ejecuta mientras tanto otras corrutinas.

```
# Servidor de eco asíncrono
import asyncio

async def atender(lector: asyncio.StreamReader,
                  escritor: asyncio.StreamWriter) -> None:
    direccion = escritor.get_extra_info("peername")
    print("Conectado", direccion)
    while linea := await lector.readline():   # cede el control mientras espera
        escritor.write(linea.upper())
        await escritor.drain()
    escritor.close()
    await escritor.wait_closed()
    print("Desconectado", direccion)
```

```python
async def main() -> None:
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())
```

	Hilos	asyncio
Cambio de tarea	Lo decide el SO en cualquier momento	Sólo en los `await`: el código entre dos `await` no se interrumpe
Sincronización	Necesaria (cerrojos)	Mucho menos necesaria
Escalabilidad	Cientos o pocos miles de conexiones	Decenas de miles
Librerías	Cualquiera	Deben ser asíncronas (una llamada bloqueante congela todo el servidor)

3.8 Depuración de aplicaciones en red

- `nc` / `ncat`: hacerse pasar por cliente o por servidor (`nc -l 5000`) para probar cada parte por separado.
- `ss -tlnp` (Linux) o `netstat -ano` (Windows): qué procesos están escuchando y en qué puertos. El error `Address already in use` indica que el puerto está ocupado (¿otra instancia del servidor?).
- **Wireshark**: captura y muestra el tráfico real, byte a byte. Con el filtro `tcp.port == 5000` verás el establecimiento de conexión, tus mensajes y el cierre.
- Registra con `logging` cada mensaje recibido y enviado (con `repr()` para ver los `\n` y caracteres especiales) y la dirección del cliente.
- Documenta siempre el **protocolo** (tabla de mensajes y ejemplo de diálogo), no sólo el código.

Resumen del tema.

- Las aplicaciones distribuidas siguen modelos como cliente–servidor, P2P o publicador–suscriptor.
- Un socket es un extremo de comunicación; los de flujo usan TCP (fiable, flujo de bytes) y los de datagrama UDP (rápido, sin garantías).
- Servidor TCP: `bind`, `listen`, `accept`; cliente: `connect`. Se envían y reciben **bytes**.
- TCP no conserva las fronteras de los mensajes: el protocolo debe definir un enmarcado (líneas o prefijo de longitud).
- Para atender a varios clientes a la vez: un hilo por cliente, un conjunto de hilos, `socketserver` o `asyncio`.

3.9 Ejercicios

Ejercicio 3.1 — Escenarios

Dificultad: ★

Para cada una de estas aplicaciones indica qué modelo de comunicación usarías, si emplearías TCP o UDP y por qué: (a) un juego de disparos en línea; (b) una app de banca; (c) una red de sensores de temperatura en un invernadero; (d) un sistema para compartir ficheros entre los ordenadores del aula sin servidor; (e) un servicio de videoconferencia.

Ejercicio 3.2 — Cliente de la calculadora

Dificultad: ★

Escribe el cliente del servidor de la calculadora. Debe recibir el host y el puerto por argumentos, mostrar el saludo del servidor, enviar las órdenes que escriba el usuario y mostrar las respuestas, distinguiendo con colores las `OK` de las `ERR`. Prueba antes el servidor con `nc`.

Ejercicio 3.3 — Adivina el número

Dificultad: ★

Servidor TCP que, para cada cliente, elige un número aleatorio entre 1 y 100 y responde `MAYOR`, `MENOR` o `ACIERTO n_intentos` a cada intento. Documenta el protocolo en una tabla antes de programarlo.

Ejercicio 3.4 — Transferencia de ficheros

Dificultad: ★★

Usando el enmarcado por prefijo de longitud, programa un servidor que permita a los clientes subir (`PUT nombre`) y descargar (`GET nombre`) ficheros binarios de cualquier tamaño, así como listarlos (`LIST`). El servidor debe impedir que un cliente acceda fuera de su carpeta (rutas con `..`). Verifica la integridad comparando el `SHA-256` del original y la copia.

Ejercicio 3.5 — Medición de UDP

Dificultad: ★★

Programa un cliente UDP que envíe 1000 datagramas numerados a un servidor que los devuelve, y calcule el porcentaje de pérdidas, los datagramas desordenados y el tiempo medio de ida y vuelta. Pruébalo en `localhost` y entre dos equipos por Wi-Fi.

Ejercicio 3.6 — Chat multiusuario

Dificultad: ★★★

Servidor de chat TCP con un hilo por cliente. Al conectarse, el cliente envía `NICK nombre`; después, todo lo que escriba se reenvía a todos los demás. Órdenes: `/lista` (usuarios conectados), `/privado nombre texto` y `/salir`. La lista de clientes conectados debe estar protegida con un cerrojo. El cliente necesita dos hilos: uno que lee del teclado y otro que muestra los mensajes que llegan del servidor.

Ejercicio 3.7 — Chat con asyncio

Dificultad: ★★★

Reescribe el servidor del ejercicio anterior con `asyncio`. Compara el número de líneas, la necesidad de cerrojos y el comportamiento al conectar 500 clientes simultáneos (escribe un script que los simule).

Proyecto: Trivial en red

Desarrolla un juego de preguntas multijugador para la red local.
Requisitos:

1. **Descubrimiento**: el servidor escucha en un puerto UDP y responde a los mensajes `DESCUBRIR` enviados por difusión con su nombre y su puerto TCP. El cliente muestra los servidores encontrados y permite elegir uno.
2. **Protocolo**: mensajes JSON con prefijo de longitud (reutiliza `mensajes.py`). Documenta todos los tipos de mensaje (`unirse`, `pregunta`, `respuesta`, `resultado`, `ranking`, `error`...) con un diagrama de secuencia.
3. **Servidor multihilo**: un hilo por jugador. La partida empieza cuando hay al menos 3 jugadores o cuando el primero envía `empezar`. Las preguntas se cargan de un fichero JSON.
4. Cada pregunta tiene 15 segundos para responder; responder antes da más puntos. El estado de la partida (jugadores, puntuaciones, respuestas) es compartido y debe estar correctamente sincronizado.
5. Si un jugador se desconecta, la partida continúa sin él.
6. Pruebas: un script que lance varios clientes automáticos que respondan al azar, para comprobar el servidor bajo carga.

Ampliación: cliente con interfaz gráfica (`tkinter`) o versión del servidor con `asyncio`.

TEMA 4

Generación de servicios en red

En este tema aprenderás a…

- Conocer los protocolos estándar de nivel de aplicación y las ventajas de usarlos.
- Entender el protocolo HTTP y usar clientes (`curl`, `requests`) para verificar servicios.
- Programar un servicio HTTP con `http.server`.
- Diseñar APIs REST siguiendo las buenas prácticas profesionales.
- Construir y probar un servicio REST completo con FastAPI.
- Enviar correo con SMTP y garantizar la eficiencia y la disponibilidad de los servicios.

Resultado de aprendizaje asociado: **RA4**.

4.1 Protocolos estándar de nivel de aplicación

En el tema anterior inventamos nuestros propios protocolos. Eso tiene sentido en casos muy concretos, pero lo habitual es usar **protocolos estándar**, definidos en documentos públicos (los RFC del IETF, las especificaciones del W3C…).

4.1.1 Ventajas de usar protocolos estándar

- **Interoperabilidad**: un servidor web escrito en Python lo puede usar cualquier navegador, una app Android o un script en Java.
- **Librerías y herramientas ya hechas**: clientes, servidores, depuradores, proxies, balanceadores… No hay que reinventarlos.
- **Fiabilidad y seguridad**: están revisados por miles de expertos y sus problemas son conocidos y documentados.
- **Infraestructura**: cortafuegos, proxies y CDN ya los entienden (casi cualquier red deja pasar HTTPS por el puerto 443; un protocolo propio en un puerto raro puede estar bloqueado).
- **Conocimiento compartido**: cualquier desarrollador los conoce; el mantenimiento es más sencillo.

4.1.2 Protocolos más importantes

Protocolo	Transp.	Puerto	Uso
HTTP / HTTPS	TCP (QUIC)	80 / 443	Web, APIs REST. El protocolo dominante.
WebSocket	TCP	80 / 443	Comunicación bidireccional en tiempo real (chats, notificaciones) sobre HTTP.
SMTP	TCP	25 / 587	Envío de correo.
IMAP / POP3	TCP	993 / 995	Lectura de correo (versiones seguras).
DNS	UDP/TCP	53	Traducción de nombres a direcciones IP.
SSH / SFTP	TCP	22	Acceso remoto seguro y transferencia de ficheros.
FTP	TCP	21	Transferencia de ficheros (histórico, sin cifrar).
MQTT	TCP	1883 / 8883	Publicador–suscriptor ligero para IoT.

Python incluye librerías estándar para casi todos: `http.client`, `http.server` y `urllib` (HTTP), `smtplib` y `email` (correo), `imaplib`, `poplib`, `ftplib`... y existen librerías externas muy extendidas para el resto: `requests` y `httpx` (clientes HTTP), FastAPI y Flask (servicios web), `websockets`, `paho-mqtt`, `paramiko` (SSH).

> **Actualización del currículo.** El currículo menciona FTP como ejemplo de protocolo estándar. Hoy está en desuso: transmite el usuario y la contraseña sin cifrar y los navegadores han eliminado su soporte; para transferir ficheros se usan HTTPS o SFTP (sobre SSH). Por eso este tema se centra en **HTTP y los servicios web**, que es lo que encontrarás en la empresa, y usa el correo electrónico (SMTP) como segundo ejemplo de protocolo estándar.

4.2 HTTP

HTTP es un protocolo de texto, de tipo petición–respuesta y **sin estado**: cada petición es independiente de las anteriores. Al estar basado en texto, podemos hablarlo «a mano» con un socket, como en el Tema 3:

```python
# Una petición HTTP con un socket
import socket

with socket.create_connection(("example.com", 80)) as s:
    s.sendall(b"GET / HTTP/1.1\r\n"
              b"Host: example.com\r\n"
              b"Connection: close\r\n"
              b"\r\n")
    respuesta = b""
    while trozo := s.recv(4096):
        respuesta += trozo

cabeceras, _, cuerpo = respuesta.partition(b"\r\n\r\n")
print(cabeceras.decode())
```

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256
Date: Thu, 15 Oct 2026 08:00:00 GMT
```

```
...
```

4.2.1 Estructura de los mensajes

- **Petición**: línea de petición (*método*, *ruta*, *versión*), cabeceras (`Nombre: valor`), una línea en blanco y, opcionalmente, un cuerpo.
- **Respuesta**: línea de estado (*versión*, *código*, *texto*), cabeceras, línea en blanco y cuerpo.
- Las líneas terminan en `\r\n`. La cabecera `Content-Length` indica el tamaño del cuerpo: es el enmarcado del que hablábamos en el tema anterior.

4.2.2 Métodos y códigos de estado

Métodos		Códigos de estado	
`GET`	Obtener un recurso	`200`	OK
`POST`	Crear un recurso / enviar datos	`201`	Creado
`PUT`	Reemplazar un recurso	`204`	Sin contenido
`PATCH`	Modificar parcialmente	`301/302`	Redirección
`DELETE`	Borrar un recurso	`400`	Petición incorrecta
`HEAD`	Como GET, sin cuerpo	`401/403`	No autenticado / prohibido
`OPTIONS`	Métodos permitidos	`404`	No encontrado
		`422`	Datos no válidos
		`500`	Error interno del servidor
		`503`	Servicio no disponible

> **Versiones de HTTP.** HTTP/1.1 (1997) es texto sobre TCP. HTTP/2 (2015) es binario y multiplexa varias peticiones por una conexión. HTTP/3 (2022) funciona sobre QUIC, un protocolo construido sobre UDP. La semántica (métodos, códigos, cabeceras) es la misma en las tres, así que para el programador apenas cambia nada.

4.3 Clientes HTTP

Para **verificar** un servicio (y para consumir servicios de terceros) necesitamos clientes. Los más usados son:

- El **navegador** y sus herramientas de desarrollo (pestaña *Red*), sólo para peticiones `GET` sencillas.
- `curl`, el cliente HTTP de línea de órdenes (incluido en Linux, macOS y Windows 10+).
- Clientes gráficos como **Bruno**, **Insomnia** o **Postman**, o la extensión *REST Client* de VS Code.
- Desde código: `urllib.request` (estándar) o `requests` (externa, mucho más cómoda).

```
$ curl -i https://api.github.com/repos/python/cpython
$ curl -X POST http://localhost:8000/tareas \
       -H "Content-Type: application/json" \
       -d '{"titulo": "Estudiar sockets", "prioridad": 2}'
```

4.3.1 `requests`

```python
import requests            # pip install requests

r = requests.get("https://api.github.com/repos/python/cpython", timeout=5)
print(r.status_code, r.headers["Content-Type"])
datos = r.json()           # convierte el cuerpo JSON en dict
print(datos["full_name"], datos["stargazers_count"])

# Parámetros en la URL (?q=...&per_page=...)
r = requests.get("https://api.github.com/search/repositories",
                 params={"q": "fastapi", "per_page": 3}, timeout=5)
for repo in r.json()["items"]:
    print(repo["full_name"])

# POST con cuerpo JSON y tratamiento de errores
try:
    r = requests.post("http://localhost:8000/tareas",
                      json={"titulo": "Repasar HTTP", "prioridad": 1},
                      timeout=5)
    r.raise_for_status()               # lanza excepción si es 4xx o 5xx
    print("Creada:", r.json())
except requests.ConnectionError:
    print("El servidor no está disponible")
except requests.HTTPError as e:
    print("Error HTTP:", e.response.status_code, e.response.text)
```

Si vamos a hacer muchas peticiones al mismo servidor, conviene crear una sesión con `requests.Session()`, que reutiliza la conexión TCP (*keep-alive*), lo que es mucho más eficiente, y mantiene cabeceras y *cookies* comunes.

4.4 Programar un servidor HTTP: `http.server`

La librería estándar incluye un servidor HTTP sencillo. Con él podemos ver cómo funciona un servicio por dentro antes de usar un *framework*.

```
$ python -m http.server 8000      # sirve los ficheros del directorio actual
```

Para programar nuestro propio servicio, heredamos de `BaseHTTPRequestHandler` y escribimos un método por cada verbo HTTP:

```python
# Servicio HTTP con la librería estándar
import json
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

visitas = 0
cerrojo = threading.Lock()

class Manejador(BaseHTTPRequestHandler):
    def _responder(self, codigo: int, datos: dict) -> None:
        cuerpo = json.dumps(datos, ensure_ascii=False).encode("utf-8")
        self.send_response(codigo)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(cuerpo)))
```

```
        self.end_headers()
        self.wfile.write(cuerpo)

    def do_GET(self) -> None:
        global visitas
        if self.path == "/saludo":
            with cerrojo:
                visitas += 1
                n = visitas
            self._responder(200, {"mensaje": "¡Hola!", "visitas": n})
        elif self.path == "/salud":
            self._responder(200, {"estado": "ok"})
        else:
            self._responder(404, {"error": "recurso no encontrado"})

    def do_POST(self) -> None:
        if self.path != "/eco":
            return self._responder(404, {"error": "recurso no encontrado"})
        longitud = int(self.headers.get("Content-Length", 0))
        try:
            datos = json.loads(self.rfile.read(longitud))
        except json.JSONDecodeError:
            return self._responder(400, {"error": "JSON no válido"})
        self._responder(200, {"recibido": datos})

if __name__ == "__main__":
    servidor = ThreadingHTTPServer(("127.0.0.1", 8000), Manejador)
    print("Servicio en http://127.0.0.1:8000")
    try:
        servidor.serve_forever()
    except KeyboardInterrupt:
        servidor.shutdown()
```

`ThreadingHTTPServer` atiende cada petición en un hilo distinto, por lo que **varios clientes pueden usar el servicio simultáneamente**; como siempre, el estado compartido (`visitas`) se protege con un cerrojo.

> **Cuidado.** `http.server` es perfecto para aprender y para herramientas internas, pero la documentación de Python advierte que **no debe usarse en producción**: no implementa los controles de seguridad ni el rendimiento de un servidor real.

4.5 Servicios web

Un **servicio web** es un componente software que ofrece funcionalidad a otras aplicaciones (no a personas) a través de la red usando protocolos web. Hay varios estilos:

REST
: El estilo dominante. Los datos se organizan en **recursos** identificados por URL (`/alumnos`, `/alumnos/42`) que se manipulan con los métodos HTTP. Normalmente intercambian JSON y son **sin estado**: cada petición lleva toda la información necesaria (incluida la autenticación).

SOAP
: Basado en mensajes XML con un formato muy estricto, descrito en un fichero WSDL. Se encuentra en sistemas antiguos (banca, administración), pero no se usa en desarrollos nue-

vos.

GraphQL
: El cliente envía una consulta indicando exactamente qué campos quiere. Útil cuando hay muchos clientes con necesidades distintas.

gRPC
: Llamadas a procedimientos remotos sobre HTTP/2 con mensajes binarios (Protocol Buffers). Muy eficiente; habitual entre microservicios.

4.6 Diseño de una API REST

Una API es un **contrato**: en cuanto otras aplicaciones empiezan a usarla, cambiarla tiene un coste, porque hay que modificar todos sus clientes (apps móviles ya instaladas, servicios de otros equipos…). Por eso se diseña *antes* de programarla, pensando en quien la va a usar.

4.6.1 Los principios de REST

REST (*Representational State Transfer*) es un estilo de arquitectura descrito por Roy Fielding en su tesis doctoral (2000). Una API es REST si respeta estas restricciones:

Cliente–servidor
: La interfaz (cliente) y el almacenamiento y la lógica (servidor) están separados y evolucionan de forma independiente.

Sin estado
: El servidor no guarda nada sobre la «conversación» con el cliente: cada petición lleva toda la información necesaria, incluida la autenticación. Así, cualquier instancia del servidor puede atender cualquier petición, lo que permite escalar añadiendo servidores.

Cacheable
: Las respuestas indican si pueden guardarse en caché y durante cuánto tiempo (cabeceras `Cache-Control`, `ETag`).

Interfaz uniforme
: Todo son **recursos** identificados por URL, que se manipulan a través de sus **representaciones** (normalmente JSON) con un conjunto fijo de operaciones: los métodos HTTP.

Sistema en capas
: Entre cliente y servidor puede haber proxies, balanceadores o cachés sin que el cliente lo note.

> **El modelo de madurez de Richardson.** Clasifica las APIs según lo bien que aprovechan HTTP. **Nivel 0**: un único punto de entrada al que se envía todo por `POST` (como SOAP). **Nivel 1**: una URL por recurso. **Nivel 2**: además, se usan correctamente los métodos y los códigos de estado. **Nivel 3**: las respuestas incluyen enlaces a las acciones y recursos relacionados (*HATEOAS*). La mayoría de las APIs profesionales están en el nivel 2, y es el que buscaremos.

4.6.2 Modelar los recursos

El primer paso es identificar los **recursos**: los «sustantivos» del dominio. Para la biblioteca del instituto: libros, socios y préstamos. Cada recurso tiene dos tipos de URL:

- La **colección**: `/libros`.
- Un **elemento** de la colección: `/libros/42`.

Reglas para las rutas:

- **Sustantivos, no verbos**: el verbo ya es el método HTTP. `POST /libros`, no `POST /crearLibro`; `DELETE /libros/42`, no `GET /borrarLibro?id=42`.
- **En plural** y siempre igual: `/libros` y `/libros/42`.
- **En minúsculas**, con guiones si hay varias palabras: `/salas-de-estudio`.
- **Subrecursos** para relaciones de pertenencia: `/socios/7/prestamos` (los préstamos del socio 7). No más de uno o dos niveles; para búsquedas más complejas, filtros: `/prestamos?socio=7&devuelto=false`.
- Sin extensiones ni detalles de implementación: ni `.json` ni `.php` ni nombres de tablas.

Acciones que no son CRUD

A veces una operación no encaja en crear, leer, modificar o borrar: devolver un libro, enviar un pedido, cancelar una reserva. Hay tres opciones:

1. Modificar un campo de estado: `PATCH /prestamos/15` con `{"devuelto": true}`.
2. Crear un recurso que representa la acción: `POST /prestamos/15/devolucion`. Es preferible cuando la acción tiene sus propios datos (fecha, estado del libro, quién la registró) o efectos (liberar un ejemplar, avisar al siguiente de la lista).
3. Como último recurso, un verbo como subrecurso: `POST /pedidos/8/cancelar`.

4.6.3 Métodos: seguros e idempotentes

Método	Seguro	Idempotente	Uso
`GET`	Sí	Sí	Leer. Nunca debe modificar nada.
`POST`	No	No	Crear en una colección o lanzar una acción.
`PUT`	No	Sí	Reemplazar el recurso completo.
`PATCH`	No	No*	Modificar sólo los campos enviados.
`DELETE`	No	Sí	Eliminar.

- **Seguro**: no cambia nada en el servidor. Los navegadores, buscadores y cachés asumen que pueden repetir un `GET` libremente.
- **Idempotente**: hacerlo una vez o varias tiene el mismo efecto. Borrar el libro 42 dos veces deja el mismo resultado que borrarlo una. Es fundamental en red: si una petición no obtiene respuesta (¿llegó o no llegó?), el cliente puede *reintentar* sin miedo una operación idempotente. Un `POST` repetido, en cambio, crearía dos préstamos.
- *`PATCH` es idempotente si sólo asigna valores (`{"ejemplares": 5}`), pero no si incrementa (`{"sumar": 1}`). Prefiere el primer estilo.

> **Claves de idempotencia.** Para que un `POST` se pueda reintentar con seguridad (pagos, pedidos), el cliente envía una cabecera `Idempotency-Key` con un identificador único generado por él. Si el servidor recibe dos peticiones con la misma clave, ejecuta la primera y a

> la segunda le devuelve la misma respuesta sin repetir la operación.

4.6.4 Códigos de estado

Usar el código correcto permite al cliente reaccionar sin tener que leer el mensaje de error.

Operación	Éxito	Errores habituales
`GET /libros`	200	400/422 (filtro no válido)
`GET /libros/42`	200	404
`POST /libros`	201 + cabecera `Location`	409 (ISBN repetido), 422
`PUT`, `PATCH /libros/42`	200 (o 204)	404, 409, 412, 422
`DELETE /libros/42`	204	404, 409 (tiene préstamos)
Cualquiera		401, 403, 429, 500, 503

- `400 Bad Request`: la petición está mal formada (JSON roto). `422 Unprocessable Content`: está bien formada pero los datos no son válidos (un ISBN con 12 dígitos).
- `409 Conflict`: la operación choca con el estado actual (duplicado, borrar algo que otros usan).
- `412 Precondition Failed`: falla una condición de concurrencia (ver más adelante).
- `429 Too Many Requests`: el cliente ha superado el límite de peticiones; la cabecera `Retry-After` indica cuándo reintentar.
- Los `5xx` son siempre fallos del servidor: el cliente no tiene la culpa. Nunca devuelvas `200` con un mensaje de error dentro.

4.6.5 Representaciones

- **JSON** con `Content-Type: application/json` y codificación UTF-8.
- Nombres de campos **coherentes** en toda la API: elige un estilo (`snake_case` o `camelCase`) y mantenlo.
- **Fechas** en formato ISO 8601 con zona horaria: `"2026-10-15T08:30:00Z"`. Nada de `"15/10/26"`.
- **Cantidades de dinero** como enteros en céntimos o como cadena (`"12.50"`), nunca como `float`, que introduce errores de redondeo.
- Distintos modelos para **entrada y salida**: el cliente no envía el `id` ni la fecha de creación, que genera el servidor; la respuesta nunca incluye datos internos (contraseñas, *hashes*).

4.6.6 Errores: un formato único

Todos los errores de la API deben tener la misma estructura para que los clientes los traten de forma genérica. Existe un estándar para ello, **Problem Details** (RFC 9457), con el tipo de contenido `application/problem+json`:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
```

```
{
  "type": "about:blank",
  "title": "Datos no válidos",
  "status": 422,
  "instance": "/api/v1/libros",
  "errores": [
    {"campo": "isbn", "mensaje": "el dígito de control del ISBN no es correcto"}
  ]
}
```

`title` es un resumen fijo del tipo de error, `detail` explica el caso concreto, `instance` indica la URL afectada, y se pueden añadir campos propios (`errores`). Los mensajes no deben revelar información interna (trazas, consultas SQL).

4.6.7 Colecciones: paginación, filtros y ordenación

Una colección puede tener miles de elementos: **nunca** se devuelven todos de golpe.

- **Paginación por desplazamiento**: `GET /libros?limite=20&desplazamiento=40`. Sencilla y permite saltar a cualquier página, pero es lenta con colecciones enormes y puede repetir o saltarse elementos si se insertan datos mientras se recorre.
- **Paginación por cursor**: `GET /libros?despues_de=8213`. La respuesta indica el cursor de la página siguiente. Eficiente y estable; es la que usan las grandes APIs.
- Pon siempre un **límite por defecto** y uno **máximo** (por ejemplo, 20 y 100).
- **Filtros** con el nombre del campo: `?autor=Cervantes`, `?disponible=true`; **búsqueda** de texto con `?q=`; **ordenación** con `?orden=-anio` (el signo menos indica orden descendente).
- La respuesta incluye los **metadatos** necesarios para recorrer la colección:

```
{"total": 132, "limite": 20, "desplazamiento": 40, "elementos": [ ... ]}
```

4.6.8 Versionado y evolución

- Incluye la versión desde el primer día: `/api/v1/libros`. Otras opciones son una cabecera (`Accept-Version: 1`) o el tipo de contenido, pero la URL es la más clara.
- **Cambios compatibles** (no requieren nueva versión): añadir un recurso, un campo nuevo en las respuestas o un parámetro opcional. Por eso los clientes deben *ignorar* los campos que no conocen.
- **Cambios incompatibles** (requieren `v2`): eliminar o renombrar campos, cambiar su tipo o su significado, hacer obligatorio un parámetro. La versión antigua se mantiene un tiempo, anunciando su retirada (cabeceras `Deprecation` y `Sunset`).

4.6.9 Concurrencia: actualizaciones perdidas

Dos bibliotecarios abren la ficha del mismo libro. El primero cambia el título y guarda; el segundo cambia los ejemplares y guarda enviando la ficha completa con `PUT`… y borra sin saberlo el cambio del primero. Es la misma condición de carrera del Tema 2, pero entre clientes. HTTP la resuelve con **peticiones condicionales**:

1. El servidor envía con cada recurso una cabecera `ETag`: un identificador de su versión (un número o un *hash* del contenido).
2. Al modificar, el cliente envía `If-Match:` con el `ETag` que recibió.
3. Si el recurso ha cambiado desde entonces, el servidor responde `412 Precondition Failed` y no aplica el cambio; el cliente vuelve a leer el recurso y decide.

4.6.10 Seguridad y protección del servicio

- Siempre **HTTPS** y autenticación en cada petición (Tema 5).
- Nunca datos sensibles en la URL (`?password=...`): las URL se guardan en registros, historiales y proxies.
- **Validar todo** lo que llega y limitar tamaños (longitud de textos, tamaño del cuerpo, tamaño de página).
- **Limitar la frecuencia** de peticiones por cliente (`429`).
- Configurar **CORS** (*Cross-Origin Resource Sharing*): por seguridad, un navegador sólo deja que una página web llame a una API de otro dominio si la API lo autoriza expresamente.

4.6.11 Documentación: OpenAPI

OpenAPI es el estándar para describir una API REST en un fichero JSON o YAML: rutas, parámetros, modelos de datos, códigos de respuesta y seguridad. A partir de él se generan documentación interactiva, clientes en cualquier lenguaje y pruebas. Hay dos enfoques: *diseño primero* (se escribe la especificación y luego el código) o *código primero* (la especificación se genera a partir del código, como hace FastAPI).

4.6.12 Ejemplo: diseño de la API de la biblioteca

Método	Ruta (prefijo `/api/v1`)	Descripción	Éxito
`GET`	`/libros`	Listar; filtros `autor`, `q`, `orden`; paginado	200
`POST`	`/libros`	Añadir un libro	201
`GET`	`/libros/{id}`	Ficha de un libro	200
`PATCH`	`/libros/{id}`	Modificar datos	200
`DELETE`	`/libros/{id}`	Dar de baja (409 si está prestado)	204
`GET`	`/socios`	Listar socios	200
`POST`	`/socios`	Dar de alta un socio	201
`GET`	`/socios/{id}/prestamos`	Préstamos de un socio	200
`POST`	`/prestamos`	Prestar un libro (409 si no quedan ejemplares)	201
`GET`	`/prestamos`	Listar; filtros `socio`, `devuelto`, `vencido`	200
`POST`	`/prestamos/{id}/devolucion`	Registrar la devolución	201
`GET`	`/salud`	Estado del servicio (sin versión)	200

En la sección 4.8 implementaremos la parte de los libros; socios y préstamos quedan como ejercicio.

Resumen del tema. Lista de comprobación del diseño de una API REST

- Recursos con sustantivos en plural; el método HTTP es el verbo.
- `GET` nunca modifica; operaciones idempotentes donde sea posible.

- Códigos de estado precisos; `201` + `Location` al crear.
- Modelos de entrada y salida separados; JSON coherente; fechas ISO 8601.
- Errores con un formato único (RFC 9457) y sin detalles internos.
- Colecciones siempre paginadas, con filtros y ordenación.
- Versión en la URL desde el principio; cambios compatibles.
- Control de concurrencia (`ETag`/`If-Match`) donde haga falta.
- HTTPS, autenticación, validación, límites y CORS.
- Documentación OpenAPI siempre actualizada.

4.7 FastAPI: primeros pasos

FastAPI es uno de los *frameworks* más usados actualmente para crear servicios web en Python. Usa las anotaciones de tipo para validar los datos automáticamente y genera la documentación de la API sin escribir nada extra.

```
(.venv) $ pip install "fastapi[standard]"
```

```python
# main.py: API REST de tareas
from datetime import datetime
from threading import Lock

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="API de tareas", version="1.0")

class TareaEntrada(BaseModel):
    """Datos que envía el cliente para crear o reemplazar una tarea."""
    titulo: str = Field(min_length=1, max_length=100)
    prioridad: int = Field(default=3, ge=1, le=5)
    hecha: bool = False

class Tarea(TareaEntrada):
    """Tarea tal como la devuelve el servicio."""
    id: int
    creada: datetime

tareas: dict[int, Tarea] = {}       # "base de datos" en memoria
siguiente_id = 1
cerrojo = Lock()

@app.get("/tareas")
def listar(hecha: bool | None = None) -> list[Tarea]:
    resultado = list(tareas.values())
    if hecha is not None:
        resultado = [t for t in resultado if t.hecha == hecha]
    return resultado

@app.get("/tareas/{id_tarea}")
def obtener(id_tarea: int) -> Tarea:
```

```
    if id_tarea not in tareas:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Tarea no encontrada")
    return tareas[id_tarea]

@app.post("/tareas", status_code=status.HTTP_201_CREATED)
def crear(entrada: TareaEntrada) -> Tarea:
    global siguiente_id
    with cerrojo:
        tarea = Tarea(id=siguiente_id, creada=datetime.now(),
                      **entrada.model_dump())
        tareas[tarea.id] = tarea
        siguiente_id += 1
    return tarea

@app.put("/tareas/{id_tarea}")
def reemplazar(id_tarea: int, entrada: TareaEntrada) -> Tarea:
    with cerrojo:
        if id_tarea not in tareas:
            raise HTTPException(status.HTTP_404_NOT_FOUND, "Tarea no encontrada")
        tareas[id_tarea] = Tarea(id=id_tarea, creada=tareas[id_tarea].creada,
                                 **entrada.model_dump())
        return tareas[id_tarea]

@app.delete("/tareas/{id_tarea}", status_code=status.HTTP_204_NO_CONTENT)
def borrar(id_tarea: int) -> None:
    with cerrojo:
        if tareas.pop(id_tarea, None) is None:
            raise HTTPException(status.HTTP_404_NOT_FOUND, "Tarea no encontrada")

@app.get("/salud")
def salud() -> dict:
    return {"estado": "ok", "tareas": len(tareas)}
```

```
(.venv) $ fastapi dev main.py
   Serving at: http://127.0.0.1:8000
   API docs:   http://127.0.0.1:8000/docs
```

Algunas cosas que FastAPI hace por nosotros:

- **Enrutado**: el decorador indica método y ruta; las partes `{id_tarea}` de la ruta se pasan como parámetros, y los parámetros que no están en la ruta se leen de la URL (`/tareas?hecha=true`).
- **Validación**: si el cliente envía `"prioridad": 9` o `/tareas/abc`, responde automáticamente un `422` explicando el error. Las clases `BaseModel` (de la librería *Pydantic*) describen la forma de los datos.
- **Serialización**: lo que devuelve la función se convierte en JSON.
- **Documentación**: en `/docs` hay una página interactiva (OpenAPI/Swagger) desde la que se puede probar cada operación. Es también un *cliente* para verificar el servicio.

4.7.1 Concurrencia en FastAPI

FastAPI se ejecuta sobre **Uvicorn**, un servidor asíncrono (Tema 3):

- Las funciones definidas con `def` normal se ejecutan en un **conjunto de hilos**, de modo

que varias peticiones se atienden a la vez. Por eso nuestro ejemplo protege el estado con un cerrojo.

- Las funciones definidas con `async def` se ejecutan en el bucle de eventos; dentro de ellas sólo deben usarse operaciones asíncronas (`await`).
- Para usar varios núcleos se lanzan varios **procesos** trabajadores: `fastapi run main.py --workers 4`. Cada proceso tiene su propia memoria, así que el estado ya no puede guardarse en variables: hay que usar una base de datos. Vuelven a aparecer los conceptos del Tema 1.

4.7.2 Probar el servicio automáticamente

```python
# test_main.py (ejecutar con pytest)
from fastapi.testclient import TestClient
from main import app

cliente = TestClient(app)

def test_crear_y_obtener():
    r = cliente.post("/tareas", json={"titulo": "Probar la API"})
    assert r.status_code == 201
    id_tarea = r.json()["id"]
    r = cliente.get(f"/tareas/{id_tarea}")
    assert r.status_code == 200
    assert r.json()["titulo"] == "Probar la API"

def test_validacion():
    r = cliente.post("/tareas", json={"titulo": "", "prioridad": 9})
    assert r.status_code == 422

def test_no_existe():
    assert cliente.get("/tareas/9999").status_code == 404
```

4.8 Un proyecto completo con FastAPI

El ejemplo de las tareas cabe en un fichero, pero guarda los datos en memoria y mezcla todo en el mismo sitio. En esta sección construimos la parte de **libros** de la API de la biblioteca aplicando el diseño de la sección 4.6: base de datos, modelos de entrada y salida, paginación, errores RFC 9457, versionado, CORS, registro de peticiones y pruebas. El código completo está en la carpeta `codigo/tema4/biblioteca` que acompaña a estos apuntes.

4.8.1 Estructura del proyecto

```
biblioteca/
├── app/
│   ├── __init__.py
│   ├── main.py            # crea la aplicación y la configura
│   ├── database.py        # conexión a la base de datos
│   ├── models.py          # modelos de datos (tabla, entrada, salida)
│   ├── errores.py         # formato de los errores
│   └── routers/
```

```
│       ├── __init__.py
│       └── libros.py      # las rutas de /libros
├── tests/
│   ├── __init__.py
│   └── test_libros.py
└── requirements.txt
```

```
$ uv venv --python 3.15t && uv pip install -r requirements.txt
$ uv run fastapi dev app/main.py          # desarrollo, con recarga automática
$ uv run pytest                           # pruebas
```

Separar en módulos permite que cada fichero tenga una única responsabilidad y que la aplicación crezca añadiendo un *router* por recurso (`socios.py`, `prestamos.py`) sin tocar el resto.

4.8.2 Base de datos y dependencias

Usaremos **SQLModel**, una librería del mismo autor que FastAPI que combina Pydantic (validación) y SQLAlchemy (acceso a bases de datos): una misma clase sirve para definir una tabla y para validar los datos. Con SQLite no hay que instalar ningún servidor de base de datos.

app/database.py

```python
from collections.abc import Iterator
from typing import Annotated

from fastapi import Depends
from sqlmodel import Session, SQLModel, create_engine

URL_BD = "sqlite:///biblioteca.db"
# check_same_thread=False: FastAPI atiende las peticiones en varios hilos
engine = create_engine(URL_BD, connect_args={"check_same_thread": False})

def crear_tablas() -> None:
    SQLModel.metadata.create_all(engine)

def obtener_sesion() -> Iterator[Session]:
    """Dependencia: abre una sesión por petición y la cierra al terminar."""
    with Session(engine) as sesion:
        yield sesion

SesionBD = Annotated[Session, Depends(obtener_sesion)]
```

La función `obtener_sesion` es una **dependencia**. Es uno de los conceptos centrales de FastAPI (*inyección de dependencias*): una ruta declara lo que necesita y FastAPI se lo proporciona en cada petición.

- Si la dependencia usa `yield`, lo que hay antes se ejecuta antes de la ruta y lo que hay después (aquí, cerrar la sesión gracias al `with`), al terminar la petición, incluso si hay un error.
- `SesionBD = Annotated[Session, Depends(obtener_sesion)]` es un alias: basta con escribir `sesion: SesionBD` en una ruta para recibir una sesión.
- En las pruebas podremos **sustituir** la dependencia por otra que use una base de datos de prueba, sin tocar el código de las rutas.

- Las dependencias sirven para mucho más: obtener el usuario autenticado (lo veremos en el Tema 5), leer parámetros comunes como la paginación, comprobar permisos...

4.8.3 Modelos: uno para cada uso

Siguiendo las buenas prácticas de diseño, cada representación del recurso tiene su propio modelo, y todos heredan los campos comunes de `LibroBase`:

app/models.py

```python
from datetime import datetime, timezone

from pydantic import BaseModel, field_validator
from sqlmodel import Field, SQLModel

class LibroBase(SQLModel):
    """Campos comunes a todas las representaciones de un libro."""
    titulo: str = Field(min_length=1, max_length=200)
    autor: str = Field(min_length=1, max_length=100, index=True)
    isbn: str = Field(unique=True, description="ISBN-13")
    anio: int | None = Field(default=None, ge=1450, le=2100)
    ejemplares: int = Field(default=1, ge=0)

    @field_validator("isbn")
    @classmethod
    def comprobar_isbn(cls, valor: str) -> str:
        """Valida un ISBN-13: 13 dígitos y dígito de control correcto."""
        valor = valor.replace("-", "")
        if len(valor) != 13 or not valor.isdigit():
            raise ValueError("el ISBN debe tener 13 dígitos")
        suma = sum(int(c) * (3 if i % 2 else 1) for i, c in enumerate(valor[:12]))
        if (10 - suma % 10) % 10 != int(valor[12]):
            raise ValueError("el dígito de control del ISBN no es correcto")
        return valor

class Libro(LibroBase, table=True):
    """Tabla de la base de datos."""
    id: int | None = Field(default=None, primary_key=True)
    creado: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

class LibroCrear(LibroBase):
    """Lo que envía el cliente al crear (sin id ni fecha)."""

class LibroLeer(LibroBase):
    """Lo que devuelve la API."""
    id: int
    creado: datetime

class LibroActualizar(SQLModel):
    """PATCH: todos los campos opcionales; el ISBN no se puede cambiar."""
    titulo: str | None = Field(default=None, min_length=1, max_length=200)
    autor: str | None = Field(default=None, min_length=1, max_length=100)
    anio: int | None = Field(default=None, ge=1450, le=2100)
    ejemplares: int | None = Field(default=None, ge=0)

class Pagina[T](BaseModel):
    """Respuesta paginada de cualquier colección."""
    total: int
```

```
    limite: int
    desplazamiento: int
    elementos: list[T]
```

`LibroBase`	Campos y reglas de validación comunes.
`Libro`	La **tabla** (`table=True`): añade la clave primaria y la fecha de creación, que genera el servidor.
`LibroCrear`	Lo que el cliente envía en el `POST`. No puede fijar el `id` ni la fecha.
`LibroLeer`	Lo que la API devuelve. Aquí se elige qué campos se exponen; si la tabla tuviera datos internos, no se incluirían.
`LibroActualizar`	Para el `PATCH`: todos los campos opcionales; el ISBN no se puede modificar.
`Pagina[T]`	Modelo **genérico** para cualquier colección paginada: `Pagina[LibroLeer]`, `Pagina[SocioLeer]`...

La validación tiene dos niveles: las restricciones declarativas de `Field` (`min_length`, `ge`, `le`...) y los validadores propios con `@field_validator`, que pueden normalizar el valor (quitar guiones del ISBN) y comprobar reglas complejas (el dígito de control). Si la validación falla, FastAPI responde `422` automáticamente.

4.8.4 Errores con el formato RFC 9457

Por defecto, FastAPI devuelve los errores como `{"detail": ...}`. Registrando nuestros propios **manejadores de excepciones**, todos los errores de la API (los que lanzamos nosotros, los de validación y hasta los de rutas inexistentes) tendrán el formato estándar:

app/errores.py

```python
from http import HTTPStatus

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException

def problema(estado: int, titulo: str, detalle: str | None = None,
             **extra) -> JSONResponse:
    """Construye una respuesta de error con el formato de la RFC 9457."""
    cuerpo = {"type": "about:blank", "title": titulo, "status": estado}
    if detalle:
        cuerpo["detail"] = detalle
    cuerpo.update(extra)
    return JSONResponse(cuerpo, status_code=estado,
                        media_type="application/problem+json")

def registrar_manejadores(app: FastAPI) -> None:
    @app.exception_handler(HTTPException)
    async def error_http(request: Request, exc: HTTPException):
        respuesta = problema(exc.status_code, HTTPStatus(exc.status_code).phrase,
                             str(exc.detail), instance=request.url.path)
        respuesta.headers.update(exc.headers or {})
        return respuesta

    @app.exception_handler(RequestValidationError)
    async def error_validacion(request: Request, exc: RequestValidationError):
```

```
        errores = [{"campo": ".".join(str(p) for p in e["loc"][1:]),
                    "mensaje": e["msg"]} for e in exc.errors()]
        return problema(422, "Datos no válidos", instance=request.url.path,
                        errores=errores)
```

4.8.5 Las rutas: `APIRouter`

app/routers/libros.py

```
from dataclasses import dataclass
from typing import Annotated, Literal

from fastapi import APIRouter, Depends, HTTPException, Query, Request, Response, status
from sqlalchemy.exc import IntegrityError
from sqlmodel import col, func, select

from ..database import SesionBD
from ..models import Libro, LibroActualizar, LibroCrear, LibroLeer, Pagina

router = APIRouter(prefix="/libros", tags=["libros"])

@dataclass
class Paginacion:
    limite: int
    desplazamiento: int

def paginacion(
    limite: Annotated[int, Query(ge=1, le=100, description="Elementos por página")] = 20,
    ↪
    desplazamiento: Annotated[int, Query(ge=0, description="Elementos a saltar")] = 0,
) -> Paginacion:
    return Paginacion(limite, desplazamiento)

Orden = Literal["titulo", "-titulo", "anio", "-anio", "creado", "-creado"]

def buscar_o_404(sesion: SesionBD, id_libro: int) -> Libro:
    libro = sesion.get(Libro, id_libro)
    if libro is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, f"No existe el libro {id_libro}")
    return libro

@router.get("", response_model=Pagina[LibroLeer], summary="Listar libros")
def listar_libros(
    sesion: SesionBD,
    pag: Annotated[Paginacion, Depends(paginacion)],
    autor: str | None = None,
    q: Annotated[str | None, Query(max_length=50, description="Texto en el título")] =
    ↪ None,
    orden: Orden = "titulo",
):
    """Devuelve los libros, filtrados, ordenados y paginados."""
    consulta = select(Libro)
    if autor:
        consulta = consulta.where(Libro.autor == autor)
    if q:
        consulta = consulta.where(col(Libro.titulo).icontains(q))
    total = sesion.exec(select(func.count()).select_from(consulta.subquery())).one()
```

```
    campo = col(getattr(Libro, orden.removeprefix("-")))
    consulta = consulta.order_by(campo.desc() if orden.startswith("-") else campo)
    libros = sesion.exec(consulta.offset(pag.desplazamiento).limit(pag.limite)).all()
    return {"total": total, "limite": pag.limite,
            "desplazamiento": pag.desplazamiento, "elementos": libros}

@router.get("/{id_libro}", response_model=LibroLeer, summary="Obtener un libro",
            responses={404: {"description": "El libro no existe"}})
def obtener_libro(sesion: SesionBD, id_libro: int):
    return buscar_o_404(sesion, id_libro)

@router.post("", response_model=LibroLeer, status_code=status.HTTP_201_CREATED,
             summary="Añadir un libro",
             responses={409: {"description": "Ya existe un libro con ese ISBN"}})
def crear_libro(sesion: SesionBD, datos: LibroCrear, request: Request,
                response: Response):
    libro = Libro.model_validate(datos)
    sesion.add(libro)
    try:
        sesion.commit()
    except IntegrityError:
        sesion.rollback()
        raise HTTPException(status.HTTP_409_CONFLICT,
                            f"Ya existe un libro con ISBN {datos.isbn}")
    sesion.refresh(libro)
    response.headers["Location"] = str(request.url_for("obtener_libro",
                                                       id_libro=libro.id))
    return libro

@router.patch("/{id_libro}", response_model=LibroLeer, summary="Modificar un libro")
def modificar_libro(sesion: SesionBD, id_libro: int, cambios: LibroActualizar):
    libro = buscar_o_404(sesion, id_libro)
    libro.sqlmodel_update(cambios.model_dump(exclude_unset=True))
    sesion.add(libro)
    sesion.commit()
    sesion.refresh(libro)
    return libro

@router.delete("/{id_libro}", status_code=status.HTTP_204_NO_CONTENT,
               summary="Eliminar un libro")
def eliminar_libro(sesion: SesionBD, id_libro: int) -> None:
    sesion.delete(buscar_o_404(sesion, id_libro))
    sesion.commit()
```

Aspectos a destacar:

- **`APIRouter`** agrupa las rutas de un recurso con un prefijo común (`/libros`) y una etiqueta (`tags`) que las agrupa en la documentación.
- **Origen de los parámetros**. FastAPI deduce de dónde sale cada parámetro: si su nombre aparece en la ruta (`{id_libro}`), de la ruta; si es un modelo Pydantic (`datos: LibroCrear`), del cuerpo JSON; si es un tipo simple, de la cadena de consulta (`?autor=`); `Request`, `Response` y las dependencias se inyectan. También existen `Header()` y `Cookie()`.
- **Validación de parámetros** con `Annotated` y `Query`: `limite` debe estar entre 1 y 100; `orden` sólo admite los valores del `Literal`. Cualquier otro valor produce un `422` sin escribir una línea de comprobación.

- **Paginación como dependencia**: `paginacion()` se reutiliza en cualquier listado.
- **`response_model`** filtra y valida lo que se devuelve: aunque la función devuelva objetos `Libro`, sólo saldrán los campos de `LibroLeer`.
- **Códigos de estado**: `201` y cabecera `Location` al crear (construida con `url_for` a partir del nombre de la función, sin escribir la URL a mano); `204` al borrar; `404` y `409` con `HTTPException`.
- **Conflictos**: la restricción `UNIQUE` de la base de datos garantiza que no haya dos libros con el mismo ISBN *aunque dos clientes lo intenten a la vez*, algo que comprobar antes con un `SELECT` no garantizaría (sería otra vez el patrón «comprobar y luego actuar» del Tema 2).
- **PATCH**: `model_dump(exclude_unset=True)` devuelve sólo los campos que el cliente envió, de modo que el resto no cambia.
- **Documentación**: `summary`, el *docstring* y `responses` aparecen en `/docs`.

4.8.6 La aplicación

app/main.py

```
import logging
import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware

from .database import crear_tablas
from .errores import registrar_manejadores
from .routers import libros

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(name)s: %(message)s")
log = logging.getLogger("biblioteca")

@asynccontextmanager
async def ciclo_de_vida(app: FastAPI):
    crear_tablas()                          # se ejecuta al arrancar
    log.info("Base de datos preparada")
    yield
    log.info("Servicio detenido")           # se ejecuta al parar

app = FastAPI(
    title="API de la biblioteca del instituto",
    version="1.0.0",
    description="Gestión del catálogo de libros. Errores en formato RFC 9457.",
    lifespan=ciclo_de_vida,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],   # la web que puede usar la API
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["*"],
)

@app.middleware("http")
```

```
async def registrar_peticiones(request: Request, call_next):
    inicio = time.perf_counter()
    respuesta = await call_next(request)
    ms = (time.perf_counter() - inicio) * 1000
    respuesta.headers["X-Tiempo-Proceso"] = f"{ms:.1f} ms"
    log.info("%s %s -> %d (%.1f ms)", request.method, request.url.path,
             respuesta.status_code, ms)
    return respuesta

registrar_manejadores(app)
app.include_router(libros.router, prefix="/api/v1")

@app.get("/salud", tags=["sistema"], summary="Estado del servicio")
def salud() -> dict[str, str]:
    return {"estado": "ok"}
```

- **Ciclo de vida** (`lifespan`): el código antes del `yield` se ejecuta una vez al arrancar el servicio (crear tablas, abrir conexiones, cargar configuración) y el de después, al detenerlo.
- **Middleware**: código que se ejecuta en *todas* las peticiones, antes y después de la ruta. Aquí mide el tiempo, lo añade como cabecera y registra cada petición con `logging`. FastAPI incluye middlewares para CORS, compresión (`GZipMiddleware`) o redirección a HTTPS.
- **CORS**: sólo la web indicada en `allow_origins` podrá llamar a la API desde un navegador.
- **Versionado**: el router se monta en `/api/v1`. Una futura `v2` sería otro conjunto de routers montado en `/api/v2`, conviviendo con el primero.

```
$ curl -i -X POST localhost:8000/api/v1/libros -H "Content-Type: application/json" \
       -d '{"titulo": "El Quijote", "autor": "Miguel de Cervantes",
            "isbn": "978-84-376-0494-7", "anio": 1605}'
HTTP/1.1 201 Created
location: http://localhost:8000/api/v1/libros/1
x-tiempo-proceso: 24.4 ms
{"titulo":"El Quijote","autor":"Miguel de Cervantes","isbn":"9788437604947",
 "anio":1605,"ejemplares":1,"id":1,"creado":"2026-10-15T08:06:19.887660Z"}

$ curl -i localhost:8000/api/v1/libros/99
HTTP/1.1 404 Not Found
content-type: application/problem+json
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"No existe el libro 99","instance":"/api/v1/libros/99"}
```

4.8.7 Pruebas

Las pruebas sustituyen la dependencia de la base de datos por una base de datos SQLite **en memoria**, nueva para cada prueba, de modo que son rápidas, independientes entre sí y no tocan los datos reales:

tests/test_libros.py

```
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool
```

```python
from app.database import obtener_sesion
from app.main import app

def isbn(n: int) -> str:
    """Genera un ISBN-13 válido a partir de un número."""
    base = f"978{n:09d}"
    suma = sum(int(c) * (3 if i % 2 else 1) for i, c in enumerate(base))
    return base + str((10 - suma % 10) % 10)

LIBRO = {"titulo": "Cien años de soledad", "autor": "Gabriel García Márquez",
         "isbn": "9788497592208", "anio": 1967, "ejemplares": 3}

@pytest.fixture
def cliente():
    # Base de datos en memoria, nueva para cada prueba
    engine = create_engine("sqlite://", poolclass=StaticPool,
                           connect_args={"check_same_thread": False})
    SQLModel.metadata.create_all(engine)

    def sesion_de_prueba():
        with Session(engine) as sesion:
            yield sesion

    app.dependency_overrides[obtener_sesion] = sesion_de_prueba
    yield TestClient(app)
    app.dependency_overrides.clear()

def test_crear_y_obtener(cliente):
    r = cliente.post("/api/v1/libros", json=LIBRO)
    assert r.status_code == 201
    libro = r.json()
    assert r.headers["Location"].endswith(f"/api/v1/libros/{libro['id']}")
    assert cliente.get(f"/api/v1/libros/{libro['id']}").json()["isbn"] == LIBRO["isbn"]

def test_isbn_duplicado(cliente):
    cliente.post("/api/v1/libros", json=LIBRO)
    r = cliente.post("/api/v1/libros", json=LIBRO)
    assert r.status_code == 409
    assert r.headers["Content-Type"] == "application/problem+json"

def test_datos_no_validos(cliente):
    r = cliente.post("/api/v1/libros", json={**LIBRO, "isbn": "123", "anio": 3000})
    assert r.status_code == 422
    campos = {e["campo"] for e in r.json()["errores"]}
    assert campos == {"isbn", "anio"}

def test_paginacion_y_orden(cliente):
    for i in range(5):
        cliente.post("/api/v1/libros",
                     json={**LIBRO, "titulo": f"Libro {i}", "isbn": isbn(i)})
    r = cliente.get("/api/v1/libros", params={"limite": 2, "desplazamiento": 1,
                                              "orden": "-titulo"})
    pagina = r.json()
    assert pagina["total"] == 5
    assert [l["titulo"] for l in pagina["elementos"]] == ["Libro 3", "Libro 2"]

def test_modificar_parcialmente(cliente):
```

```
    id_libro = cliente.post("/api/v1/libros", json=LIBRO).json()["id"]
    r = cliente.patch(f"/api/v1/libros/{id_libro}", json={"ejemplares": 5})
    assert r.status_code == 200
    assert r.json()["ejemplares"] == 5
    assert r.json()["titulo"] == LIBRO["titulo"]       # lo demás no cambia

def test_eliminar(cliente):
    id_libro = cliente.post("/api/v1/libros", json=LIBRO).json()["id"]
    assert cliente.delete(f"/api/v1/libros/{id_libro}").status_code == 204
    r = cliente.get(f"/api/v1/libros/{id_libro}")
    assert r.status_code == 404
    assert r.json()["title"] == "Not Found"
```

```
$ uv run pytest -v
tests/test_libros.py::test_crear_y_obtener PASSED
tests/test_libros.py::test_isbn_duplicado PASSED
tests/test_libros.py::test_datos_no_validos PASSED
tests/test_libros.py::test_paginacion_y_orden PASSED
tests/test_libros.py::test_modificar_parcialmente PASSED
tests/test_libros.py::test_eliminar PASSED
```

Pruebas de los casos de error. Fíjate en que la mitad de las pruebas comprueban *errores*: duplicados, datos no válidos, recursos inexistentes. Un cliente se apoya en el contrato de la API tanto para los éxitos como para los fallos.

4.8.8 Puesta en producción

- `fastapi run app/main.py --workers 4` lanza el servicio sin recarga automática y con 4 procesos. Con varios procesos, el estado debe estar en la base de datos, nunca en variables.
- SQLite es suficiente para un servicio pequeño; con mucha concurrencia de escritura se cambia a PostgreSQL modificando sólo `URL_BD`.
- Delante se coloca un proxy inverso con HTTPS (Tema 5).

4.9 Otro protocolo estándar: el correo electrónico

Enviar correos (avisos, recuperación de contraseña, informes) es una función habitual de los servicios. Se hace con SMTP, usando `smtplib` y `email`:

```
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "servicio@miinstituto.local"
msg["To"] = "alumno@miinstituto.local"
msg["Subject"] = "Tarea completada"
msg.set_content("La tarea 'Estudiar sockets' se ha marcado como hecha.")

with smtplib.SMTP("localhost", 1025, timeout=10) as smtp:
    # Con un servidor real: smtp.starttls(); smtp.login(usuario, clave)
    smtp.send_message(msg)
```

Servidor de correo para pruebas. No uses tu cuenta real para las prácticas. Herramientas como **Mailpit** (`docker run -p 8025:8025 -p 1025:1025 axllent/mailpit`) o `aiosmtpd` (`pip install aiosmtpd` y luego `python -m aiosmtpd -n -l localhost:1025`) ofrecen un servidor SMTP local que recibe los correos sin enviarlos; Mailpit, además, los muestra en `http://localhost:8025`.

WebSockets. HTTP es petición–respuesta: el servidor no puede enviar nada al cliente si este no pregunta. Para notificaciones en tiempo real (chats, cotizaciones, juegos web) existe el protocolo WebSocket, que, tras una petición HTTP inicial, convierte la conexión en un canal bidireccional permanente. FastAPI lo soporta con el decorador `@app.websocket("/ws")`.

4.10 Conexiones, eficiencia y disponibilidad

4.10.1 Establecimiento y finalización de conexiones

- **Reutilización** (*keep-alive*): abrir una conexión TCP (y más aún una TLS) es costoso. HTTP/1.1 mantiene la conexión abierta para varias peticiones; en el cliente se aprovecha con `requests.Session`.
- **Timeouts**: tanto el cliente como el servidor deben limitar el tiempo de espera para que una conexión colgada no bloquee recursos indefinidamente.
- **Cierre ordenado** (*graceful shutdown*): al detener el servicio, se deja de aceptar conexiones nuevas, se terminan las peticiones en curso y se liberan recursos. Uvicorn lo hace al recibir `SIGTERM` o `Ctrl+C`.

4.10.2 Eficiencia

- Atender peticiones concurrentemente (hilos, `asyncio`, varios procesos trabajadores).
- No bloquear el servidor con tareas largas: enviar un correo o generar un informe puede hacerse en segundo plano y responder enseguida.
- Paginar los listados largos (`?pagina=2&tamano=50`).
- Usar cachés para datos que se consultan mucho y cambian poco.

4.10.3 Disponibilidad y monitorización

La **disponibilidad** es el porcentaje de tiempo que el servicio funciona correctamente. Un 99,9 % permite unas 8,8 horas de caída al año; un 99,99 %, menos de una hora. Para conseguirla y verificarla:

- **Punto de salud** (*health check*): una ruta como `/salud` que indica si el servicio funciona (y si puede acceder a sus dependencias, como la base de datos).
- **Monitorización externa**: un programa que consulta periódicamente el servicio y avisa si falla. Existen herramientas dedicadas como *Uptime Kuma*, *Prometheus* + *Grafana* o servicios en la nube.

- **Reinicio automático**: ejecutar el servicio como servicio del sistema (`systemd` con `Restart=always`) o en contenedores con política de reinicio.
- **Redundancia**: varias instancias detrás de un balanceador de carga, de modo que si una cae las demás siguen atendiendo.
- **Registro** (*logs*) de cada petición: Uvicorn lo hace por defecto (método, ruta, código y tiempo).
- **Pruebas de carga** para conocer los límites: *Locust* (en Python), `hey` o `ab`.

```python
# Monitor de disponibilidad
import logging
import time
import requests

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
URL, INTERVALO = "http://127.0.0.1:8000/salud", 5

comprobaciones = fallos = 0
while True:
    comprobaciones += 1
    try:
        inicio = time.perf_counter()
        r = requests.get(URL, timeout=2)
        ms = (time.perf_counter() - inicio) * 1000
        r.raise_for_status()
        logging.info("OK %d en %.0f ms", r.status_code, ms)
    except requests.RequestException as e:
        fallos += 1
        logging.error("CAÍDA: %s", e)
    disponibilidad = 100 * (comprobaciones - fallos) / comprobaciones
    logging.info("Disponibilidad: %.2f %%", disponibilidad)
    time.sleep(INTERVALO)
```

4.11 Documentación y depuración de servicios

- La documentación de una API es su **especificación OpenAPI**: FastAPI la genera en `/openapi.json` y la muestra en `/docs`. Añade `summary` y `description` a las rutas y docstrings a las funciones para enriquecerla.
- Usa los **códigos de estado** correctos: facilitan enormemente la depuración en el lado del cliente.
- Ejecuta en modo desarrollo (`fastapi dev`), que recarga el código al guardar y muestra las trazas de error.
- Escribe **pruebas automáticas** con `TestClient` y `pytest`; ejecútalas antes de cada cambio.
- Para ver exactamente qué se envía, `curl -v` muestra la petición y la respuesta completas.

Resumen del tema.

- Los protocolos estándar aportan interoperabilidad, herramientas y seguridad. HTTP es el más importante.

- HTTP es texto, petición–respuesta y sin estado: método, ruta, cabeceras y cuerpo; la respuesta lleva un código de estado.
- Clientes: navegador, `curl`, Bruno/Postman, `requests`.
- Servidores: `http.server` (aprendizaje) y FastAPI (real). Un servicio REST expone recursos manipulados con los métodos HTTP.
- Varias peticiones simultáneas exigen proteger el estado compartido.
- La disponibilidad se verifica con puntos de salud, monitorización y pruebas de carga.

4.12 Ejercicios

Ejercicio 4.1 — HTTP a mano

Dificultad: ★

Conéctate con `nc` o con un socket de Python a `example.com` en el puerto 80 y haz una petición `GET` y otra `HEAD`. Después, programa un servidor TCP con sockets (sin `http.server`) que responda a cualquier petición con una página HTML que muestre la hora actual, y ábrelo con el navegador.

Ejercicio 4.2 — Cliente de una API pública

Dificultad: ★

Usando `requests` y la API gratuita de Open-Meteo (`https://open-meteo.com`), escribe un programa que reciba el nombre de una ciudad de Castilla y León y muestre la previsión de temperatura de los próximos 3 días. Trata los errores de red y de respuesta.

Ejercicio 4.3 — Peticiones concurrentes

Dificultad: ★★

Haz 50 peticiones a `/salud` del servicio de tareas primero de forma secuencial y después con un `ThreadPoolExecutor` de 10 hilos, usando una `Session` por hilo. Compara los tiempos. Repite añadiendo un `time.sleep(0.2)` en la ruta del servidor y explica qué ocurre.

Ejercicio 4.4 — Completar la API

Dificultad: ★★

Añade a la API de tareas: la operación `PATCH /tareas/{id}` (modificar sólo los campos enviados), paginación en el listado, búsqueda por texto en el título (`?q=`) y ordenación por prioridad. Escribe las pruebas con `pytest` de cada operación.

Ejercicio 4.5 — Persistencia

Dificultad: ★★

Sustituye el diccionario en memoria de la API por una base de datos SQLite (`sqlite3`). Comprueba que los datos sobreviven a un reinicio del servicio y que funciona al lanzarlo con `--workers 2`.

Ejercicio 4.6 — Revisión de diseño

Dificultad: ★

Esta API tiene, al menos, diez errores de diseño. Encuéntralos, explica cada uno y propón el diseño correcto: `GET /getAlumnos`, `POST /alumno/borrar?id=3`, `GET /alumnos/3/aprobar`, `PUT /Alumnos/3` con sólo el campo `nota`, `GET /alumnos` que devuelve los 40 000 alumnos, una respuesta `200` con `{"error": "no existe"}`, fechas como `"3/10/26"`, notas como `7.499999` y una respuesta de alumno que incluye el *hash* de su contraseña.

Ejercicio 4.7 — Socios y préstamos

Dificultad: ★★★

Completa el proyecto de la biblioteca siguiendo el diseño de la tabla de la sección 4.6: routers de `socios` y `prestamos`, con sus modelos de entrada y salida. Un préstamo resta un ejemplar disponible y la devolución lo suma; no se puede prestar un libro sin ejemplares (`409`) ni dar de baja un libro con préstamos pendientes (`409`). El listado de préstamos debe admitir los filtros `socio`, `devuelto` y `vencido`. Escribe las pruebas, incluida una que lance 20 peticiones de préstamo simultáneas con hilos sobre un libro con 5 ejemplares y compruebe que sólo se conceden 5.

Ejercicio 4.8 — Control de concurrencia con ETag

Dificultad: ★★★

Añade a la API de la biblioteca el control de actualizaciones perdidas: añade a la tabla un campo `version` que se incrementa en cada modificación, devuélvelo en la cabecera `ETag` en `GET` y `PATCH`, y haz que `PATCH` exija la cabecera `If-Match` (`428` si falta, `412` si no coincide). Demuéstralo con una prueba.

Ejercicio 4.9 — Notificaciones por correo

Dificultad: ★★★

Añade a la API la posibilidad de suscribirse a una tarea con un correo electrónico. Cuando la tarea se marque como hecha, se enviará un correo a los suscriptores **sin retrasar la respuesta** de la API (investiga `BackgroundTasks` de FastAPI). Pruébalo con Mailpit.

Proyecto: Servicio de reservas de espacios del instituto

Desarrolla un servicio web para reservar espacios del centro (aulas de informática, salón de actos, carros de portátiles) y un cliente para usarlo.
Requisitos del servicio (FastAPI):

1. Recursos: `/espacios`, `/reservas` y `/salud`, con operaciones CRUD siguiendo los principios REST y los códigos de estado adecuados.
2. Una reserva tiene espacio, fecha, franja horaria, profesor y motivo. El servicio debe rechazar (`409 Conflict`) las reservas que se solapen con otra existente, **también cuando dos clientes intentan reservar a la vez la misma franja** (piensa en la sincronización o en las restricciones de la base de datos).
3. Persistencia en SQLite.
4. Documentación completa en `/docs` y pruebas automáticas con `pytest` (incluida una

prueba de reservas simultáneas con hilos).

Requisitos del cliente:

5. Programa de consola con `requests` que permita listar espacios, ver la ocupación de un día, reservar y anular.

Disponibilidad:

6. Script de monitorización que registre la disponibilidad y el tiempo de respuesta y avise por correo si el servicio cae.
7. Prueba de carga con Locust y un breve informe con los resultados (peticiones por segundo, tiempo de respuesta medio y percentil 95) con 1 y con 4 procesos trabajadores.

Ampliación: una página web sencilla (HTML + JavaScript con `fetch`) servida por la propia API que muestre la ocupación semanal.

TEMA 5

Técnicas de programación segura

En este tema aprenderás a…

- Aplicar las prácticas de programación segura y evitar las vulnerabilidades más frecuentes.
- Usar funciones resumen, HMAC, criptografía simétrica y asimétrica, y certificados digitales.
- Guardar contraseñas de forma segura y definir políticas de control de acceso basadas en roles.
- Autenticar y autorizar a los usuarios de un servicio web con JWT.
- Proteger las comunicaciones con TLS desde Python.

Resultado de aprendizaje asociado: **RA5**.

5.1 Prácticas de programación segura

La seguridad no es una capa que se añade al final, sino una forma de programar. La mayoría de los ataques no rompen la criptografía: aprovechan errores corrientes del código, como confiar en datos que envía el usuario.

5.1.1 Principios

No confíes en la entrada
Todo dato que viene de fuera (formularios, parámetros de la URL, mensajes de un socket, ficheros subidos, cabeceras, variables de entorno) puede ser malicioso. Se **valida** (tipo, longitud, formato, rango) usando listas de valores permitidos mejor que de prohibidos.

Mínimo privilegio
Cada usuario, proceso o componente debe tener sólo los permisos imprescindibles. Un servicio web no debe ejecutarse como `root` ni conectarse a la base de datos como administrador.

Defensa en profundidad
Varias capas de protección independientes: si una falla, quedan las demás.

Fallo seguro
Ante un error, el sistema debe quedar en un estado seguro: si no se puede comprobar un permiso, se deniega el acceso. Los mensajes de error que ve el usuario no deben revelar detalles internos (trazas, rutas, consultas).

Seguridad por diseño y por defecto
La configuración por defecto debe ser la segura; no confíes en que nadie descubra cómo funciona tu sistema (*no a la seguridad por oscuridad*).

Secretos fuera del código
Contraseñas, claves y tokens nunca se escriben en el código fuente ni se suben al repositorio: se leen de variables de entorno o de un gestor de secretos.

> **OWASP.** La fundación OWASP publica el **OWASP Top 10**, la lista de los riesgos de seguridad más críticos en aplicaciones web, y las *Cheat Sheets*, guías prácticas para evitarlos. Son la referencia profesional en este campo.

5.1.2 Vulnerabilidades frecuentes y cómo evitarlas en Python

Inyección SQL

```
import sqlite3
con = sqlite3.connect("app.db")
usuario = input("Usuario: ")       # el atacante escribe:  ' OR '1'='1

# MAL: la entrada se convierte en parte de la consulta
con.execute(f"SELECT * FROM usuarios WHERE nombre = '{usuario}'")

# BIEN: consulta parametrizada; el dato nunca se interpreta como SQL
con.execute("SELECT * FROM usuarios WHERE nombre = ?", (usuario,))
```

Inyección de órdenes

```
import subprocess
fichero = input("Fichero a comprimir: ")    # "a.txt; rm -rf ~"

subprocess.run(f"gzip {fichero}", shell=True)   # MAL
subprocess.run(["gzip", "--", fichero])         # BIEN: lista de argumentos
```

Recorrido de directorios (*path traversal*)

Si un servidor de ficheros construye la ruta con el nombre que envía el cliente, `../../etc/passwd` le permite salir de la carpeta permitida:

```
from pathlib import Path

BASE = Path("compartido").resolve()

def ruta_segura(nombre: str) -> Path:
    ruta = (BASE / nombre).resolve()
    if not ruta.is_relative_to(BASE):
        raise PermissionError("acceso fuera del directorio permitido")
    return ruta
```

Ejecución y deserialización de datos no confiables

- Nunca uses `eval()` ni `exec()` con datos externos. Para convertir texto en números usa `int()`/`float()`; para estructuras, `json.loads()`.

- Nunca uses `pickle.loads()` con datos que vengan de la red o de un fichero no confiable: deserializar un *pickle* puede ejecutar código arbitrario. Para intercambiar datos, JSON.

Aleatoriedad

El módulo `random` es predecible y **no sirve para seguridad**. Para generar tokens, contraseñas temporales o claves, `secrets`:

```
import secrets
token = secrets.token_urlsafe(32)              # token para enlaces o sesiones
codigo = "".join(secrets.choice("0123456789") for _ in range(6))
```

Otras buenas prácticas

- No registres datos sensibles (contraseñas, tokens, datos personales) en los *logs*.
- Limita el tamaño de lo que recibes (un `recv` o un fichero subido de 10 GB puede agotar la memoria) y pon *timeouts*.
- Mantén las dependencias actualizadas y revisa sus vulnerabilidades conocidas con `pip-audit`.
- Analiza tu código con herramientas de análisis estático como `bandit` (`bandit -r proyecto/`), que detecta muchos de los errores de esta sección.

5.2 Criptografía

La criptografía proporciona cuatro servicios de seguridad:

- **Confidencialidad**: sólo quien está autorizado puede leer la información (cifrado).
- **Integridad**: se puede detectar si la información ha sido modificada (resúmenes, MAC).
- **Autenticación**: se puede comprobar quién envió la información (MAC, firma digital).
- **No repudio**: el emisor no puede negar haberla enviado (firma digital).

> **No inventes tu propia criptografía.** Diseñar algoritmos o protocolos criptográficos es extremadamente difícil y los errores no se notan hasta que alguien los explota. Usa siempre **algoritmos estándar** a través de **librerías reconocidas**; en Python, `hashlib`, `hmac`, `secrets`, `ssl` (estándar) y `cryptography` (`pip install cryptography`).

5.2.1 Funciones resumen (*hash*)

Una función *hash* criptográfica transforma datos de cualquier tamaño en un resumen de tamaño fijo, de forma que:

- es imposible en la práctica obtener los datos a partir del resumen;
- un cambio mínimo en los datos produce un resumen completamente distinto;
- es imposible en la práctica encontrar dos entradas con el mismo resumen (colisión).

```python
import hashlib

print(hashlib.sha256(b"Hola").hexdigest())
print(hashlib.sha256(b"hola").hexdigest())     # totalmente distinto

def hash_fichero(ruta: str) -> str:
    h = hashlib.sha256()
    with open(ruta, "rb") as f:
        while bloque := f.read(65536):
            h.update(bloque)
    return h.hexdigest()
```

Algoritmos actuales: SHA-256, SHA-512 (familia SHA-2), SHA-3 y BLAKE2. **MD5 y SHA-1 están rotos** (se pueden fabricar colisiones) y no deben usarse para seguridad.

Un hash por sí solo garantiza integridad frente a errores, pero no frente a un atacante, que puede cambiar el mensaje y recalcular el hash. Para eso existe el **HMAC**, un resumen que depende además de una clave secreta compartida:

```python
import hmac, hashlib, secrets

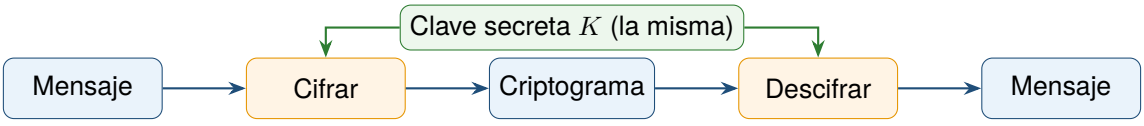
clave = secrets.token_bytes(32)
mensaje = b'{"orden": "transferir", "importe": 100}'
etiqueta = hmac.new(clave, mensaje, hashlib.sha256).hexdigest()

# El receptor, que conoce la clave, recalcula y compara
def verificar(mensaje: bytes, etiqueta: str) -> bool:
    esperado = hmac.new(clave, mensaje, hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperado, etiqueta)   # tiempo constante
```

Nota. `hmac.compare_digest` compara en tiempo constante. Con == la comparación termina en el primer carácter distinto, y midiendo el tiempo de respuesta un atacante podría adivinar la etiqueta carácter a carácter (*ataque de temporización*).

5.2.2 Criptografía de clave privada (simétrica)

Emisor y receptor comparten **la misma clave secreta**, que se usa para cifrar y para descifrar.



- Algoritmos actuales: **AES** (con claves de 128 o 256 bits) y **ChaCha20**. DES y 3DES están obsoletos.
- Son muy **rápidos**: se usan para cifrar grandes volúmenes de datos.
- Problema: la **distribución de la clave**. ¿Cómo hacemos llegar la clave al otro extremo sin que nadie la intercepte? Además, con n participantes hacen falta $n(n-1)/2$ claves.
- Hoy se usa **cifrado autenticado** (AES-GCM, ChaCha20-Poly1305), que además de cifrar garantiza la integridad.

Fernet

La forma más sencilla y segura de cifrar datos en Python es `Fernet`, de la librería `cryptography`. Combina AES y HMAC y se encarga de todos los detalles (vector de inicialización, autenticación, formato):

```
from cryptography.fernet import Fernet, InvalidToken

clave = Fernet.generate_key()        # guárdala en lugar seguro (¡no en el código!)
f = Fernet(clave)

token = f.encrypt("DNI: 12345678Z".encode())
print(token)                         # b'gAAAAAB...'
print(f.decrypt(token).decode())     # DNI: 12345678Z

try:
    Fernet(Fernet.generate_key()).decrypt(token)   # clave incorrecta
except InvalidToken:
    print("No se puede descifrar: clave incorrecta o datos manipulados")
```

AES-GCM

Si necesitamos más control (por ejemplo, para interoperar con otro sistema), podemos usar AES-GCM directamente. El *nonce* debe ser distinto en cada cifrado con la misma clave y se transmite junto al criptograma:

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

clave = AESGCM.generate_key(bit_length=256)
aes = AESGCM(clave)
nonce = os.urandom(12)                          # 96 bits, NUNCA se repite
cifrado = aes.encrypt(nonce, b"datos secretos", None)
paquete = nonce + cifrado                       # lo que se guarda o envía

descifrado = aes.decrypt(paquete[:12], paquete[12:], None)
```

Claves a partir de contraseñas

Una contraseña no es una clave: es corta y predecible. Para cifrar con una contraseña se **deriva** una clave con una función lenta y una **sal** aleatoria (que se guarda junto a los datos):

```
import base64, os
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

def clave_desde_contrasena(contrasena: str, sal: bytes) -> bytes:
    kdf = Scrypt(salt=sal, length=32, n=2**15, r=8, p=1)
    return base64.urlsafe_b64encode(kdf.derive(contrasena.encode()))

sal = os.urandom(16)
f = Fernet(clave_desde_contrasena("mi contraseña larga", sal))
datos_guardados = sal + f.encrypt(b"contenido del fichero")
```

5.2.3 Criptografía de clave pública (asimétrica)

Cada participante tiene **un par de claves** relacionadas matemáticamente: una **pública**, que puede conocer todo el mundo, y una **privada**, que sólo conoce su dueño. Lo que se cifra con una sólo se puede descifrar con la otra.

- **Cifrado**: para enviar un mensaje confidencial a Ana, se cifra con la *clave pública de Ana*; sólo Ana, con su clave privada, puede descifrarlo.
- **Firma digital**: Ana firma con su *clave privada*; todo el mundo puede verificar la firma con su clave pública. Proporciona autenticación, integridad y no repudio.
- Algoritmos: **RSA** (claves de al menos 2048 bits) y **curvas elípticas** (ECDSA, Ed25519, X25519), con claves mucho más cortas para la misma seguridad.
- Son **lentos** (cientos o miles de veces más que AES) y sólo pueden cifrar datos pequeños.

```
# Cifrado y firma con RSA
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa

privada = rsa.generate_private_key(public_exponent=65537, key_size=3072)
publica = privada.public_key()

# --- Cifrado: cualquiera cifra con la pública, sólo el dueño descifra ---
oaep = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                    algorithm=hashes.SHA256(), label=None)
cifrado = publica.encrypt(b"clave de sesion AES", oaep)
print(privada.decrypt(cifrado, oaep))

# --- Firma: el dueño firma con la privada, cualquiera verifica ---
pss = padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                  salt_length=padding.PSS.MAX_LENGTH)
documento = b"Acta de evaluacion 2DAM - 1.a evaluacion"
firma = privada.sign(documento, pss, hashes.SHA256())
try:
    publica.verify(firma, documento, pss, hashes.SHA256())
    print("Firma válida")
    publica.verify(firma, documento + b"!", pss, hashes.SHA256())
except InvalidSignature:
    print("El documento ha sido modificado")

# Guardar la clave privada cifrada con una contraseña (formato PEM)
pem = privada.private_bytes(
    serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8,
    serialization.BestAvailableEncryption("contraseña-muy-larga".encode()))
```

5.2.4 Cifrado híbrido

En la práctica se combinan ambos tipos: se genera una **clave de sesión** simétrica aleatoria, se cifran los datos con ella (rápido) y se envía la clave de sesión cifrada con la clave pública del destinatario (o se acuerda con un intercambio de claves Diffie-Hellman). Así funcionan TLS, PGP o el cifrado de las apps de mensajería.

5.2.5 Certificados digitales

¿Cómo sé que una clave pública pertenece realmente a mi banco y no a un atacante? Un **certificado digital** (formato X.509) es un documento que contiene una clave pública y la identidad de su dueño (por ejemplo, el dominio `www.mibanco.es`), **firmado por una Autoridad de Certificación** (CA) de confianza. Los sistemas operativos y navegadores incluyen las claves públicas de las CA raíz reconocidas. Este sistema de confianza se llama **infraestructura de clave pública** (PKI). El DNI electrónico y los certificados de la FNMT funcionan igual.

5.2.6 Principales aplicaciones de la criptografía

Aplicación	Técnicas
Almacenar contraseñas	Hash lento con sal (Argon2, bcrypt, scrypt)
Comunicaciones seguras (HTTPS, VPN, SSH)	TLS / SSH: cifrado híbrido, certificados
Cifrado de discos y ficheros	AES (BitLocker, LUKS, FileVault)
Firma de documentos y de software	Firma digital, certificados
Mensajería cifrada de extremo a extremo	Protocolo Signal (X25519, AES)
Tokens de autenticación	HMAC o firma (JWT)
Verificación de descargas	Hash SHA-256
Criptomonedas y *blockchain*	Hash, firmas con curvas elípticas

Criptografía poscuántica. Un ordenador cuántico suficientemente grande podría romper RSA y las curvas elípticas. Por eso, el NIST estandarizó en 2024 los primeros algoritmos poscuánticos (ML-KEM para el intercambio de claves y ML-DSA para firmas), y los navegadores y servidores ya usan modos híbridos en TLS. La criptografía simétrica (AES-256) y los hashes se consideran seguros frente a ellos.

5.3 Política de seguridad y control de acceso

Una **política de seguridad** define qué está permitido en un sistema: quién puede acceder, a qué recursos, qué operaciones puede hacer y en qué condiciones. Para aplicarla se distinguen dos fases:

- **Autenticación** (*¿quién eres?*): comprobar la identidad del usuario (contraseña, certificado, segundo factor…).
- **Autorización** (*¿qué puedes hacer?*): decidir si el usuario autenticado tiene permiso para la operación solicitada.

Ejemplos de reglas de una política: contraseñas de al menos 12 caracteres; bloqueo temporal de la cuenta tras 5 intentos fallidos; sesiones que caducan a los 30 minutos; sólo los administradores pueden borrar usuarios; todas las operaciones de modificación quedan registradas (auditoría).

5.3.1 Almacenamiento seguro de contraseñas

Las contraseñas **nunca** se guardan en claro ni cifradas (quien tenga la clave las recupera todas): se guarda un **hash**. Pero un hash rápido como SHA-256 no sirve: con una tarjeta gráfica se prueban miles de millones de contraseñas por segundo. Se necesita:

- Una **sal**: valor aleatorio distinto para cada usuario, guardado junto al hash. Impide usar tablas precalculadas y que dos usuarios con la misma contraseña tengan el mismo hash.
- Un algoritmo **deliberadamente lento** y configurable: **Argon2id** (recomendado por OWASP), **bcrypt** o **scrypt**.

```python
import bcrypt              # pip install bcrypt

def generar_hash(contrasena: str) -> bytes:
    return bcrypt.hashpw(contrasena.encode(), bcrypt.gensalt(rounds=12))

def comprobar(contrasena: str, hash_guardado: bytes) -> bool:
    return bcrypt.checkpw(contrasena.encode(), hash_guardado)

h = generar_hash("Correcta-Caballo-Pila-Grapa")
print(h)        # b'$2b$12$...'  (algoritmo, coste, sal y hash en uno)
print(comprobar("Correcta-Caballo-Pila-Grapa", h), comprobar("1234", h))
```

5.3.2 Control de acceso basado en roles (RBAC)

En lugar de asignar permisos a cada usuario, se definen **roles** (administrador, profesor, alumno…) con un conjunto de permisos, y a cada usuario se le asignan uno o varios roles. Es mucho más fácil de gestionar y auditar: cuando alguien cambia de puesto, basta con cambiar su rol.

Permiso	alumno	profesor	admin
Ver sus notas	✓	✓	✓
Poner notas		✓	✓
Gestionar usuarios			✓
Ver el registro de auditoría			✓

```python
# RBAC con un decorador
from functools import wraps

PERMISOS = {
    "alumno":   {"ver_notas"},
    "profesor": {"ver_notas", "poner_notas"},
    "admin":    {"ver_notas", "poner_notas", "gestionar_usuarios", "auditar"},
}

class AccesoDenegado(Exception):
    pass

def requiere_permiso(permiso: str):
    def decorador(funcion):
        @wraps(funcion)
        def envoltura(usuario: dict, *args, **kwargs):
            permitidos = set().union(*(PERMISOS.get(r, set())
                                       for r in usuario["roles"]))
            if permiso not in permitidos:        # fallo seguro: denegar
                raise AccesoDenegado(f"{usuario['nombre']} no puede {permiso}")
            return funcion(usuario, *args, **kwargs)
        return envoltura
```

```
    return decorador

@requiere_permiso("poner_notas")
def poner_nota(usuario: dict, alumno: str, nota: float) -> None:
    print(f"{usuario['nombre']} pone un {nota} a {alumno}")

profe = {"nombre": "marta", "roles": ["profesor"]}
alumna = {"nombre": "lucia", "roles": ["alumno"]}
poner_nota(profe, "lucia", 8.5)
try:
    poner_nota(alumna, "lucia", 10)
except AccesoDenegado as e:
    print("Acceso denegado:", e)
```

5.3.3 Control de acceso en un servicio web

Como HTTP no tiene estado, después de autenticarse el usuario necesita demostrar en cada petición quién es. Lo habitual en las APIs es que el servidor le entregue un **token**. Un **JWT** (*JSON Web Token*) es un token que contiene datos (usuario, roles, caducidad) y está **firmado** por el servidor, por lo que este puede verificarlo sin guardar nada. Ojo: el contenido de un JWT está codificado, *no cifrado*; cualquiera puede leerlo, pero nadie puede modificarlo sin invalidar la firma.

```
# Autenticación con JWT y roles en FastAPI (api_segura.py)
import os
from datetime import datetime, timedelta, timezone
from typing import Annotated

import bcrypt
import jwt                                      # pip install pyjwt
from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

CLAVE_JWT = os.environ["CLAVE_JWT"]             # secreto fuera del código
DURACION = timedelta(minutes=30)

USUARIOS = {   # en una aplicación real, en la base de datos
    "marta": {"hash": bcrypt.hashpw(b"profe-segura-2026", bcrypt.gensalt()),
              "roles": ["profesor"]},
    "admin": {"hash": bcrypt.hashpw(b"admin-muy-segura", bcrypt.gensalt()),
              "roles": ["admin"]},
}

app = FastAPI()
oauth2 = OAuth2PasswordBearer(tokenUrl="login")

@app.post("/login")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]) -> dict:
    usuario = USUARIOS.get(form.username)
    if not usuario or not bcrypt.checkpw(form.password.encode(), usuario["hash"]):
        # Mismo mensaje en ambos casos: no revelar si el usuario existe
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Credenciales incorrectas")
    datos = {"sub": form.username, "roles": usuario["roles"],
             "exp": datetime.now(timezone.utc) + DURACION}
    return {"access_token": jwt.encode(datos, CLAVE_JWT, algorithm="HS256"),
            "token_type": "bearer"}
```

```python
def usuario_actual(token: Annotated[str, Depends(oauth2)]) -> dict:
    try:
        return jwt.decode(token, CLAVE_JWT, algorithms=["HS256"])
    except jwt.InvalidTokenError:              # firma incorrecta o caducado
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Token no válido",
                            headers={"WWW-Authenticate": "Bearer"})

def requiere_rol(*roles: str):
    def comprobar(usuario: Annotated[dict, Depends(usuario_actual)]) -> dict:
        if not set(roles) & set(usuario["roles"]):
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Permiso insuficiente")
        return usuario
    return comprobar

@app.get("/notas")
def ver_notas(u: Annotated[dict, Depends(usuario_actual)]) -> dict:
    return {"usuario": u["sub"], "notas": []}

@app.delete("/usuarios/{nombre}")
def borrar_usuario(nombre: str,
                   u: Annotated[dict, Depends(requiere_rol("admin"))]) -> dict:
    return {"borrado": nombre, "por": u["sub"]}
```

```
$ export CLAVE_JWT=$(python -c "import secrets; print(secrets.token_hex(32))")
$ fastapi dev api_segura.py
$ curl -X POST localhost:8000/login -d "username=marta&password=profe-segura-2026"
{"access_token":"eyJhbGciOiJIUzI1NiIs...","token_type":"bearer"}
$ curl -X DELETE localhost:8000/usuarios/pepe -H "Authorization: Bearer eyJhbGci..."
{"detail":"Permiso insuficiente"}
```

> **Nota.** Fíjate en la diferencia entre `401 Unauthorized` (no sabemos quién eres: autenticación) y `403 Forbidden` (sabemos quién eres, pero no tienes permiso: autorización).

5.4 Comunicaciones seguras

Todo lo que viaja por la red sin cifrar puede ser leído y modificado por cualquiera que esté en el camino (la Wi-Fi de una cafetería, un router comprometido...). Los métodos para asegurar la información transmitida son:

- Cifrar el **canal**: todo lo que se envía por la conexión va cifrado y autenticado. Es lo que hace **TLS**.
- Cifrar el **mensaje**: se cifra (y firma) el contenido antes de enviarlo, por ejemplo con Fernet o PGP. Protege incluso si pasa por intermediarios (cifrado de extremo a extremo).
- Garantizar sólo la **integridad** y el origen con HMAC o firmas, cuando el contenido no es confidencial.

5.4.1 Protocolos seguros

TLS (*Transport Layer Security*)
Sucesor de SSL (que está obsoleto, aunque se siga usando el nombre). Se coloca entre TCP y

el protocolo de aplicación: HTTPS es HTTP sobre TLS; lo mismo ocurre con SMTPS, IMAPS, etc. La versión actual es TLS 1.3; TLS 1.0 y 1.1 están prohibidas. En la **negociación** (*handshake*), el servidor envía su certificado, el cliente lo verifica, ambos acuerdan una clave de sesión mediante Diffie-Hellman y, a partir de ahí, todo se cifra con criptografía simétrica.

SSH
Acceso remoto seguro, ejecución de órdenes y transferencia de ficheros (SFTP, SCP). Suele autenticar a los usuarios con pares de claves.

Otros
WireGuard e IPsec (VPN), S/MIME y OpenPGP (correo), protocolo Signal (mensajería).

5.4.2 Sockets seguros con `ssl`

El módulo `ssl` de Python envuelve un socket normal en un socket TLS, que se usa exactamente igual. Primero necesitamos un certificado. Para pruebas generamos uno **autofirmado** con OpenSSL:

```
$ openssl req -x509 -newkey rsa:3072 -nodes -days 365 \
      -keyout clave.pem -out cert.pem -subj "/CN=localhost" \
      -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

```python
# servidor_tls.py
import socket
import ssl

contexto = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
contexto.minimum_version = ssl.TLSVersion.TLSv1_2
contexto.load_cert_chain("cert.pem", "clave.pem")

with socket.create_server(("127.0.0.1", 8443)) as servidor:
    with contexto.wrap_socket(servidor, server_side=True) as servidor_tls:
        print("Servidor TLS en el puerto 8443")
        while True:
            try:
                conexion, direccion = servidor_tls.accept()   # hace el handshake
            except ssl.SSLError as e:
                print("Negociación TLS fallida:", e)
                continue
            with conexion:
                print(direccion, conexion.version(), conexion.cipher()[0])
                datos = conexion.recv(1024)
                conexion.sendall(b"Recibido de forma segura: " + datos)
```

```python
# cliente_tls.py
import socket
import ssl

# Confiamos en nuestro certificado autofirmado. Con un servidor real
# (certificado firmado por una CA) bastaría con create_default_context().
contexto = ssl.create_default_context(cafile="cert.pem")

with socket.create_connection(("127.0.0.1", 8443)) as s:
    with contexto.wrap_socket(s, server_hostname="localhost") as s_tls:
        print("Versión:", s_tls.version())            # TLSv1.3
        print("Certificado:", s_tls.getpeercert()["subject"])
```

```
        s_tls.sendall(b"mensaje confidencial")
        print(s_tls.recv(1024).decode())
```

Nunca desactives la verificación. Con `create_default_context()` el cliente comprueba que el certificado está firmado por una CA de confianza y que corresponde al nombre del servidor. Desactivarlo (`verify=False` en `requests`, `CERT_NONE` en `ssl`) para «quitar un error molesto» elimina la protección contra ataques de intermediario (*man-in-the-middle*): la conexión va cifrada... con el atacante.

5.4.3 HTTPS en nuestros servicios

- **Cliente**: `requests` y `urllib` usan HTTPS y verifican certificados automáticamente al usar URL `https://`. Para un certificado propio: `requests.get(url, verify="cert.pem")`.
- **Servidor en desarrollo**: Uvicorn acepta el certificado directamente: `uvicorn main:app --ssl-keyfile clave.pem --ssl-certfile cert.pem`.
- **Servidor en producción**: lo habitual es que un proxy inverso (Nginx, Caddy, Traefik) gestione TLS con certificados gratuitos de *Let's Encrypt* y reenvíe las peticiones a la aplicación.

5.5 Depuración y documentación de aplicaciones seguras

- Ejecuta `bandit` y `pip-audit` sobre tus proyectos e incluye su informe en la documentación.
- Documenta la **política de seguridad**: roles y permisos, reglas de contraseñas, dónde se guardan las claves y cómo se rotan.
- Registra los eventos de seguridad (inicios de sesión, fallos, accesos denegados, cambios de permisos) en un registro de **auditoría**, sin incluir contraseñas ni tokens.
- Comprueba el tráfico con Wireshark: con TLS no debe verse nada legible después de la negociación.
- Prueba los casos negativos: contraseña incorrecta, token caducado, token manipulado, usuario sin permiso, rutas con `..`, entradas enormes.
- Para diagnosticar TLS: `openssl s_client -connect localhost:8443` muestra la negociación y el certificado.

Resumen del tema.

- Programar de forma segura: validar la entrada, mínimo privilegio, fallo seguro, consultas parametrizadas, `secrets`, nada de `eval`/`pickle` con datos externos, secretos fuera del código.
- Hash (integridad), HMAC (integridad y autenticación con clave compartida), cifrado simétrico (confidencialidad, rápido), asimétrico (intercambio de claves y firma) y certificados (confianza).
- Contraseñas: hash lento con sal (bcrypt, Argon2).
- Control de acceso: autenticación + autorización; RBAC; tokens JWT en servicios web.

- Comunicaciones: TLS con verificación de certificados (`ssl`, HTTPS).

5.6 Ejercicios

Ejercicio 5.1 — Caza de vulnerabilidades

Dificultad: ★

Escribe un pequeño programa deliberadamente vulnerable (consulta SQL construida con f-string, `subprocess` con `shell=True`, uso de `random` para generar un token, `eval` de una entrada, contraseña escrita en el código). Pasa `bandit` y corrige cada problema que detecte, explicando el riesgo en un comentario.

Ejercicio 5.2 — Verificador de integridad

Dificultad: ★

Programa una herramienta que calcule el SHA-256 de todos los ficheros de un directorio y los guarde en un fichero de referencia; ejecutada después con `--verificar`, debe indicar qué ficheros se han modificado, añadido o borrado. Amplíala para que el fichero de referencia esté protegido con HMAC y así un atacante no pueda alterarlo.

Ejercicio 5.3 — Cifrador de ficheros

Dificultad: ★★

Herramienta de línea de órdenes que cifre y descifre ficheros con una contraseña: deriva la clave con Scrypt y una sal aleatoria y cifra con Fernet. El fichero cifrado debe contener la sal. Comprueba que con una contraseña incorrecta o un fichero manipulado el programa lo detecta.

Ejercicio 5.4 — Firma de documentos

Dificultad: ★★

Programa con tres subórdenes: `generar` (crea un par de claves RSA o Ed25519 y guarda la privada cifrada con contraseña), `firmar fichero` (genera `fichero.firma`) y `verificar fichero clave_publica.pem`.

Ejercicio 5.5 — Inicio de sesión robusto

Dificultad: ★★

Implementa un sistema de usuarios en SQLite con registro e inicio de sesión que cumpla esta política: contraseñas de al menos 12 caracteres que no estén en una lista de contraseñas comunes, hash con bcrypt, bloqueo de la cuenta durante 5 minutos tras 5 intentos fallidos, y registro de auditoría de todos los intentos.

Ejercicio 5.6 — Chat cifrado

Dificultad: ★★★

Toma el chat del Tema 3 y protégelo con TLS. Después, añade autenticación (usuario y contraseña al conectar) y un rol de moderador que pueda expulsar usuarios con `/expulsar nombre`. Comprueba con Wireshark que los mensajes no son legibles.

Ejercicio 5.7 — Intercambio de claves

Dificultad: ★★★

Implementa un intercambio de claves con X25519 (`cryptography`) entre un cliente y un servidor por sockets: ambos generan un par de claves, se intercambian las públicas, calculan el secreto compartido, derivan una clave con HKDF y la usan para cifrar los mensajes con AES-GCM. Explica por qué este esquema, sin certificados, es vulnerable a un ataque de intermediario.

Proyecto: Bóveda de secretos para equipos

Desarrolla un servicio para que un equipo de desarrollo guarde y comparta secretos (contraseñas de servidores, claves de API) de forma segura.

Requisitos:

1. API REST con FastAPI y persistencia en SQLite, servida por **HTTPS**.
2. Usuarios con contraseñas protegidas con bcrypt o Argon2 y autenticación con JWT de caducidad corta.
3. Tres roles: `lector` (consulta secretos de sus proyectos), `editor` (además crea y modifica) y `admin` (gestiona usuarios, proyectos y permisos). Documenta la matriz de permisos.
4. Los secretos se guardan **cifrados** en la base de datos (Fernet o AES-GCM). La clave maestra se lee de una variable de entorno, nunca del código ni de la base de datos.
5. Registro de auditoría: quién consultó o modificó cada secreto y cuándo; los valores de los secretos nunca aparecen en el registro.
6. Cliente de línea de órdenes con `requests` que verifique el certificado del servidor.
7. Pruebas automáticas de los casos negativos (accesos denegados, tokens manipulados o caducados) e informe de `bandit` y `pip-audit` sin problemas de severidad alta.

Ampliación: rotación de la clave maestra (volver a cifrar todos los secretos con una clave nueva sin detener el servicio) y segundo factor de autenticación con códigos TOTP (librería `pyotp`).

APÉNDICE A

Preparación del entorno de trabajo

A.1 Python con y sin GIL

En este módulo conviene tener instaladas **dos versiones del intérprete**: la compilación **sin GIL** de Python 3.15 (`python3.15t`), que usaremos por defecto, y la versión normal de la misma versión (`python3.15`), para comparar resultados y para las bibliotecas que todavía no admitan la primera (el apartado 0.1.2 del Tema 0 explica la diferencia). Si en tu sistema aún no está disponible la 3.15, la 3.14 (`python3.14t`) sirve igual.

A.1.1 Instalación con uv (recomendada, cualquier sistema)

uv es un gestor de versiones de Python, entornos virtuales y paquetes. Funciona igual en Windows, macOS y Linux, no necesita permisos de administrador y permite tener varias versiones instaladas a la vez. Es la opción más sencilla para el aula.

```
# 1. Instalar uv
$ curl -LsSf https://astral.sh/uv/install.sh | sh                  # Linux / macOS
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"  #
    ↪ Windows

# 2. Instalar Python 3.15 sin GIL y la versión normal
$ uv python install 3.15t
$ uv python install 3.15
$ uv python list --only-installed

# 3. Crear el entorno virtual del proyecto con la versión sin GIL
$ uv venv --python 3.15t
$ uv pip install psutil requests "fastapi[standard]"
$ uv run python programa.py          # ejecuta usando el entorno .venv
```

Elegir explícitamente la versión. En uv, `3.15t` (o `3.15+freethreaded`) pide la compilación sin GIL y `3.15+gil`, la normal. Si escribes sólo `3.15`, uv puede elegir cualquiera de las dos según lo que tengas instalado, así que, para este módulo, indica siempre la `t`.

A.1.2 Instalación con los instaladores oficiales

Windows: Python Install Manager. Desde Python 3.14, la forma oficial de instalar Python en Windows es el Python Install Manager, que se instala desde la Microsoft Store o con `winget` y proporciona la orden `py`. El instalador `.exe` clásico dejará de publicarse con Python 3.16.

```
> winget install 9NQ7512CXL7T          # instala el Python Install Manager
> py list --online                      # versiones disponibles
> py install 3.15t                      # Python 3.15 sin GIL
> py install 3.15                       # versión normal
> py -V:3.15t --version                 # ejecutar una versión concreta
> py -V:3.15t -m venv .venv             # entorno virtual con 3.15t
```

macOS. El instalador `.pkg` de `https://www.python.org/downloads/` para Python 3.15 instala la compilación sin GIL por defecto, como `python3.15t`, junto a la normal (`python3.15`). Con Homebrew existe el paquete `python-freethreading`.

Linux. Las distribuciones empaquetan la versión sin GIL por separado. Por ejemplo, en Fedora el paquete es `python3.14-freethreading` y en Ubuntu el repositorio *deadsnakes* ofrece `python3.14-nogil`. Como los nombres y las versiones disponibles cambian según la distribución, en Linux te recomendamos uv. Hay una guía actualizada para cada sistema en `https://py-free-threading.github.io/installing-cpython/`.

A.1.3 Comprobar la instalación

```
$ python3.15t -VV
Python 3.15.0 free-threading build (main, Oct  9 2026, ...)
$ python3.15t -c "import sys; print(sys._is_gil_enabled())"
False
```

A.2 El editor

1. Instala un editor: **Visual Studio Code** con la extensión *Python* de Microsoft (incluye Pylance y el depurador), o **PyCharm Community**.
2. Para cada proyecto, crea un entorno virtual (`uv venv --python 3.15t` o `python3.15t -m venv .venv`) y selecciónalo como intérprete en el editor (en VS Code: *Python: Select Interpreter*).
3. Para depurar programas que crean procesos, añade `"subProcess": true` a la configuración de depuración (`launch.json`); el panel *Call Stack* muestra todos los hilos del programa.

A.3 Bibliotecas del curso

Paquetes que usaremos a lo largo del curso (con uv, anteponiendo `uv` a cada orden `pip`):

```
$ pip install pytest mypy pydantic                  # todo el curso
$ pip install psutil                                # Tema 1
$ pip install requests "fastapi[standard]" sqlmodel  # Tema 4
$ pip install cryptography bcrypt pyjwt bandit pip-audit   # Tema 5
```

Cuidado. Si al importar una biblioteca aparece un aviso de que el GIL se ha vuelto a activar (*RuntimeWarning: The global interpreter lock (GIL) has been enabled…*), esa biblioteca aún no está preparada para la versión sin GIL. El programa funciona igual, pero los hilos dejan de ejecutarse en paralelo. Comprueba si hay una versión más reciente de la biblioteca o usa en ese proyecto la versión normal del intérprete.

A.4 Herramientas de red

Para probar y depurar los programas de red de los temas 3 a 5 usaremos estas herramientas:

Herramienta	Para qué la usaremos y cómo se instala
`curl`	Cliente HTTP de línea de órdenes. Incluido en Linux, macOS y Windows 10 o superior.
`nc` (*netcat*)	Conectarse a cualquier servidor TCP o UDP, o hacerse pasar por uno (`nc -l 5000`). En Linux y macOS suele venir instalado; en Windows se usa `ncat`, incluido en Nmap (`https://nmap.org`).
Wireshark	Captura y muestra el tráfico de red, byte a byte (`https://www.wireshark.org`).
`openssl`	Crear certificados de prueba y depurar conexiones TLS (Tema 5). Incluido en Linux y macOS; en Windows viene con Git for Windows.
Bruno o Postman	Clientes gráficos para probar APIs HTTP (Tema 4). Bruno es libre: `https://www.usebruno.com`.

Cortafuegos. Para probar un servidor desde otro equipo del aula, el cortafuegos del sistema debe permitir las conexiones entrantes al puerto que uses (Windows lo pregunta la primera vez que un programa abre un puerto). Mientras pruebas en un solo equipo, usa `127.0.0.1` (`localhost`) y no hace falta tocar nada.

A.5 Servidor de correo de pruebas con Docker

En el Tema 4 enviaremos correos desde Python. Para no usar una cuenta real, levantaremos un servidor SMTP de pruebas, Mailpit, que recibe los correos sin enviarlos y los muestra en una página web. La forma más cómoda es con **Docker** (Docker Desktop en Windows y macOS; Docker Engine en Linux). Guarda este fichero como `compose.yaml` en una carpeta del curso:

```
services:
  mailpit:
    image: axllent/mailpit:latest
    ports:
      - "1025:1025"      # SMTP
      - "8025:8025"      # interfaz web
```

```
$ docker compose up -d              # arranca el servidor
$ docker compose ps                 # comprueba que está en marcha
$ docker compose down               # lo elimina
```

Conexión desde Python: servidor `localhost`, puerto `1025`, sin usuario ni contraseña. Los correos recibidos se ven en `http://localhost:8025`.

Nota. Si no puedes usar Docker, la biblioteca `aiosmtpd` ofrece un servidor SMTP mínimo que muestra los correos por la terminal: `pip install aiosmtpd` y `python -m aiosmtpd -n -l localhost:1025`.

Cuidado. Estos servidores de pruebas no tienen autenticación ni cifrado: son adecuados sólo para el equipo de desarrollo, nunca para un servidor accesible desde Internet.

APÉNDICE B

Pruebas y documentación del código

Todos los resultados de aprendizaje del módulo terminan con el mismo criterio: *"se han depurado y documentado las aplicaciones desarrolladas"*. La depuración y el registro de eventos se explican en el Tema 0 (apartado 0.18). Este anexo explica cómo hacer el resto en Python de forma sistemática: escribir **pruebas automáticas** con `pytest` y **documentar** módulos, funciones y programas.

B.1 Por qué escribir pruebas automáticas

Probar un programa a mano (ejecutarlo, introducir datos y mirar si el resultado parece correcto) es lento, poco fiable y hay que repetirlo cada vez que se cambia algo. Una **prueba automática** es un pequeño fragmento de código que ejecuta una parte del programa con unos datos concretos y **comprueba** que el resultado es el esperado. Una vez escritas, las pruebas se ejecutan en segundos tantas veces como haga falta, y avisan inmediatamente si un cambio ha estropeado algo que antes funcionaba (una *regresión*).

Las pruebas que veremos son **pruebas unitarias**: cada una comprueba una unidad pequeña (una función, un método) de forma aislada. Además sirven de documentación: muestran con ejemplos cómo se usa cada función y qué debe devolver.

B.2 De `assert` a las pruebas

En el Tema 0 vimos que `assert condición` lanza un `AssertionError` si la condición no se cumple. Una prueba no es más que una función que llama al código y comprueba el resultado con `assert`:

```
def test_media_basica():
    assert media([7, 8, 9]) == 8.0
```

Podríamos llamar a mano a todas estas funciones, pero un **marco de pruebas** se encarga de encontrarlas, ejecutarlas una a una (aunque alguna falle), mostrar un informe claro de las que fallan y por qué, y ofrecer herramientas para los casos habituales. En Python hay dos: `unittest`, incluido en la biblioteca estándar y de estilo similar a JUnit de Java, y `pytest`, que es el más utilizado por su sencillez. Usaremos `pytest`.

```
$ pip install pytest
```

B.3 Un ejemplo completo

Partimos de un módulo `notas.py` con dos funciones:

```
# notas.py
def media(notas: list[float]) -> float:
    """Calcula la media de una lista de notas, redondeada a dos decimales."""
```

```
    if not notas:
        raise ValueError("La lista de notas está vacía")
    if any(not 0 <= n <= 10 for n in notas):
        raise ValueError("Las notas deben estar entre 0 y 10")
    return round(sum(notas) / len(notas), 2)


def calificacion(nota: float) -> str:
    """Devuelve la calificación textual de una nota (Suspenso ... Sobresaliente)."""
    if not 0 <= nota <= 10:
        raise ValueError(f"Nota fuera de rango: {nota}")
    if nota < 5:
        return "Suspenso"
    if nota < 7:
        return "Aprobado"
    if nota < 9:
        return "Notable"
    return "Sobresaliente"
```

Las pruebas se guardan en una carpeta `tests`, en ficheros cuyo nombre empieza por `test_`:

```
proyecto/
├── pyproject.toml
├── notas.py
└── tests/
    ├── conftest.py
    └── test_notas.py
```

```
# tests/test_notas.py
import pytest
from notas import media, calificacion


def test_media_basica():
    assert media([7, 8, 9]) == 8.0


def test_media_redondea():
    assert media([7, 8, 8]) == 7.67


def test_media_lista_vacia():
    with pytest.raises(ValueError, match="vacía"):
        media([])
```

B.3.1 Reglas que sigue `pytest`

- Busca las pruebas en los ficheros `test_*.py` (o `*_test.py`) y, dentro de ellos, ejecuta las funciones cuyo nombre empieza por `test_`.
- Cada prueba es independiente: si una falla, las demás se siguen ejecutando.
- Una prueba **pasa** si termina sin lanzar ninguna excepción y **falla** si un `assert` no se cumple o se produce una excepción no esperada.

- Los nombres de las pruebas deben describir qué se comprueba: cuando una falle, el nombre será lo primero que veas.

B.3.2 Ejecutar las pruebas

Para que las pruebas puedan importar los módulos del proyecto, se indica en `pyproject.toml` dónde están el código y las pruebas:

```
# pyproject.toml
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
```

```
$ pytest              # ejecuta todas las pruebas
$ pytest -v           # muestra cada prueba
$ pytest tests/test_notas.py::test_media_basica    # sólo una prueba
$ pytest -k media     # sólo las pruebas cuyo nombre contiene "media"
$ pytest -x           # se detiene en el primer fallo
```

Cuando todo va bien, `pytest` muestra un punto por cada prueba superada. Si una prueba falla, muestra exactamente qué línea ha fallado y con qué valores. Por ejemplo, si en `test_media_redondea` hubiéramos escrito `7.66` en lugar de `7.67`:

```
$ pytest
tests/test_notas.py .F.........                                          [100%]

================================== FAILURES ===================================
_____________________________ test_media_redondea _____________________________

    def test_media_redondea():
>       assert media([7, 8, 8]) == 7.66
E       assert 7.67 == 7.66
E        +  where 7.67 = media([7, 8, 8])

tests/test_notas.py:11: AssertionError
============================ short test summary info ============================
FAILED tests/test_notas.py::test_media_redondea - assert 7.67 == 7.66
========================= 1 failed, 11 passed in 0.04s =========================
```

B.4 Herramientas de `pytest`

B.4.1 Comprobar que se lanza una excepción

Probar los casos de error es tan importante como probar los correctos. `pytest.raises` comprueba que el bloque lanza la excepción indicada (y, opcionalmente, que su mensaje contiene un texto); si no la lanza, la prueba falla:

```
def test_media_lista_vacia():
    with pytest.raises(ValueError, match="vacía"):
        media([])
```

B.4.2 Pruebas parametrizadas

Para probar la misma función con muchos datos distintos no hace falta escribir una prueba por cada caso. Con `@pytest.mark.parametrize` se escribe una sola y `pytest` la ejecuta una vez por cada juego de datos, informando de cada uno por separado:

```
@pytest.mark.parametrize("nota, esperada", [
    (0, "Suspenso"),
    (4.99, "Suspenso"),
    (5, "Aprobado"),
    (7, "Notable"),
    (9, "Sobresaliente"),
    (10, "Sobresaliente"),
])
def test_calificacion(nota, esperada):
    assert calificacion(nota) == esperada


@pytest.mark.parametrize("nota", [-1, 10.5])
def test_calificacion_fuera_de_rango(nota):
    with pytest.raises(ValueError):
        calificacion(nota)
```

```
$ pytest -v
...
tests/test_notas.py::test_calificacion[0-Suspenso] PASSED                [ 41%]
tests/test_notas.py::test_calificacion[4.99-Suspenso] PASSED             [ 50%]
tests/test_notas.py::test_calificacion[5-Aprobado] PASSED                [ 58%]
...
tests/test_notas.py::test_calificacion_fuera_de_rango[10.5] PASSED       [100%]
```

> **Qué casos probar.** Además del caso "normal", conviene probar los **valores límite** (justo antes y justo después de cada frontera: 4.99 y 5, 0 y 10), los **casos vacíos** (lista vacía, cadena vacía, fichero sin datos) y los **datos no válidos**, que deben producir la excepción adecuada.

B.4.3 Números decimales

Por cómo se representan los números reales en el ordenador, `0.1 + 0.2 == 0.3` es `False`. Para comparar resultados con decimales se usa `pytest.approx`, que admite un pequeño margen de error:

```
def test_suma_decimales():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

B.4.4 *Fixtures*: preparar los datos de las pruebas

Muchas pruebas necesitan preparar algo antes de empezar: un fichero con datos, una base de datos en memoria, una conexión... Una **fixture** es una función decorada con `@pytest.fixture` que prepara ese recurso; cualquier prueba que tenga un **parámetro con el mismo nombre** lo recibe automáticamente. Las *fixtures* que se usan en varios ficheros de

pruebas se guardan en `tests/conftest.py`, que `pytest` carga solo.

`pytest` incluye algunas *fixtures* muy útiles. La más usada en este módulo es `tmp_path`: un directorio temporal nuevo y vacío para cada prueba, en el que se pueden crear ficheros sin ensuciar el proyecto.

```python
# tests/conftest.py
import json
import pytest


@pytest.fixture
def fichero_notas(tmp_path):
    """Crea un fichero JSON de notas de prueba y devuelve su ruta."""
    ruta = tmp_path / "notas.json"
    ruta.write_text(json.dumps({"Ana": [7, 8], "Luis": [4, 5]}), encoding="utf-8")
    return ruta
```

```python
# tests/test_fichero.py
import json
from notas import media


def test_medias_desde_fichero(fichero_notas):    # recibe la ruta creada
    datos = json.loads(fichero_notas.read_text(encoding="utf-8"))
    medias = {nombre: media(n) for nombre, n in datos.items()}
    assert medias == {"Ana": 7.5, "Luis": 4.5}
```

Si la *fixture* tiene que liberar el recurso al terminar la prueba (cerrar una conexión, borrar datos), se usa `yield` en lugar de `return`: lo que va después del `yield` se ejecuta al acabar la prueba, como en un gestor de contexto. En el apartado B.5 hay una *fixture* que arranca un servidor y lo detiene al terminar, y en el Tema 4, otra que crea una base de datos de pruebas.

Fixture	Qué proporciona
`tmp_path`	Un directorio temporal (`Path`) distinto para cada prueba.
`capsys`	Lo que el código ha escrito por pantalla: `capsys.readouterr().out`.
`monkeypatch`	Permite sustituir temporalmente variables de entorno, atributos o funciones (por ejemplo, simular `input()`).

B.4.5 Cobertura

La **cobertura** indica qué porcentaje de las líneas del código se ejecuta durante las pruebas, y sobre todo **qué líneas no se prueban nunca**. Se mide con el complemento `pytest-cov`:

```
$ pip install pytest-cov
$ pytest --cov=notas --cov-report=term-missing
Name       Stmts   Miss  Cover   Missing
----------------------------------------
notas.py      16      1    94%   7
----------------------------------------
TOTAL         16      1    94%
```

La línea 7 de `notas.py` (el `raise` de `media` para notas fuera de rango) no la ejecuta ninguna

prueba: falta ese caso. Una cobertura alta no garantiza que el código sea correcto, pero una baja indica con seguridad que hay partes sin probar.

B.5 Probar código concurrente y de red

Los programas de este módulo tienen dos dificultades añadidas a la hora de probarlos: el resultado puede depender del orden, impredecible, en que se ejecutan los hilos y procesos, y muchas funciones necesitan que haya un servidor escuchando al otro lado. Estas son las pautas básicas.

B.5.1 Pruebas con hilos

- **Provoca la concurrencia**: lanza muchos hilos que hagan muchas operaciones. Un error de sincronización que aparece una vez de cada mil ejecuciones no se detecta con dos hilos y diez operaciones.
- **Nunca esperes indefinidamente**: usa `join(timeout=...)`, `get(timeout=...)` o `result(timeout=...)` y comprueba después que el hilo ha terminado. Si hay un interbloqueo, la prueba falla en lugar de quedarse colgada.
- Recuerda que **una prueba que pasa no demuestra que el código sea correcto**: sólo que no ha fallado esa vez. Las pruebas de concurrencia complementan, no sustituyen, el razonamiento sobre las secciones críticas.

```
# contador.py
import threading

class ContadorSeguro:
    def __init__(self) -> None:
        self._valor = 0
        self._cerrojo = threading.Lock()

    def incrementar(self) -> None:
        with self._cerrojo:
            self._valor += 1

    @property
    def valor(self) -> int:
        return self._valor
```

```
# tests/test_contador.py
import threading
from contador import ContadorSeguro


def test_incrementos_concurrentes():
    contador = ContadorSeguro()

    def trabajar():
        for _ in range(10_000):
            contador.incrementar()

    hilos = [threading.Thread(target=trabajar) for _ in range(8)]
    for h in hilos:
```

```
        h.start()
    for h in hilos:
        h.join(timeout=10)                # nunca esperar indefinidamente
        assert not h.is_alive(), "un hilo no ha terminado: ¿interbloqueo?"
    assert contador.valor == 80_000
```

Si se quita el `with self._cerrojo:`, la prueba falla con un valor menor que 80 000 (y distinto en cada ejecución), sobre todo con la versión sin GIL del intérprete.

B.5.2 Pruebas de un servidor

Para probar un servidor no hace falta arrancarlo a mano en otra terminal: una *fixture* lo arranca en un hilo antes de cada prueba y lo detiene después. Dos detalles importantes:

- El servidor se asocia al **puerto 0**: el sistema operativo elige un puerto libre, de modo que las pruebas no chocan con otros programas ni entre sí. La dirección real se obtiene después con `server_address`.
- Los clientes de prueba usan siempre un *timeout*, para que un servidor que no responde haga fallar la prueba en lugar de bloquearla.

```
# eco.py
import socketserver

class ManejadorEco(socketserver.StreamRequestHandler):
    def handle(self) -> None:
        for linea in self.rfile:
            self.wfile.write(linea.upper())

class ServidorEco(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True
```

```
# tests/test_eco.py
import socket
import threading

import pytest
from eco import ManejadorEco, ServidorEco


@pytest.fixture
def servidor():
    """Arranca el servidor en un puerto libre y lo detiene al terminar."""
    srv = ServidorEco(("127.0.0.1", 0), ManejadorEco)    # puerto 0: lo elige el SO
    hilo = threading.Thread(target=srv.serve_forever, daemon=True)
    hilo.start()
    yield srv.server_address                              # ('127.0.0.1', 54321)
    srv.shutdown()
    srv.server_close()


def test_eco_en_mayusculas(servidor):
    with socket.create_connection(servidor, timeout=2) as s:
        s.sendall(b"hola\n")
```

```
        assert s.makefile("rb").readline() == b"HOLA\n"


def test_varios_clientes_a_la_vez(servidor):
    clientes = [socket.create_connection(servidor, timeout=2) for _ in range(5)]
    try:
        for i, c in enumerate(clientes):
            c.sendall(f"cliente {i}\n".encode())
        for i, c in enumerate(clientes):
            assert c.makefile("rb").readline() == f"CLIENTE {i}\n".encode()
    finally:
        for c in clientes:
            c.close()
```

```
$ pytest -v
tests/test_contador.py::test_incrementos_concurrentes PASSED             [ 33%]
tests/test_eco.py::test_eco_en_mayusculas PASSED                         [ 66%]
tests/test_eco.py::test_varios_clientes_a_la_vez PASSED                  [100%]
============================== 3 passed in 1.08s ===============================
```

> **Servicios web.** Para los servicios de FastAPI no hace falta ni siquiera arrancar un servidor: su cliente de pruebas, `TestClient`, llama a la aplicación directamente. Se explica en el Tema 4, junto con la forma de sustituir la base de datos por una de pruebas.

B.6 Documentar el código

La documentación tiene dos destinatarios: quien **usa** el código (necesita saber qué hace cada función, qué recibe y qué devuelve) y quien lo **mantiene** (necesita entender por qué está escrito así). Python ofrece herramientas distintas para cada caso.

B.6.1 Comentarios y *docstrings*

- Los **comentarios** (`# ...`) explican el *porqué* de un fragmento concreto de código para quien lo lee. No deben repetir lo que el código ya dice (`i += 1  # suma 1 a i` no aporta nada).
- Las ***docstrings*** son cadenas entre triples comillas situadas en la **primera línea** de un módulo, clase o función. Describen *qué* hace y *cómo* se usa. A diferencia de los comentarios, forman parte del programa: las muestran los editores al pasar el ratón, la función `help()` y las herramientas de documentación.

Las convenciones para escribirlas están en la PEP 257 (`https://peps.python.org/pep-0257/`): una primera línea con un resumen breve terminado en punto, una línea en blanco y, si hace falta, una descripción más detallada. Para describir parámetros, valores devueltos y excepciones se usa un formato estructurado; el más extendido es el **estilo Google**:

```
def media(notas: list[float]) -> float:
    """Calcula la media aritmética de una lista de notas.

    Args:
        notas: Notas entre 0 y 10. No puede estar vacía.
```

```
    Returns:
        La media, redondeada a dos decimales.

    Raises:
        ValueError: Si la lista está vacía o alguna nota está fuera de rango.
    """
```

Las anotaciones de tipo complementan a la *docstring*: el tipo de cada parámetro ya está en la firma, así que la documentación se centra en su significado y sus restricciones.

B.6.2 Documentar un *script*

Un *script* (un programa que se ejecuta directamente, como las utilidades de línea de comandos de los ejercicios) debería incluir:

1. Una ***docstring* de módulo** al principio del fichero, con qué hace el programa, cómo se usa (con ejemplos de llamada), qué ficheros o datos necesita y qué produce.
2. *Docstrings* en todas las funciones y clases públicas.
3. Las importaciones agrupadas al principio (primero la biblioteca estándar, luego las de terceros y por último los módulos propios) y las constantes en mayúsculas.
4. Una función `main()` y el bloque `if __name__ == "__main__":`, para que el fichero se pueda importar (por ejemplo, desde las pruebas) sin ejecutar el programa.
5. Ayuda en la línea de comandos con `argparse`, que se puede generar a partir de la propia *docstring* del módulo.

```
"""Calcula las medias de las notas de los alumnos a partir de un fichero JSON.

El fichero de entrada debe contener un objeto cuyas claves son los nombres
de los alumnos y cuyos valores son listas de notas entre 0 y 10:

    {"Ana": [7, 8, 9], "Luis": [4, 5]}

Uso:
    python medias.py notas.json
    python medias.py notas.json --minimo 5      # sólo los aprobados

Autor: 2.º DAM - IES Ejemplo
"""
import argparse
import json
import sys
from pathlib import Path

from notas import calificacion, media

NOTA_APROBADO = 5.0


def cargar_notas(ruta: Path) -> dict[str, list[float]]:
    """Lee el fichero JSON de notas y devuelve un diccionario alumno -> notas."""
    with open(ruta, encoding="utf-8") as f:
        return json.load(f)
```

```
def main(argv: list[str] | None = None) -> int:
    """Punto de entrada del programa. Devuelve el código de salida."""
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.
    ↪ RawDescriptionHelpFormatter)
    parser.add_argument("fichero", type=Path, help="fichero JSON con las notas")
    parser.add_argument("--minimo", type=float, default=0.0,
                        help="muestra sólo los alumnos con media >= MINIMO")
    args = parser.parse_args(argv)

    try:
        notas = cargar_notas(args.fichero)
    except (OSError, json.JSONDecodeError) as e:
        print(f"No se pudo leer {args.fichero}: {e}", file=sys.stderr)
        return 1

    for alumno, lista in sorted(notas.items()):
        m = media(lista)
        if m >= args.minimo:
            print(f"{alumno:<10} {m:5.2f}  {calificacion(m)}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

B.6.3 Consultar y generar la documentación

- En el intérprete, `help(media)` o `help(notas)` muestran las *docstrings*. Desde la terminal, `python -m pydoc notas` hace lo mismo.
- `pdoc` genera a partir de las *docstrings* y las anotaciones de tipo una documentación HTML navegable:

```
$ pip install pdoc
$ pdoc notas.py -o docs/
```

- Todo proyecto debería tener un fichero `README.md` en la carpeta raíz que explique qué hace, cómo se instala (dependencias, `requirements.txt`), cómo se ejecuta y cómo se lanzan las pruebas.

B.6.4 *Doctests*: ejemplos que también son pruebas

Los ejemplos de uso escritos en una *docstring* con el formato del intérprete (`>>>`) se pueden ejecutar como pruebas con el módulo `doctest` o con `pytest --doctest-modules`. Así se garantiza que los ejemplos de la documentación siguen siendo correctos:

```
def media(notas: list[float]) -> float:
    """Calcula la media aritmética de una lista de notas.

    >>> media([5, 6])
    5.5
    >>> media([7, 8, 9])
    8.0
```

```
    """
```

```
$ pytest --doctest-modules
```

B.7 Guía de estilo

La PEP 8 (`https://peps.python.org/pep-0008/`) es la guía de estilo oficial de Python: sangría de cuatro espacios, nombres de funciones y variables en `minusculas_con_guiones`, clases en `CamelCase`, constantes en `MAYUSCULAS`, líneas de longitud moderada, etc. Herramientas como `ruff` comprueban el estilo y detectan errores habituales automáticamente (`ruff check .`) e incluso dan formato al código (`ruff format .`).

B.8 Ejercicios

Ejercicio B.1 — Pruebas de funciones

Dificultad: ★

Escribe pruebas con `pytest` para la función `validar_dni` del Tema 0: varios DNI correctos (parametrizados), DNI con la letra equivocada, con menos dígitos, con letras en lugar de números y en minúsculas. Comprueba con `pytest-cov` que se ejecutan todas las líneas de la función.

Ejercicio B.2 — Documentar un programa

Dificultad: ★★

Toma uno de los programas de línea de comandos que hayas hecho en el curso y documéntalo siguiendo este anexo: *docstring* de módulo con ejemplos de uso, *docstrings* en estilo Google en todas las funciones, `README.md` y ayuda de `argparse` generada a partir de la *docstring*. Genera la documentación HTML con `pdoc` y comprueba el estilo con `ruff`.

Ejercicio B.3 — Pruebas con ficheros

Dificultad: ★★

Escribe una *fixture* que cree en `tmp_path` un fichero CSV de alumnos de prueba y úsala en varias pruebas de una función `importar_csv(ruta) -> list[Alumno]`: fichero correcto, fichero vacío, fila con un campo de más, nota no numérica y fichero inexistente.

Ejercicio B.4 — Pruebas de un servidor

Dificultad: ★★★

Escribe pruebas con `pytest` para el servidor de la calculadora del Tema 3: una *fixture* que lo arranque en un hilo en el puerto 0, una prueba parametrizada para cada orden del protocolo (incluidas las órdenes incorrectas y los argumentos no numéricos), una prueba que compruebe que `STATS` cuenta bien las peticiones de 10 clientes simultáneos y otra que compruebe que el servidor sigue funcionando después de que un cliente cierre la conexión a mitad de un mensaje. Para que el servidor pueda probarse así, tendrás que separar la creación del *socket* de escucha del bucle principal.

APÉNDICE C

Bibliografía y recursos

Este anexo reúne la documentación oficial, las especificaciones y los libros en los que se apoyan los apuntes, organizados por temas. Además, a lo largo del documento, cada vez que se menciona una biblioteca o un módulo de Python, su nombre aparece en este color y enlaza directamente con su documentación.

> **Sobre las versiones.** La documentación en línea corresponde normalmente a la última versión publicada. Si algo no funciona como se describe, comprueba qué versión tienes instalada (`python -VV`, `pip show nombre`) y consulta la documentación de esa versión.

C.1 Documentación general

- Documentación oficial de Python (en español): `https://docs.python.org/es/3/`
- Tutorial oficial de Python (en español): `https://docs.python.org/es/3/tutorial/`
- Índice de la biblioteca estándar: `https://docs.python.org/es/3/library/`
- PEP 8, guía de estilo para el código Python: `https://peps.python.org/pep-0008/`
- Guía de empaquetado de Python (*Python Packaging User Guide*): `https://packaging.python.org/en/latest/`
- uv: `https://docs.astral.sh/uv/`
- Documentación de Docker: `https://docs.docker.com/`

C.2 Tema 0: Repaso de Python

- Python sin GIL, guía oficial (*Python support for free threading*): `https://docs.python.org/3/howto/free-threading-python.html`
- Guía de la comunidad sobre Python sin GIL (instalación y compatibilidad de bibliotecas): `https://py-free-threading.github.io/`
- Python en Windows (Python Install Manager): `https://docs.python.org/es/3/using/windows.html`
- Anotaciones de tipo: `https://docs.python.org/es/3/library/typing.html` y `https://typing.python.org/`
- mypy: `https://mypy.readthedocs.io/`
- Dataclasses: `https://docs.python.org/es/3/library/dataclasses.html`
- Pydantic: `https://docs.pydantic.dev/`
- Fechas y zonas horarias: `https://docs.python.org/es/3/library/datetime.html`, `https://docs.python.org/es/3/library/zoneinfo.html`

- argparse: `https://docs.python.org/es/3/library/argparse.html` y su tutorial, `https://docs.python.org/es/3/howto/argparse.html`
- logging: `https://docs.python.org/es/3/library/logging.html`; *Logging HOWTO*: `https://docs.python.org/es/3/howto/logging.html`; *Logging Cookbook*: `https://docs.python.org/es/3/howto/logging-cookbook.html`
- pdb, el depurador de Python: `https://docs.python.org/es/3/library/pdb.html`

C.3 Tema 1: Programación multiproceso

- subprocess: `https://docs.python.org/es/3/library/subprocess.html`
- multiprocessing: `https://docs.python.org/es/3/library/multiprocessing.html`
- concurrent.futures: `https://docs.python.org/es/3/library/concurrent.futures.html`
- os: `https://docs.python.org/es/3/library/os.html`
- psutil: `https://psutil.readthedocs.io/`
- G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities". *AFIPS Spring Joint Computer Conference*, 1967.

C.4 Tema 2: Programación multihilo

- threading: `https://docs.python.org/es/3/library/threading.html`
- queue: `https://docs.python.org/es/3/library/queue.html`
- faulthandler: `https://docs.python.org/es/3/library/faulthandler.html`
- tkinter (proyecto del tema): `https://docs.python.org/es/3/library/tkinter.html`
- E. G. Coffman, M. Elphick y A. Shoshani, "System Deadlocks". *ACM Computing Surveys*, 3(2), 1971.

C.5 Tema 3: Programación de comunicaciones en red

- socket: `https://docs.python.org/es/3/library/socket.html`
- *Socket Programming HOWTO*: `https://docs.python.org/es/3/howto/sockets.html`
- socketserver: `https://docs.python.org/es/3/library/socketserver.html`
- selectors: `https://docs.python.org/es/3/library/selectors.html`
- struct: `https://docs.python.org/es/3/library/struct.html`
- asyncio, *Streams*: `https://docs.python.org/es/3/library/asyncio-stream.html`
- Wireshark, *User's Guide*: `https://www.wireshark.org/docs/`

C.6 Tema 4: Generación de servicios en red

- http.server: `https://docs.python.org/es/3/library/http.server.html`

- urllib.request: `https://docs.python.org/es/3/library/urllib.request.html`
- smtplib y email: `https://docs.python.org/es/3/library/smtplib.html`, `https://docs.python.org/es/3/library/email.html`
- Requests: `https://requests.readthedocs.io/`
- FastAPI: `https://fastapi.tiangolo.com/es/`
- SQLModel: `https://sqlmodel.tiangolo.com/`
- Uvicorn: `https://www.uvicorn.org/`
- Locust (pruebas de carga): `https://docs.locust.io/`
- Mailpit (servidor de correo de pruebas): `https://mailpit.axllent.org/`
- Mozilla, *HTTP* (MDN Web Docs): `https://developer.mozilla.org/es/docs/Web/HTTP`
- Zalando, *RESTful API Guidelines*: `https://opensource.zalando.com/restful-api-guidelines/`
- Microsoft, *REST API Guidelines*: `https://github.com/microsoft/api-guidelines`
- R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. Tesis doctoral, University of California, Irvine, 2000. Capítulo 5: REST. `https://ics.uci.edu/~fielding/pubs/dissertation/top.htm`

C.7 Tema 5: Técnicas de programación segura

- hashlib, hmac y secrets: `https://docs.python.org/es/3/library/hashlib.html`, `https://docs.python.org/es/3/library/hmac.html`, `https://docs.python.org/es/3/library/secrets.html`
- ssl: `https://docs.python.org/es/3/library/ssl.html`
- cryptography: `https://cryptography.io/`
- bcrypt: `https://pypi.org/project/bcrypt/`
- PyJWT: `https://pyjwt.readthedocs.io/`
- Bandit: `https://bandit.readthedocs.io/`
- pip-audit: `https://pypi.org/project/pip-audit/`
- OWASP Top 10: `https://owasp.org/www-project-top-ten/`
- OWASP Cheat Sheet Series (en especial *Password Storage*, *Authentication* y *REST Security*): `https://cheatsheetseries.owasp.org/`
- Let's Encrypt: `https://letsencrypt.org/docs/`

C.8 Anexo B: pruebas y documentación

- pytest: `https://docs.pytest.org/`
- pytest-cov: `https://pytest-cov.readthedocs.io/`
- doctest: `https://docs.python.org/es/3/library/doctest.html`
- PEP 257, convenciones para las *docstrings*: `https://peps.python.org/pep-0257/`

- Guía de estilo de Google para Python (formato de las *docstrings*): `https://google.github.io/styleguide/pyguide.html`
- pdoc: `https://pdoc.dev/`
- Ruff: `https://docs.astral.sh/ruff/`

C.9 Especificaciones y estándares

- PEP 703, *Making the Global Interpreter Lock Optional in CPython*: `https://peps.python.org/pep-0703/`
- PEP 779, *Criteria for supported status for free-threaded Python*: `https://peps.python.org/pep-0779/`
- PEP 649, *Deferred Evaluation Of Annotations Using Descriptors*: `https://peps.python.org/pep-0649/`
- RFC 9110, *HTTP Semantics*: `https://www.rfc-editor.org/rfc/rfc9110`
- RFC 9457, *Problem Details for HTTP APIs*: `https://www.rfc-editor.org/rfc/rfc9457`
- RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*: `https://www.rfc-editor.org/rfc/rfc8259`
- OpenAPI Specification: `https://spec.openapis.org/oas/latest.html`
- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*: `https://www.rfc-editor.org/rfc/rfc8446`
- RFC 7519, *JSON Web Token (JWT)*: `https://www.rfc-editor.org/rfc/rfc7519`
- NIST, FIPS 203, *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (criptografía poscuántica): `https://csrc.nist.gov/pubs/fips/203/final`

C.10 Libros

- L. Ramalho, *Fluent Python*, 2.ª ed. O'Reilly Media, 2022.
- A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Wiley, 2018 (procesos, planificación de la CPU y sincronización).
- A. B. Downey, *The Little Book of Semaphores*. Green Tea Press. Libre en `https://greenteapress.com/wp/semaphores/`
- M. Fowler, *Python Concurrency with asyncio*. Manning, 2022.
- J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 8.ª ed. Pearson, 2021.
- J. J. Geewax, *API Design Patterns*. Manning, 2021.
- B. Lubanovic, *FastAPI: Modern Python Web Development*. O'Reilly Media, 2023.
- J.-P. Aumasson, *Serious Cryptography*, 2.ª ed. No Starch Press, 2024.
- B. Okken, *Python Testing with pytest*, 2.ª ed. The Pragmatic Bookshelf, 2022.

C.11 Sitios de práctica

- APIs públicas gratuitas para practicar clientes HTTP: Open-Meteo (`https://open-meteo.com`), JSONPlaceholder (`https://jsonplaceholder.typicode.com/`).
- Servicio de pruebas de peticiones HTTP: `https://httpbin.org/`
- Comprobación de la configuración TLS de un servidor: `https://www.ssllabs.com/ssltest/`
- Portal de datos abiertos de la Junta de Castilla y León: `https://datosabiertos.jcyl.es/`